

Faculty of Engineering
Department of Informatics Engineering
Software and Information System Engineering



AI Store Generator

(SaaS Multi-Tenant E-Commerce Platform)

A senior 1 project report - submitted to complete the requirements for obtaining a bachelor's degree in informatics engineering - Software Engineering and Information Systems

Prepared by

Hiba Reman

Omar Masoud

Remaz Qubaia

Supervised by

Dr.Kadan Juma

Eng. Mohamed Al Balkhi

2025-2026

AI Store Generator (SaaS Multi-Tenant E-Commerce Platform)

مشروع (تخرج 1) - قدم استكمالاً لمتطلبات الحصول على درجة البكالوريوس في هندسة المعلوماتية - هندسة
البرمجيات ونظم المعلومات الذكية

إعداد

هبة ريمان عمر مسعود ريماز قبيعة

إشراف

د. كادان جمعة م. محمد البلخي

ABSTRACT

ملخص

تُعد التجارة الإلكترونية من المجالات التقنية والتجارية التي شهدت نمواً كبيراً في السنوات الأخيرة، حيث أصبحت المتاجر الإلكترونية وسيلة أساسية لعرض المنتجات والوصول إلى العملاء بسرعة ومرونة. ومع هذا التوسع، ظهرت حاجة واضحة إلى أنظمة تساعد أصحاب المشاريع على إنشاء متاجرهم الإلكترونية بسهولة، دون الحاجة إلى خبرة تقنية كبيرة أو وقت طويل في الإعداد والتجهيز.

يأتي هذا المشروع ضمن مجال أنظمة التجارة الإلكترونية المدعومة بالذكاء الاصطناعي، حيث يهدف إلى بناء نظام يساعد المستخدم على إنشاء متجر إلكتروني مبدئي من خلال إدخال وصف بسيط لفكرة المتجر. يقوم النظام بالاعتماد على الذكاء الاصطناعي لتحليل هذا الوصف واقتراح بيانات أساسية مثل اسم المتجر، وصفه، المنتجات الأولية، والفئات المناسبة، مما يجعل عملية إنشاء المتجر أسرع وأسهل.

من الناحية التجارية، يقدم المشروع قيمة مهمة لأصحاب الأعمال والمشاريع الصغيرة، لأنه يقلل الجهد والتكلفة المطلوبة لبدء متجر إلكتروني، ويساعد المستخدمين على تحويل أفكارهم التجارية إلى متجر قابل للإدارة والتطوير خلال وقت قصير. كما يدعم المشروع سرعة دخول السوق ويمنح أصحاب المشاريع فرصة للتركيز على تطوير منتجاتهم وخدمة عملائهم بدلاً من الانشغال بتفاصيل الإعداد الأولية.

Table of content

Chapter1: Introduction	13
1.0 Introduction.....	14
1.1 Project Introduction.....	14
1.2 Problem Statement and Practical / Academic Importance.....	14
1.3 Project Objectives	14
1.4 Project Scope, Constraints, and Assumptions	15
1.4.1 Project Scope	15
1.4.2 Out of Scope.....	16
1.4.3 Constraints	16
1.4.4 Assumptions.....	16
1.5 Project Contribution to Knowledge and the Field	17
1.6 Research Methodology and Scientific Methods Used	17
1.7 Organizational Structure of the Document	18
1.8 Basic Definitions and Key Concepts.....	19
Chapter 2: Project Development and Management Methodology	20
2.1 Introduction:	21
2.2 Development Methodology:	21
2.3 Project Schedule (Gantt Chart):.....	21
2.4 Risk Management:.....	23
2.5 Project Charter:.....	24
2.6 The SOW document:	25
2.7 Product Quality Assurance:	26
2.8 Project Management Tools Used	27
2.9 Monitoring and Continuous Evaluation Mechanisms.....	28
2.10 Software Configuration Management (SCM).....	28
Chapter3: Literature Review and Related Studies.....	31
3.0 Introduction.....	32
3.1 Previous Studies in Intelligent and Similar Systems	32
3.2 Modern Technologies and Frameworks in the Field.....	33

3.3 Comparative Analysis of Existing Systems and Products.....	35
3.4 Research Gap and Proposed Innovation.....	36
3.5 Programming Framework and Reference Foundations	36
3.6 Modern and Future Trends in the Field.....	37
Chapter4: System analysis.....	38
4.1 Feasibility Study	39
4.2 Requirements Elicitation	39
4.3 Stakeholders Identification	40
4.4 Functional Requirements.....	41
4.4.1 Account Management and Authorization.....	41
4.4.2 Store Management and Multi-Tenant Architecture	41
4.4.3 Category and Product Management.....	41
4.4.4 Order Management	42
4.4.5 AI-Assisted Store Creation	42
4.4.6 Theme System and Appearance Customization.....	42
4.4.7 Store Visitor Interface.....	42
4.4.8 Super Admin Dashboard.....	43
4.4.9 Mock Billing and Basic Support.....	43
4.5 Non-Functional Requirement.....	43
4.6 Requirements Prioritization and Management Mechanism	44
4.7 Requirements Modeling.....	48
4.7.1 AI-Assisted Store Creation.....	48
4.7.2 Tenant Data Isolation.....	52
4.7.3 Store Management.....	54
4.7.4 Product Management.....	57
4.7.5 Category Management	60
4.7.6 Order Management	62
4.7.7 Customer Store Browsing	65
4.8 initial test cases Definition.....	67
SCE-01:AI-Assisted Store Creation.....	67
SCE-02: Tenant Data Isolation.....	68

SCE-03: Product Management	68
SCE-04: Category Management	69
SCE-05: Store Management	70
SCE-06: Order Management.....	71
SCE-07: Customer Store Browsing.....	72
4.9 RTM – version 01.....	74
Chapter5: System Design.....	79
5.1 High-Level Design.....	80
5.1.0 Introduction.....	80
5.1.1 System Block Diagram.....	80
5.1.2 System Architecture.....	80
5.1.3 System Architecture Design.....	81
5.2 AI Store Creation Pipeline Design	84
5.2.1 AI Pipeline Overview	84
5.2.2 Store Description Submission	84
5.2.3 Input Assessment by the AI.....	85
5.2.4 Clarification Session (MCQs)	85
5.2.5 Store Draft Generation and Interactive Presentation	85
5.2.6 Validation (Mandatory).....	86
5.2.7 User Confirmation and Store Creation.....	87
5.2.8 Fallback Handling.....	88
5.2.9 AI Pipeline Design Alternatives	88
5.2.10 Architectural Evaluation Criteria.....	89
5.2.11 Design Decision for AI.....	90
5.3 Database Design	92
5.3.1 ER Diagram.....	92
5.3.2 Relational Schema	92
5.3.3 Database Constraints and Keys.....	95
5.3.4 Multi-Tenant Data Isolation.....	95
5.3.5 AI Generated Store Data Structure.....	96
5.3.6 Database Design Summary	97

5.4 API Design	97
5.4.1 API Design Overview	97
5.4.2 API Routing Structure	97
5.4.3 API Endpoints	98
5.5 Security and Permissions Design	102
5.5.1 Security Design Overview	102
5.5.2 Authentication Mechanism.....	102
5.5.3 User Roles and Permissions.....	103
5.5.4 Permission Enforcement	103
5.5.5 Tenant-Based Data Isolation.....	103
5.5.6 Error Handling for Security.....	103
Chapter6: Implementation Environment and Tools.....	104
6.1 Software Tools and Libraries Used.....	105
6.2 Development Environment and Project Structure	105
6.3 Programming Languages and Frameworks.....	105
6.4 Database and Operating Environment.....	106
6.5 Dependencies and Packages.....	106
6.6 Version Control.....	107
6.7 CI/CD Pipeline.....	107
6.8 Testing Environment.....	107
Chapter7: Software Implementation.....	109
7.1 Project Files and Structure.....	110
7.1.1 Backend Structure.....	110
7.1.2 Frontend Structure	110
7.2 Modules Implementation.....	111
7.3 APIs Implementation	112
7.4 Data Access Layer Implementation	112
7.5 Business Logic Layer Implementation.....	113
7.6 UI Implementation	115
7.7 Intelligent Algorithms Implementation.....	115

Chapter8: Testing and Evaluation.....	117
8.1 Test Plan.....	118
8.2 Types of Testing.....	118
8.3 Test Case Summary	119
8.4 Test Results and Analysis	123
8.5 Quality Metrics.....	123
8.2 Final Requirements Traceability Matrix (RTM Version 2).....	124
Chapter9: Results and Analysis.....	130
9.1 System Implementation Results.....	131
9.2 Comparative Analysis with Existing Systems.....	131
9.3 Achievement of Objectives.....	132
9.4 Strengths and Weaknesses Analysis	133
9.5 Key Performance Indicators (KPIs)	134
Chapter10: Conclusion and Recommendations.....	135
10.1 Project Summary and Main Achievements	136
10.2 Challenges Faced During the Project	136
10.3 Future Development Suggestions	137
10.4 Recommendations.....	137

List of tables

Table 2.1 Risk Management.....	23
Table 2.2 Project Charter.....	25
Table 2.3 Statement Of Work.....	26
Table 3.1 Similar Systems	33
Table 3.2 Technologies & Frameworks	34
Table 3.3 Comparative between Existing Systems And Products.....	35
Table 3.4 Modern & Future Trends In The Field	37
Table 4.1 Summary Of Requirements.....	39
Table 4.2 Stakeholders Identification	40
Table 4.3 Non-Functional Requirements.....	44
Table 4.4 Requirements Prioritization	47
Table 4.5 AI-Assisted Store Creation Use case specifications.....	50
Table 4.6 Tenant Data Isolation Use case specifications.....	53
Table 4.7 Store Management Use case specifications	56
Table 4.8 Product Management Use case specifications	58
Table 4.9 Category Management Use case specifications	61
Table 4.10 Order Management Use case specifications	63
Table 4.11 Customer Store Browsing Use case specifications	65
Table 4.12 AI-Assisted Store Creation initial TC.....	67
Table 4.13 Tenant Data Isolation initial TC	68
Table 4.14 Product Management initial TC.....	69
Table 4.15 Category Management initial TC	70
Table 4.16 Store Management initial TC	71
Table 4.17 Order Management initial TC	71
Table 4.18 Customer Store Browsing initial TC	73
Table 4.19 RTM – version 01.....	78

\

Table Of Figures

Figure 2.1 Gantt Chart 1.....	22
Figure 2. 2 Gantt Chart 2	22
Figure 2.3 Git Repository Structure	28
Figure 2. 4 Branching Strategy.....	29
Figure 2.5 Git Branches.....	29
Figure 2.6 Merge Operations 1	30
Figure 2.7 Merge Operations 2	30
Figure 4.1 AI-Assisted Store Creation Use Case Diagram.....	48
Figure 4.2 AI-Assisted Store Creation Use Case Diagram.....	48
Figure 4. 3 AI-Assisted Store Creation Activity Diagram	50
Figure 4.4 AI-Assisted Store Creation Sequence Diagram	51
Figure 4.5 Tenant Data Isolation Use Case Diagram	52
Figure 4.6 Tenant Data Isolation Use Case Diagram	52
Figure 4.7 Tenant Data Isolation Activity Diagram.....	53
Figure 4.8 Tenant Data Isolation Sequence Diagram	54
Figure 4.9 Store Management Use Case Diagram.....	54
Figure 4.10 Store Management Sequence Diagram.....	56
Figure 4.11 Product Management Use Case Diagram.....	57
Figure 4.12 Product Management Activity Diagram.....	58
Figure 4.13 Product Management Sequence Diagram.....	59
Figure 4.14 Category Management Use Case Diagram.....	60
Figure 4.15 Category Management Sequence Diagram.....	62
Figure 4.16 Order Management Use Case Diagram.....	62
Figure 4.17 Order Management Activity Diagram	64
Figure 4.18 Order Management Sequence Diagram	64
Figure 4.19 Customer Store Browsing Activity Diagram	66

Chapter 1: Introduction

1.0 Introduction

In this chapter, we introduce the project and provide a general overview of its background, problem statement, and importance. We also define the project objectives, scope, constraints, assumptions, contribution to the field, research methodology, report organization, and the main concepts used throughout the document.

1.1 Project Introduction

This project belongs to the field of AI-assisted e-commerce, aiming to make online store creation easier for business owners without requiring advanced technical knowledge. The system allows the user to enter a simple description of the store idea, then analyzes it and suggests basic data such as the store name, description, initial products, and suitable categories. From a business perspective, the project helps reduce the time, effort, and cost required to start an online store. It also gives business owners a faster way to enter the market and focus on improving their products and serving their customers.

1.2 Problem Statement and Practical / Academic Importance

The main problem is that creating an online store usually requires time, effort, and some technical or design knowledge, especially when defining the store idea, organizing products, selecting categories, and preparing the initial store data. This can be a challenge for small business owners or non-technical users who want to enter the e-commerce market quickly.

Solving this problem is practically important because it makes digital business creation easier and reduces the time and cost required to launch an online store. Academically, the project is important because it applies AI in e-commerce in a practical way, showing how intelligent systems can support users in making initial decisions and generating organized content that can be managed and improved later.

1.3 Project Objectives

The main objective of this project is to build a **Multi-Tenant SaaS system** for creating and managing e-commerce stores, powered by artificial intelligence. The system enables business owners to create an

initial online store based on a simple textual description, and then manage store data, products, and orders through one easy-to-use platform.

To achieve this objective, the system aims to:

- Enable users to create an initial online store draft by entering a simple description of the store idea.
- Use AI to generate basic store data, such as the store name, store description, product categories, initial products, and a suitable template.
- Support clarification questions when the user's description is insufficient, in order to improve the quality of the generated data before creating the store.
- Allow users to review, edit, regenerate, or approve the generated draft before saving the store permanently.
- Validate store and product data before saving it, to ensure that the required fields are complete and the data is correct.
- Support a **Multi-Tenant** architecture, ensuring that each store and its owner's data are isolated from other stores and users.
- Provide basic store management features, including product management, category management, store settings, and order management.
- Enable visitors to browse the published store, view products, and use a simple mock cart.
- Achieve a suitable response time for the initial AI generation process, approximately **3 to 5 seconds** under normal conditions.
- Provide a simple user interface that helps non-technical business owners enter the e-commerce market quickly, while reducing the time, effort, and cost required to create an online store.

1.4 Project Scope, Constraints, and Assumptions

1.4.1 Project Scope

This project includes the development of a web-based platform called **Souq Engine** as a **Multi-Tenant SaaS** system for creating and managing e-commerce stores. At this stage, the project focuses on building the core foundation of the platform, allowing users to create an online store, manage

store data, products, categories, orders, and customers, while ensuring that each store's data is isolated from other stores.

The project also includes a core AI-based feature, which is generating an initial online store draft from a textual description provided by the user. The system analyzes the description and suggests store settings, categories, sample products, and a suitable theme. In addition, the project provides basic store appearance customization, a basic store owner dashboard, a Super Admin dashboard, a mock billing system, and basic support for Arabic and English.

1.4.2 Out of Scope

In its current stage, this project does not include advanced features such as AI-based product image processing, generating product descriptions from images, or using advanced AI agents to execute direct commands inside the system. The project also does not include real payment gateway integration or actual payment processing, as it will rely on a mock billing system or a sandbox payment environment.

1.4.3 Constraints

This project is developed under specific constraints that should be considered when evaluating it. One of the main constraints is the limited time available within the **Senior 1** phase, which makes the main focus on building a stable system core rather than implementing all advanced features. The project also has a limited budget, so it relies as much as possible on free or trial-based tools and services.

In addition, the project depends on an external AI service, which may often be costly. Due to the limited budget, free or experimental AI solutions will be used whenever possible. This means that response time or service availability may be affected by factors outside the system's control. Real payment integration and advanced features have also been postponed to later stages in order to reduce technical complexity and implementation risks.

1.4.4 Assumptions

The project assumes that users have basic knowledge of using web applications and computers, and that the store owner is able to enter a simple textual description explaining the store idea. It also

assumes that each store is associated with a specific owner inside the system, and that an internet connection is available to use the platform and AI services.

The project also assumes that the goal of this stage is to build a stable and extendable prototype, not a complete commercial product with all advanced features. Therefore, features such as real payment processing, AI image processing, and advanced AI agents are considered outside the scope of this stage and may be considered in future development.

1.5 Project Contribution to Knowledge and the Field

This project provides a different application in the field of e-commerce by combining the **Multi-Tenant SaaS** model with AI to create online stores from a simple textual description. It does not only focus on traditional store management, but also adds an intelligent step that helps the store owner generate initial data such as store information, categories, sample products, and a suitable theme.

The contribution of this project is to present a practical model showing how AI can reduce the time and effort required in the store setup phase, while maintaining a suitable architecture for isolating store data within one platform. Therefore, the project combines e-commerce, Software as a Service, and AI in an extendable applied case.

1.6 Research Methodology and Scientific Methods Used

This project follows the **descriptive analytical approach** and the **applied engineering approach**. The descriptive analytical approach is used to describe the problem of creating e-commerce stores and analyze the need for a system that helps business owners create their stores faster and more easily. It is also used to study the system requirements and identify the main functions needed by users.

The applied engineering approach is used because the project aims to design and build a real working system, not only present a theoretical study. Therefore, technical knowledge is applied to develop a multi-tenant web platform supported by artificial intelligence, and the system is evaluated by verifying its ability to perform the required core functions.

1.7 Organizational Structure of the Document

The report will be organized according to the following structure:

Part 01: Project Introduction

Chapter 01: General Introduction

Chapter 02: Project Development and Management Methodology

Part 02: Analysis and Previous Studies

Chapter 03: Literature Review and Related Studies

Chapter 04: Requirements Analysis and Modeling

Part 03: System Design

Chapter 05: System Architectural Design

Part 04: Implementation and Testing

Chapter 06: Implementation Environment and Tools Used

Chapter 07: Software Implementation

Chapter 08: Testing and Evaluation

Part 05: Results and Conclusion

Chapter 09: Results and Analysis

Chapter 10: Conclusion and Recommendations

Part 06: Appendices and References

Appendices

References

1.8 Basic Definitions and Key Concepts

- **Artificial Intelligence (AI):** A technology that enables the system to analyze inputs, make decisions, or generate results that support the user intelligently.
- **Large Language Models (LLMs):** Advanced AI models capable of understanding text and generating structured textual content based on user instructions.
- **SaaS (Software as a Service):** A software delivery model in which users access the system through the internet without needing to install it locally.
- **Multi-Tenant Architecture:** An architecture that allows multiple users or stores to use the same system while keeping each store's data isolated from others.
- **Tenant:** An independent entity within the system, such as a store or store owner, with its own data and settings.
- **E-Commerce:** The process of buying and selling products or services online through a digital store or platform.
- **Store Creation:** The process of creating a new online store that includes basic information such as name, description, categories, products, and theme.
- **AI-Assisted Store Generation:** A feature that uses artificial intelligence to generate an initial store draft based on a textual description provided by the user.
- **Prompt:** The text or description entered by the user to request a specific result from the AI system.

Chapter 2: Project Development and Management Methodology

2.1 Introduction:

In this chapter, we address the project management phase, which is essential for ensuring project success and organizing the workflow. We will cover the Project Charter, Project Plan, and Statement of Work (SOW), along with stakeholder analysis and risk management. In addition, we will present the selected development methodology, project timeline, team roles, quality management, project management tools, continuous monitoring and evaluation mechanisms, and Software Configuration Management (SCM).

2.2 Development Methodology:

The **Incremental Development Model** was selected for this project, as it allows the system to be developed in a series of smaller, manageable parts (increments), where each part is completed and delivered before moving to the next.

This methodology was chosen for the following reasons:

- **Requirements are not fully clear from the beginning**, which requires flexibility for updates and improvements during development.
- **Gradual delivery of system features**, enabling early user feedback and continuous improvement.
- **Risk reduction**, as issues can be identified and resolved in each phase instead of at the end of the project.
- **More flexibility compared to the Waterfall model**, which requires fixed requirements upfront.
- **Easier project management** by dividing the system into smaller, manageable components.

Therefore, the Incremental model provides a balance between structure and flexibility, making it suitable for the nature of this project.

2.3 Project Schedule (Gantt Chart):

The project schedule was prepared based on the Incremental Development Model, where the work is divided into multiple increments. Each increment includes analysis, design, implementation, and testing tasks. The Gantt chart illustrates the timeline of tasks, including their duration and start and end dates, which helps in organizing the workflow and effectively tracking project progress.

Initial scope determination	4	2/21/2026	2/24/2026
system planning	4	2/25/2026	2/28/2026
requirement defined	6	3/1/2026	3/6/2026
system analysis high level	5	3/7/2026	3/11/2026
system design high level	5	3/12/2026	3/16/2026
increment01-core platform setup-Analysis&Design	4	3/17/2026	3/20/2026
increment01-core platform setup-Implementaion(Backend)	10	3/21/2026	3/30/2026
increment01-core platform setup-Implementaion(Frontend)	10	3/21/2026	3/30/2026
increment01-core platform setup-Testing	1	3/31/2026	3/31/2026
increment02-AI based store creation- Analysis&Design	3	4/1/2026	4/3/2026
increment02-AI based store creation-Implementaion(Backend)	10	4/4/2026	4/13/2026
increment02-AI based store creation-Implementaion(Frontend)	10	4/4/2026	4/13/2026
increment02-AI based store creation-Testing	1	4/14/2026	4/14/2026
increment03-customer storefront&order management-Analysis&Design	3	4/15/2026	4/17/2026
increment03-customer storefront&order management-Implementaion(Backend)	12	4/17/2026	4/28/2026
increment03-customer storefront&order management-Implementaion(Frontend)	12	4/17/2026	4/28/2026
increment03-customer storefront&order management-Testing	1	4/28/2026	4/28/2026
increment04-super admin&Billing&SEO-Analysis&Design	2	4/29/2026	4/30/2026
increment04-super admin&Billing&SEO-Implementaion(Backend)	7	5/1/2026	5/7/2026
increment04-super admin&Billing&SEO-Implementaion(Frontend)	7	5/1/2026	5/7/2026
increment04-super admin&Billing&SEO-Testing	4	5/9/2026	5/9/2026
Documentation	77	2/21/2026	5/9/2026

Figure 2.1 Gantt Chart 1

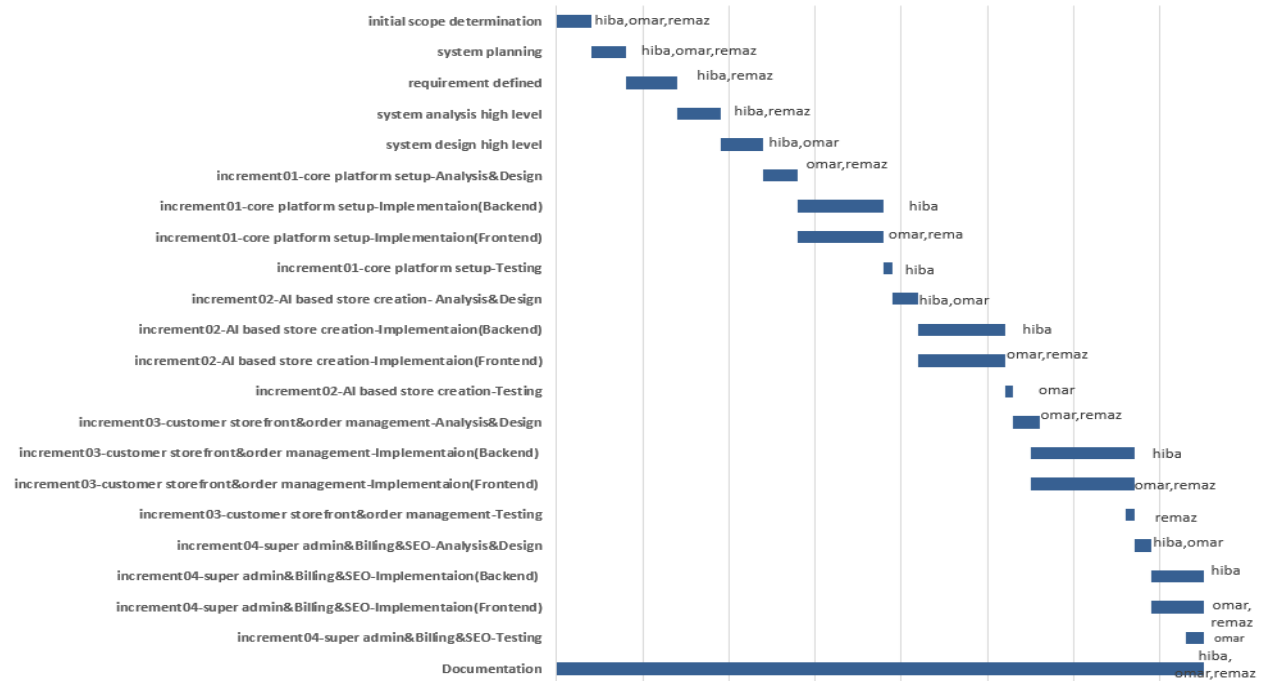
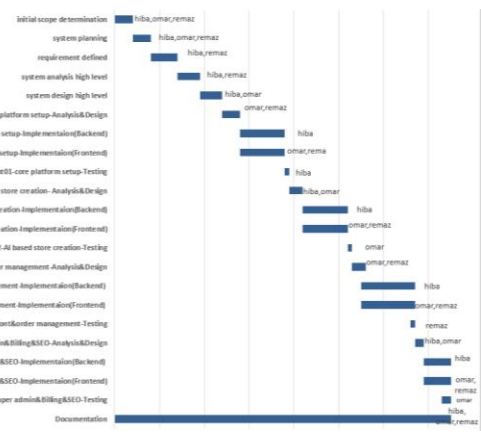


Figure 2. 2 Gantt Chart 2

2.4 Risk Management:

the process of identifying, evaluating, and mitigating potential risks that could impact the success of the project or the team.

Risk_ID	Risk 1	Risk 2	Risk 3	Risk 4	Risk 5
Risk Title	Tight Schedule & Changing Requirements	Environmental Issues (Power/Internet)	Integration Issues Between System Components	Poor Coordination & Time Management	Insufficient Test Coverage
Risk Description	The project duration is relatively short, with possible requirement changes during development affecting the timeline	Power outages or unstable internet may affect team productivity	Difficulty in integrating frontend, backend, or working with APIs	Lack of communication or team member unavailability may delay progress	Bugs may appear after delivery due to inadequate testing
Raised Date	21/02/2026	21/02/2026	01/03/2026	21/02/2026	15/03/2026
Tracking Frequency	Weekly	Daily	Weekly	Weekly	Weekly
State	Active	Active	Active	Active	Active
Impact	High	Medium	High	Medium	High
Mitigation Plan	Divide work into increments, prioritize core features, and formally manage any changes with time estimation	Save work regularly, use backup internet sources, and rely on cloud storage	Document APIs clearly and perform regular integration testing	Hold regular meetings, clearly assign tasks, and track progress continuously	Allocate sufficient time for testing, test each feature after implementation, and define clear test scenarios

Table 2.1 Risk Management

2.5 Project Charter:

A project charter is a formal document that officially authorizes the start of a project. It serves as a reference point throughout the project, providing a clear understanding of the project's purpose and establishing a foundation for decision-making and project governance.

Project title		Multi-Tenant SaaS AI-Creation Store													
Project start date		October 25, 2025													
Project finish date		January 14, 2026													
Project supervisor		Eng. Mohamed Al-Balkhi													
Project objectives		Develop a multi-tenant SaaS e-commerce platform that enables users to create and manage their online stores, while leveraging artificial intelligence to support store content generation such as product descriptions, data organization, and management enhancement, with strong tenant isolation and a secure, scalable system architecture.													
Approach		1. Determine the initial project scope. 2. Plan the system and define the requirements. 3. Perform high-level system analysis and design. 4. Develop the first increment: core platform setup, then test it. 5. Develop the second increment: AI-based store creation, then test it. 6. Develop the third increment: customer storefront and order management, then test it. 7. Develop the fourth increment: super admin, billing, and SEO, then test it. 8. Document all development phases and project outcomes.													
Roles and Responsibilities		<table><tr><th>Name</th><th>Role</th><th>Responsibility</th></tr><tr><td>Eng. Mohamed Al-Balkhi</td><td>Supervisor</td><td>Project supervision, progress monitoring, and technical guidance.</td></tr><tr><td>Hiba</td><td>Software Engineer</td><td>Backend development using Django, implementing authentication, building the multi-tenant architecture, developing APIs, integrating the database, and preparing AI feature integration on the backend side.</td></tr><tr><td>Omar</td><td>Software Engineer</td><td>Frontend development using React, implementing user interfaces, integrating</td></tr></table>		Name	Role	Responsibility	Eng. Mohamed Al-Balkhi	Supervisor	Project supervision, progress monitoring, and technical guidance.	Hiba	Software Engineer	Backend development using Django, implementing authentication, building the multi-tenant architecture, developing APIs, integrating the database, and preparing AI feature integration on the backend side.	Omar	Software Engineer	Frontend development using React, implementing user interfaces, integrating
Name	Role	Responsibility													
Eng. Mohamed Al-Balkhi	Supervisor	Project supervision, progress monitoring, and technical guidance.													
Hiba	Software Engineer	Backend development using Django, implementing authentication, building the multi-tenant architecture, developing APIs, integrating the database, and preparing AI feature integration on the backend side.													
Omar	Software Engineer	Frontend development using React, implementing user interfaces, integrating													

		<i>frontend with backend, and presenting AI-generated outputs within the system.</i>
<i>Rimaz</i>	<i>Software Engineer</i>	<i>Frontend development using React, implementing dashboards, improving user experience, and designing interaction with AI-supported features.</i>

Table 2.2 Project Charter

2.6 The SOW document:

Statement of Work is a comprehensive document that defines the scope of work for a project. It outlines the specific tasks, deliverables, timeline, and responsibilities. The SOW document provides a clear understanding of what needs to be accomplished, the project objectives, and the criteria for success.

Project Title: Multi-Tenant SaaS AI-Creation Store

Project Description and Objectives: The project aims to develop a multi-tenant SaaS e-commerce platform that enables users to create and manage online stores efficiently. The system integrates AI-based features to support store creation, product content generation, and management enhancement, while ensuring tenant isolation, secure access control, and scalable system architecture.

Project Scope: This project involves developing a full-stack software system that supports multi-tenant online store management with AI-assisted features. The platform will provide secure authentication, tenant-based data isolation, store management, category and product management, AI-based store creation support, customer storefront features, order management, billing support, SEO-related features, and a super admin dashboard for monitoring and controlling the system.

Project goals:

- Develop a software system that enables users to create and manage online stores in a multi-tenant SaaS environment.*
- Use AI features to assist in generating and improving store-related content such as product descriptions and store setup.*
- Provide secure tenant isolation and role-based access to ensure safe and scalable platform usage.*
- Build an interactive and user-friendly dashboard for store owners, customers, and administrators.*

Project Deliverables:

- Project charter.*
- Statement of Work document.*
- Software Requirements Specification (SRS).*
- Working software – backend system.*

	• <i>Working software – frontend system.</i> • <i>AI-supported store creation features.</i> • <i>Final project report.</i>
<i>Technology and Tools:</i>	○ <i>Programming Languages: Python, JavaScript, HTML, CSS.</i> ○ <i>Frameworks: Django for backend, React for frontend.</i> ○ <i>Database: PostgreSQL.</i> ○ <i>AI Models / Services: AI APIs for content generation and store creation assistance.</i> ○ <i>Development Tools: Git, GitHub, Postman, VS Code.</i>
<i>Assumptions:</i>	• <i>Continuous availability of project team members.</i> • <i>Regular supervision and feedback from the project supervisor.</i> • <i>Continuous integration and delivery of working software components.</i> • <i>Availability of the required tools, frameworks, and AI service access during development.</i>
<i>Project Resources:</i>	<i>Human Resources:</i> <i>Eng. Mohamed Al-Balkhi – Project Supervisor.</i> <i>Hiba – SE Developer (Backend – Django).</i> <i>Omar – SE Developer (Frontend – React).</i> <i>Rimaz – SE Developer (Frontend – React).</i>
<i>Schedules:</i>	<i>Project Start Date: October 25, 2025</i> <i>Final Project Seminar: December 23, 2025</i> <i>Project Finish Date: January 14, 2026</i>

Table 2.3 Statement Of Work

2.7 Product Quality Assurance:

Product quality will be ensured by applying a set of practices and procedures aimed at verifying that the system is correct, secure, maintainable, and ready for practical use. These procedures include the following:

2.7.1 Code Reviews

Regular code reviews will be conducted to ensure that the code follows the agreed development standards in terms of structure, naming clarity, maintainability, and reusability. Code reviews will also help detect logic issues early, especially in sensitive parts related to tenant isolation, permissions, and secure access control.

2.7.2 Automated Testing

Automated testing will be used to verify that the system functions correctly. This includes:

- **Unit Testing** to validate individual components and business logic.

- **Integration Testing** to verify communication between system modules.
- **API Testing** to ensure that requests and responses behave correctly.
- **Multi-tenant Isolation Testing** to confirm that one tenant cannot access another tenant's data.
- **Frontend and Backend Validation** The integration between the Django backend and the React frontend will be tested to ensure that data flows correctly between both sides and is displayed properly to users without inconsistencies.

2.7.3 Static Code Analysis

Static analysis tools will be used to identify potential issues in the source code, such as formatting problems, inconsistent coding style, duplicated code, and possible implementation mistakes before runtime.

2.7.4 Clear Acceptance Criteria

Each feature will have clear acceptance criteria before it is considered complete. These criteria include:

- The feature works as specified in the requirements.
- Related tests pass successfully.
- Security and permission checks are satisfied.
- The user interface is clear and usable.
- The new feature does not break existing functionality.

2.7.5 Security and Access Control Testing

Since the project is a multi-tenant system, special attention will be given to access permissions, tenant isolation, and ensuring that each user can only access data that belongs to their own store or tenant scope.

2.7.6 AI Feature Validation

The AI-based functionalities, such as store content generation or product description support, will be evaluated to ensure that their outputs are relevant, accurate, and useful within the system context.

2.7.7 Continuous Documentation and Monitoring

Errors, issues, feedback, and improvements will be documented continuously throughout the development process. At the end of each increment, the implemented features will be reviewed and validated to ensure that the expected quality level has been achieved.

2.8 Project Management Tools Used

The project team used **GitHub** as the primary project management and development collaboration tool. It was used to organize the source code, manage versions, track changes made to the project, and support collaboration among team members throughout the different development phases. GitHub also helped document development progress, maintain the history of modifications, and facilitate coordination between backend and frontend tasks within a shared development environment.

2.9 Monitoring and Continuous Evaluation Mechanisms

Simple and effective mechanisms were adopted to monitor project progress and ensure quality. Continuous communication was maintained among team members and with the supervisor to discuss progress and resolve issues promptly.

Ongoing evaluation was conducted by the supervisor to review both progress and work quality. In addition, tasks were regularly tracked to ensure alignment with the project schedule, and weekly meetings were held to review completed work and plan the next steps.

2.10 Software Configuration Management (SCM)

2.10.1 Overview

Software Configuration Management was implemented using Git and GitHub to manage the project's source code, track changes, and support collaboration among team members throughout the development process. This approach ensured system stability and clear traceability of all modifications.

2.10.2 Git Repository Structure

The project is hosted on GitHub, where the source code is organized into separate repositories for the Backend and Frontend to improve structure and maintainability.

Backend Repository:

Contains all server-side components, including business logic, database operations, and API endpoints.

Frontend Repository:

Contains the user interface, components, pages, and services responsible for interacting with the backend.

This separation provides:

- Parallel development between team members

- Better code organization

- Improved maintainability and scalability

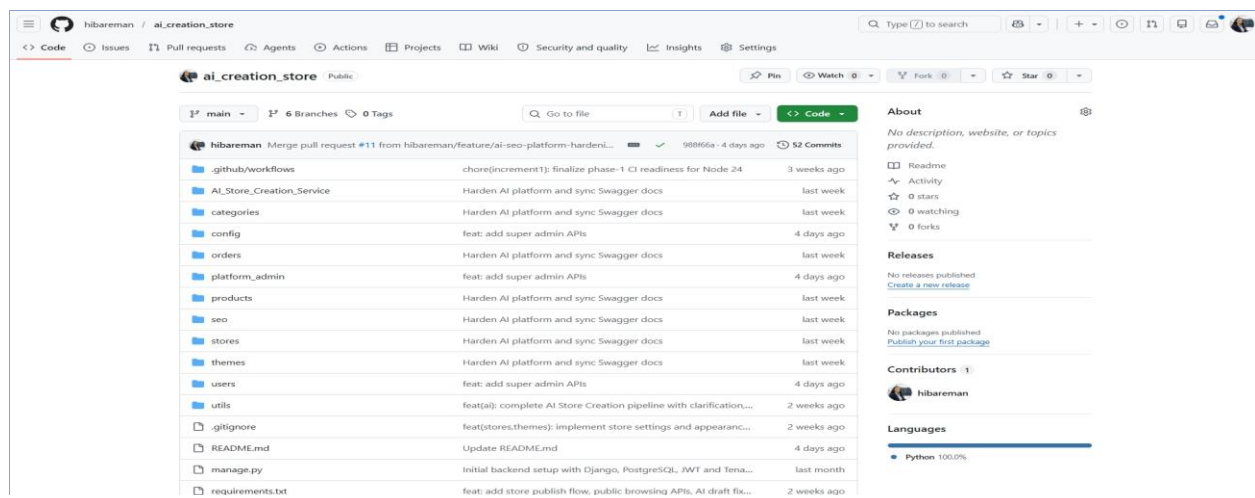


Figure 2.3 Git Repository Structure

2.10.3 Branching Strategy

A simple and clear branching strategy was adopted to support incremental development:

main: contains the stable and production-ready version

dev: used for ongoing development and integration

feature/: temporary branches for implementing specific features

This strategy helps in organizing development, reducing conflicts, and maintaining system stability.

2.10.4 Development Workflow

The development process follows a structured workflow:

A feature or task is identified

A feature branch is created from the dev branch

The feature is implemented and tested locally

The feature is merged into the dev branch

After validation, changes are merged into the main branch

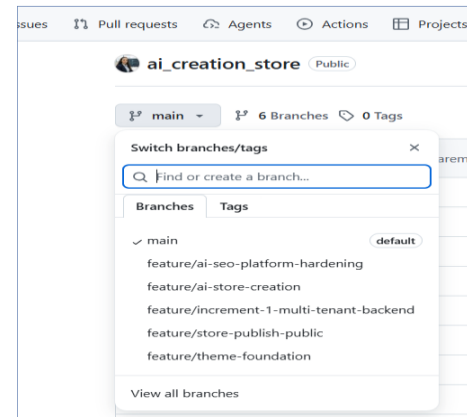


Figure 2. 4 Branching Strategy

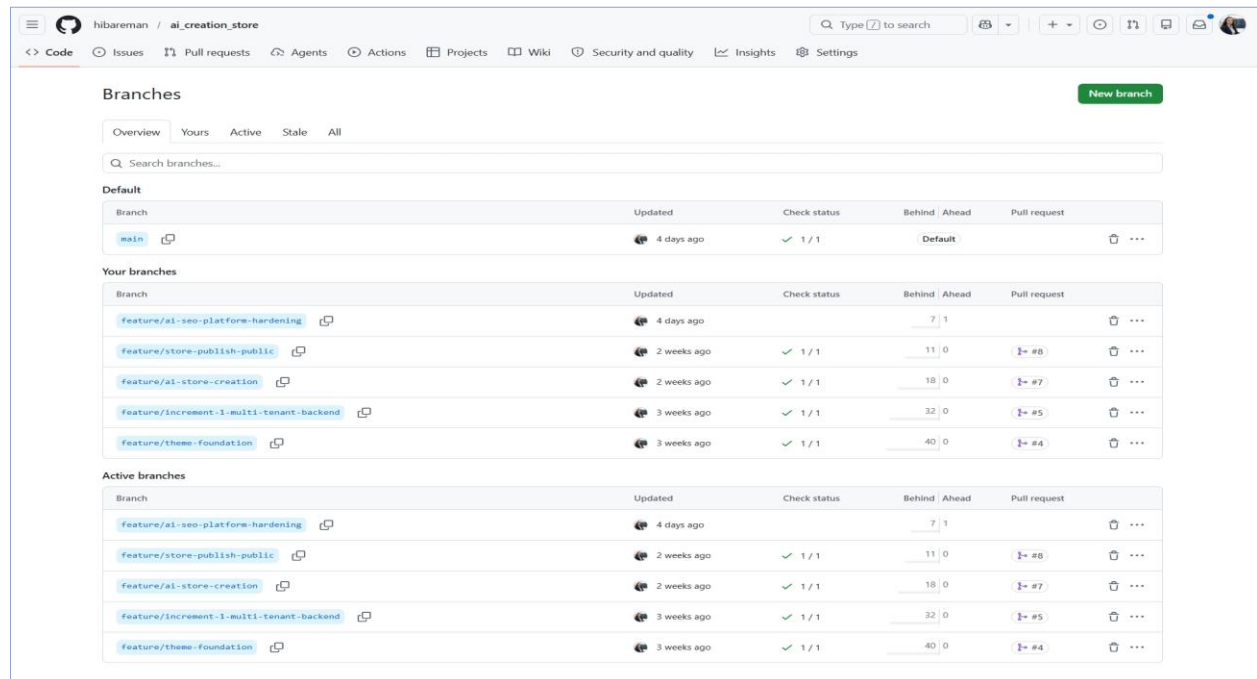


Figure 2.5 Git Branches

Merge operations were used in a controlled manner to ensure smooth integration without affecting the stability of the system.

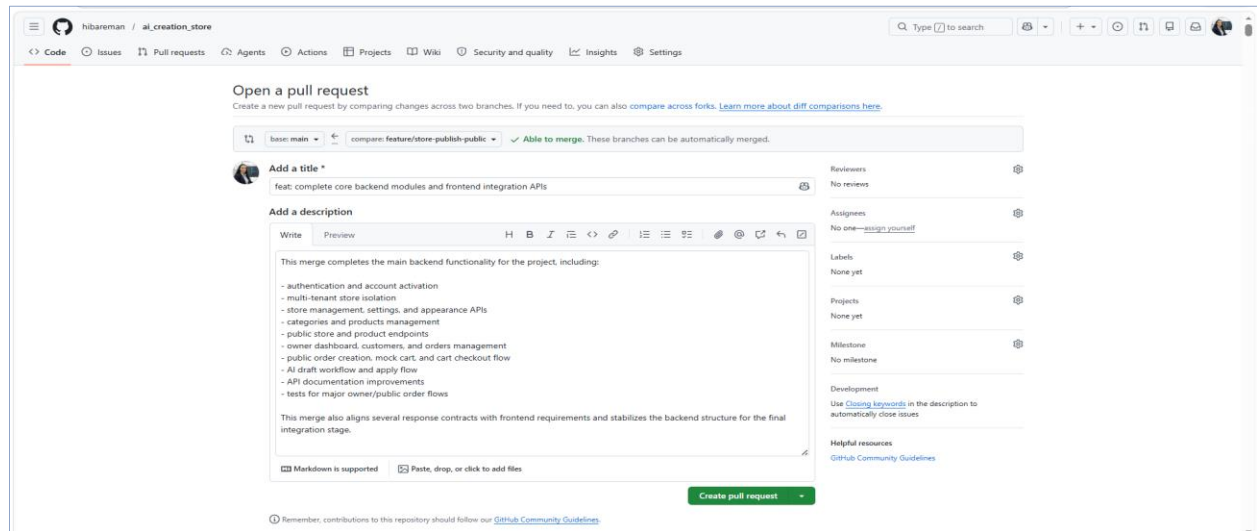


Figure 2.6 Merge Operations 1

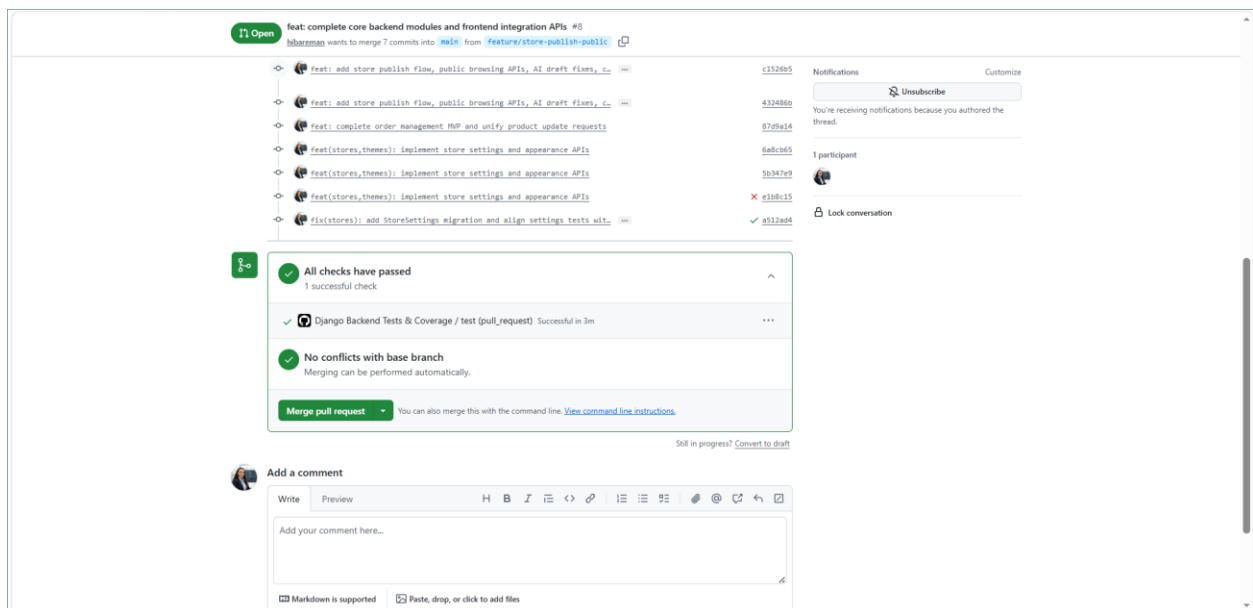


Figure 2.7 Merge Operations 2

Chapter3: Literature Review and Related Studies

3.0 Introduction

This chapter presents a review of previous studies and modern technologies related to the **Souq Engine** project. It focuses on the use of large language models for structured output generation and the technologies that support building a Multi-Tenant e-commerce platform. It also provides a brief comparison with existing systems to identify the research gap and the proposed innovation.

3.1 Previous Studies in Intelligent and Similar Systems

The studies focus on structured output generation using LLMs, AI applications in e-commerce, Multi-Tenant SaaS architecture, and existing e-commerce platforms.

Study	What It Presented	What It Lacks	Relation to Our Project
JSONSchemaBench	Evaluated the ability of LLMs to generate structured JSON outputs according to predefined schemas.	It showed that LLMs may still produce missing fields, incorrect data types, or hallucinated keys when schemas become complex.	Supports the decision to use a clear JSON schema and validation layer before saving AI-generated store data.
SchemaBench	Studied the performance of language models in following structured output requirements.	It focuses mainly on schema compliance and does not provide a complete business application.	Helps identify the risks of generating invalid store configuration data from user descriptions.
Prompt Engineering for Structured Data	Showed that including the JSON schema directly inside the prompt improves the structural quality of LLM outputs.	Prompting alone may not guarantee fully correct or meaningful business data.	Supports using Schema-Guided Prompting to generate store name, categories, products, and theme data.
Prompt Pattern Catalog	Presented reusable prompt patterns to improve interaction with ChatGPT and similar LLMs.	It is general and not specifically designed for e-commerce store generation.	Helps design clearer prompts for generating structured store drafts.
Structured Outputs / Function Calling	Introduced mechanisms that force the model to follow a predefined JSON structure during generation.	It mainly ensures structural correctness, but semantic validation is still required.	Supports the use of structured outputs with an additional validation layer in the proposed system.

Structure-Aware Fine-Tuning for Table Generation	Used fine-tuning techniques to improve the model's ability to generate structured data.	It requires additional training resources, which makes it unsuitable for the first project phase.	Justifies using prompt-based and schema-guided generation instead of fine-tuning in Senior 1.
EcomInstruct	Presented an instruction dataset for e-commerce tasks such as product descriptions, categories, and product-related content.	It focuses on improving model performance for e-commerce tasks, but it does not build a full store creation platform.	Supports the idea that AI can assist in generating e-commerce content such as products and categories.
AI-Based Product Description Generation / EcomGPT	Focused on using AI to generate product descriptions and improve e-commerce content quality.	It focuses mainly on product-level content, not complete store generation.	Shows that the proposed project is broader because it generates an initial store draft, not only product descriptions.

Table 3.1 Similar Systems

The reviewed studies show that LLMs can generate structured outputs from free-text inputs. However, relying only on free generation may lead to issues such as missing fields, incorrect data types, or unwanted generated keys. Therefore, these studies support the use of methods such as Schema-Guided Prompting and Structured Outputs, along with a validation layer before saving the data

3.2 Modern Technologies and Frameworks in the Field

This section presents the main modern technologies and frameworks related to the proposed system. These technologies support web development, artificial intelligence, structured data generation, and the Multi-Tenant SaaS architecture used in the project.

Technology / Framework	Role in the Project
Django	Used as the main backend framework for building the server-side logic, managing data models, authentication, authorization, and business rules.
Django REST Framework	Used to build RESTful APIs that connect the backend with the frontend and allow different system components to exchange data.
React	Used to build interactive user interfaces for the store owner dashboard, Super Admin dashboard, and public store pages.

Large Language Models (LLMs)	Used to analyze the user's textual store description and generate initial store data such as store name, description, categories, products, and theme suggestions.
Natural Language Processing (NLP)	Used as part of the AI process to understand and process human language input provided by the store owner.
JSON Schema / Structured Outputs	Used to define the expected structure of AI-generated data and ensure that the output can be validated before being saved in the system.
Redis	Used for temporary storage of AI-generated store drafts, supporting interactive editing, partial regeneration, and automatic expiration using TTL.
Database System	Used to store users, stores, products, categories, orders, permissions, and other core platform data.
Multi-Tenant Architecture	Used to support multiple stores within the same platform while keeping each store's data logically isolated from others.
Authentication and Authorization	Used to protect the system and control access according to user roles such as Store Owner, Store Admin, and Super Admin.
External AI API	Used to connect the system with an AI model capable of generating structured store configuration data from a textual prompt.

Table 3.2 Technologies & Frameworks

These technologies are suitable for the proposed system because they support the main requirements of the project: building a web-based SaaS platform, enabling AI-assisted store generation, managing store data securely, and ensuring that each tenant's data remains isolated within the system.

3.3 Comparative Analysis of Existing Systems and Products

System / Product	Main Features	Technologies / Approach	Strengths	Weaknesses
Shopify	Store creation, product management, order management, themes, domain support	SaaS platform, web-based store builder, third-party apps	Mature platform, rich ecosystem, strong store management features	Store setup is mainly manual, limited AI role in automatic store generation, less focused on Arabic market
Salla	Store creation, product and order management, Arabic support, themes, domain support	SaaS e-commerce platform focused on Arab users	Easy to use, supports Arabic language and regional market needs	Does not use AI as a core feature to automatically generate a complete store from a textual description
Proposed System: Souq Engine	AI-assisted store generation, product and category management, order management, themes, Multi-Tenant architecture	Django, React, LLMs, JSON Schema, Multi-Tenant SaaS architecture	Generates an initial store from a simple user description, supports non-technical users, combines AI generation with store management	In Senior 1, advanced features such as real payment, AI image processing, and advanced AI agents are outside the scope

Table 3.3 Comparative between Existing Systems And Products

The comparison shows that existing commercial platforms such as Shopify and Salla provide strong e-commerce management features, including product management, order management, themes, and domain support. However, they mainly depend on manual store setup, while AI is not used as a core mechanism for generating a complete store from a textual description.

Therefore, the proposed system fills this gap by combining **AI-assisted store generation**, **structured store configuration**, and **Multi-Tenant SaaS e-commerce management** within one platform.

3.4 Research Gap and Proposed Innovation

- Based on the previous studies and comparative analysis, most existing solutions focus on either traditional e-commerce store management or isolated AI tasks such as product description generation, category suggestion, or structured output generation. However, they do not fully provide an integrated system that can generate an initial online store from a simple textual description and then allow the user to manage that store within the same platform.
- The main research gap is that current e-commerce platforms usually depend on manual store setup, while AI-based studies mainly focus on generating specific content rather than creating a complete store configuration. In addition, many existing solutions do not combine AI-assisted generation with a Multi-Tenant SaaS architecture that supports multiple stores while keeping each store's data isolated.
- The proposed innovation in **Souq Engine** is to combine these aspects in one system: AI-assisted store generation, structured and validated store configuration, and Multi-Tenant SaaS store management. This allows non-technical business owners to create an initial store faster, with less effort, while still having a manageable and scalable platform for products, categories, orders, and store settings.

3.5 Programming Framework and Reference Foundations

- The project is based on several computational and architectural concepts that support building a structured, scalable, and AI-assisted e-commerce system. The main foundations include **REST Architecture**, **Multi-Tenant SaaS Architecture**, **Large Language Models (LLMs)**, **JSON Schema / Structured Outputs**, and **Validation Layer**.
- The system uses **REST Architecture** to build APIs that allow organized communication between the backend and frontend. It also relies on **Multi-Tenant SaaS Architecture** to support multiple stores within the same platform while keeping each store's data isolated. In addition, **JSON Schema / Structured Outputs** are used to define and validate the AI-generated data before saving it.
- The project also applies **Authentication and Authorization** to verify user identity and control access permissions, along with a **Validation Layer** to ensure data correctness before storage. These concepts provide a reliable foundation for developing an AI-assisted e-commerce store creation system.

3.6 Modern and Future Trends in the Field

The fields of e-commerce and intelligent systems are rapidly evolving with the growth of generative artificial intelligence and large language models. In the context of **Souq Engine**, the most relevant modern and future trends include the following:

Trend	Relation to the Project
Generative AI	Supports the generation of store content such as descriptions, categories, sample products, and theme suggestions from a simple textual input.
Large Language Models (LLMs)	Used to understand the user's store description and transform it into structured data that can be used by the system.
Structured Outputs	Helps ensure that AI-generated results follow a predefined format such as JSON, making them easier to validate and store.
AI-Assisted E-Commerce Automation	Supports automating tasks such as store setup, product data generation, category suggestion, and content creation.
Multi-Tenant SaaS Platforms	Represents a modern approach for supporting multiple stores within one platform while keeping each store's data isolated.
MLOps / AI Operations	Can be used in future stages to monitor AI model performance, track output quality, and manage external AI service reliability.
AI Agents	Can be considered as a future extension where the AI could execute direct actions inside the store, such as adding products or changing themes.
Personalization in E-Commerce	Can support future features such as product recommendations and customized shopping experiences based on visitor behavior.
Edge AI	May be useful in the future to reduce response time or run some AI tasks closer to the user, but it is not essential in the current phase.
Trustworthy and Validated AI	Directly supports the project by ensuring that AI-generated data is checked and validated before being saved or shown to users.

Table 3.4 Modern & Future Trends In The Field

Chapter4: System analysis

4.1 Feasibility Study

- The feasibility study shows that building the system is technically and financially possible within the current project scope. From a technical perspective, the platform depends on available and suitable technologies such as web application development, databases, APIs, and the **Multi-Tenant SaaS** architecture, which allows multiple stores to be managed within the same system while keeping each store's data isolated.
- Integrating artificial intelligence into the system is also technically possible through language models that can analyze textual descriptions and generate initial store data. However, running an AI model locally requires a device with high specifications, while the available device does not have suitable capabilities for this type of execution.
- From a financial perspective, the project is feasible with an acceptable cost because most of the used technologies are free or open source. In addition, adopting a **Multi-Tenant** architecture reduces the need to create a separate system instance for each store, which helps reduce future server and maintenance costs.

4.2 Requirements Elicitation

- The system requirements were collected by analyzing the problem of online store creation and studying the needs of the target users, especially non-technical business owners who need an easier and faster way to create their stores. Similar e-commerce platforms such as **Shopify** and **Salla** were also analyzed to identify the core features required in e-commerce store creation and management systems.
- The requirements elicitation process also depended on reviewing the project scope and constraints in order to define the requirements suitable for the **Senior 1** phase and focus on the core foundation of the system. Based on this, the main requirements were identified, including AI-assisted store creation, product and category management, order management, **Multi-Tenant** support, and dashboards for both the store owner and the Super Admin.

• Summary of Requirements Elicitation Results:

Source Used	Extracted Result
Problem analysis	The need to reduce the time and effort required to create an online store
Target user analysis	The need for a simple interface suitable for non-technical business owners
Review of similar systems	Identification of core features such as product management, orders, themes, and domain support
Project scope review	Defining the features suitable for the Senior 1 phase
Project constraints review	Postponing advanced features and focusing on the system core

Table 4.1 Summary Of Requirements

4.3 Stakeholders Identification

This section focuses on the main stakeholders who directly interact with the system, because their needs have a direct impact on the functional requirements and platform design. In the **Souq Engine** project, three main stakeholders were identified: the Store Owner, the Store Visitor / Customer, and the Super Admin.

Stakeholder	Description	Needs	Expectations
Store Owner	The main user of the system, representing the business owner or merchant who wants to create and manage an online store without needing advanced technical knowledge. The Store Owner starts by entering a simple description of the store idea, then reviews the generated draft and manages the store data later.	Needs an easy and fast way to create an online store, generate initial store data using AI, such as store name, description, categories, products, and suitable theme. The Store Owner also needs to manage products, categories, orders, and customize the store appearance in a simple way.	Expects the platform to be easy to use and to reduce the time and effort required to create the store. The Store Owner also expects the AI-generated data to be reviewable and editable before approval, with clear control over the store and its data.
Store Visitor / Customer	The user who visits the published store to view products and interact with the public store interface. This user does not use the dashboard, but interacts with the store pages created and managed by the Store Owner.	Needs to browse products easily, view product details clearly, and navigate between categories. The visitor also needs a simple experience that helps them understand the store content and use the mock cart without complexity.	Expects the store to be organized, easy to browse, and clear in presenting products and information. The visitor also expects fast browsing and a responsive design that works on different screen sizes.
Super Admin	The user responsible for managing the platform as a whole, not a single store. The Super Admin monitors store status, follows general statistics, and controls platform-level elements such as themes and store activation status.	Needs a centralized dashboard to view registered stores and their statuses, activate or suspend stores, monitor the number of active and inactive stores, track product counts, and manage available themes.	Expects the platform to provide clear tools for monitoring the system and managing stores efficiently. The Super Admin also expects suitable permissions that allow platform control without unintentionally affecting store owners' data.

Table 4.2 Stakeholders Identification

4.4 Functional Requirements

This section presents the main functional requirements that the **Souq Engine** system must provide. The requirements are organized according to the main system areas in order to clearly describe the functions of each part.

4.4.1 Account Management and Authorization

Requirement_1.1: The system shall allow users to create a new account using email and password.

Requirement_1.2: The system shall allow users to log in securely.

Requirement_1.3: The system shall allow users to log out from their accounts.

Requirement_1.4: The system shall verify user permissions before allowing access to protected pages or operations.

Requirement_1.5: The system shall support basic roles such as Store Owner and Super Admin.

4.4.2 Store Management and Multi-Tenant Architecture

Requirement_2.1: The system shall allow the Store Owner to create an online store linked to their account.

Requirement_2.2: The system shall automatically assign the store creator as the owner of that store.

Requirement_2.3: The system shall keep each store's data isolated from other stores using a Multi-Tenant architecture.

Requirement_2.4: The system shall prevent users of one store from accessing the data of any other store.

Requirement_2.5: The system shall create a unique subdomain for each store when it is created.

Requirement_2.6: The system shall allow the Store Owner to view the current store information.

Requirement_2.7: The system shall allow the Store Owner to update basic store information such as name, description, language, currency, and logo.

Requirement_2.8: The system shall support basic store statuses such as Draft, Setup, Active, and Inactive.

Requirement_2.9: The system shall allow the Store Owner to activate or publish the store after completing the basic setup.

4.4.3 Category and Product Management

Requirement_3.1: The system shall allow the Store Owner to create product categories inside their store.

Requirement_3.2: The system shall allow the Store Owner to update and delete product categories related to their store.

Requirement_3.3: The system shall allow the Store Owner to add a new product with basic information such as name, description, price, image, and stock quantity.

Requirement_3.4: The system shall allow the Store Owner to update only the products that belong to their store.

Requirement_3.5: The system shall allow the Store Owner to delete products from their store after confirmation.

Requirement_3.6: The system shall link each product and category only to its related store to prevent it from appearing in other stores.

Requirement_3.7: The system shall support basic inventory management for each product, including updating the available quantity.

4.4.4 Order Management

Requirement_4.1: The system shall allow the Store Owner to view orders related to their store.

Requirement_4.2: The system shall allow the Store Owner to update the order status, such as New, Processing, Completed, or Cancelled.

Requirement_4.3: The system shall store basic order data, including products, quantities, total price, and order date.

4.4.5 AI-Assisted Store Creation

Requirement_5.1: The system shall provide a text input field that allows the user to enter a store idea description to start AI-assisted store creation.

Requirement_5.2: The system shall send the user's description to the AI service to generate an initial store draft.

Requirement_5.3: The system shall generate structured outputs including the store name, description, categories, initial products, and suitable theme.

Requirement_5.4: The system shall validate the AI-generated outputs before saving them in the system.

Requirement_5.5: The system shall ask clarification questions if the user's description is not sufficient to create a suitable store draft.

Requirement_5.6: The system shall allow the user to review the AI-generated store draft before approval.

Requirement_5.7: The system shall allow the user to edit or regenerate the draft before final saving.

Requirement_5.8: The system shall not save the AI-generated outputs permanently until the user explicitly approves them.

4.4.6 Theme System and Appearance Customization

Requirement_6.1: The system shall provide a set of ready-made themes that the Store Owner can choose from.

Requirement_6.2: The system shall allow simple store appearance customization, such as colors, logo, and font.

Requirement_6.3: The system shall apply the selected theme to the published store interface.

4.4.7 Store Visitor Interface

Requirement_7.1: The system shall allow visitors to browse products displayed in a specific store.

Requirement_7.2: The system shall allow visitors to view product details such as name, description, price, and image.

Requirement_7.3: The system shall allow visitors to browse products by categories.

Requirement_7.4: The system shall provide a mock cart that allows visitors to add products without performing real payment.

4.4.8 Super Admin Dashboard

Requirement_8.1: The system shall provide a Super Admin dashboard to view all registered stores and monitor their status.

Requirement_8.2: The system shall allow the Super Admin to activate or suspend stores.

Requirement_8.3: The system shall display general indicators for the Super Admin, such as the number of active stores, inactive stores, and total products.

4.4.9 Mock Billing and Basic Support

Requirement_9.1: The system shall provide a mock billing system to simulate the SaaS subscription model without real payment processing.

Requirement_9.2: The system shall support Arabic and English in the interface and basic content.

Requirement_9.3: The system shall support basic SEO elements such as meta tags for the store.

4.5 Non-Functional Requirement

Non-functional requirements define the quality attributes that the system must satisfy, such as performance, security, usability, scalability, and maintainability. These requirements do not describe direct system functions, but they define how the system should operate to ensure a stable and secure user experience.

Requirement ID	Non-Functional Requirement	Description / Measurable Criteria
NFR-01	Data Isolation	The system shall ensure that each store's data is isolated from other stores, so no user can access another store's data.
NFR-02	Secure Access Control	The system shall enforce authentication and authorization on protected pages and APIs, and prevent unauthorized access.
NFR-03	AI Output Validation	The system shall validate AI-generated outputs against the expected structure before saving or displaying them to the user.
NFR-04	AI Draft Generation Performance	The initial AI-generated store draft should be generated within approximately 3 to 5 seconds under normal conditions.
NFR-05	Standard Operation Response Time	Standard operations such as login, product browsing, and store data management should respond within an acceptable time under normal conditions.
NFR-06	Continuity During AI Failure	If the AI service fails or becomes slow, the system shall provide an alternative response or allow the user to continue the store creation process.
NFR-07	Scalability	The system shall support an increasing number of stores, products, users, and AI requests without requiring major architectural changes.

NFR-08	Usability for Non-Technical Users	The system shall provide a simple and clear user flow that enables non-technical store owners to create and manage their stores easily.
NFR-09	Clear Feedback and Error Handling	The system shall display loading indicators, success messages, validation errors, and clear user-friendly failure messages.
NFR-10	Maintainability	The system shall follow an organized and layered structure that allows features such as AI generation, store management, and product management to be modified or extended with minimal impact on other components.

Table 4.3 Non-Functional Requirements

4.6 Requirements Prioritization and Management Mechanism

The **MoSCoW** method was used to prioritize the system requirements. This method helps classify the requirements based on their importance in the current project phase. The requirements are divided into **Must Have**, **Should Have**, **Could Have**, and **Won't Have**.

Requirement ID	Requirement	Category	MoSCoW Priority
FR-01	The system shall allow users to create a new account using email and password.	Authentication	Must Have
FR-02	The system shall allow users to log in securely.	Authentication	Must Have
FR-03	The system shall allow users to log out from their accounts.	Authentication	Should Have
FR-04	The system shall verify user permissions before allowing access to protected pages or operations.	Authorization	Must Have
FR-05	The system shall support basic roles such as Store Owner and Super Admin.	Authorization	Must Have
FR-06	The system shall allow the Store Owner to create an online store linked to their account.	Store Management	Must Have
FR-07	The system shall automatically assign the store creator as the owner of that store.	Store Management	Must Have
FR-08	The system shall keep each store's data isolated from other stores using a Multi-Tenant architecture.	Multi-Tenancy	Must Have
FR-09	The system shall prevent users of one store from accessing the data of any other store.	Multi-Tenancy	Must Have

FR-10	The system shall create a unique subdomain for each store when it is created.	Multi-Tenancy	Should Have
FR-11	The system shall allow the Store Owner to view the current store information.	Store Management	Must Have
FR-12	The system shall allow the Store Owner to update basic store information such as name, description, language, currency, and logo.	Store Management	Must Have
FR-13	The system shall support basic store statuses such as Draft, Setup, Active, and Inactive.	Store Management	Should Have
FR-14	The system shall allow the Store Owner to activate or publish the store after completing the basic setup.	Store Management	Should Have
FR-15	The system shall allow the Store Owner to create product categories inside their store.	Category Management	Must Have
FR-16	The system shall allow the Store Owner to update and delete product categories related to their store.	Category Management	Must Have
FR-17	The system shall allow the Store Owner to add a new product with basic information such as name, description, price, image, and stock quantity.	Product Management	Must Have
FR-18	The system shall allow the Store Owner to update only the products that belong to their store.	Product Management	Must Have
FR-19	The system shall allow the Store Owner to delete products from their store after confirmation.	Product Management	Must Have
FR-20	The system shall link each product and category only to its related store to prevent it from appearing in other stores.	Product Management / Multi-Tenancy	Must Have
FR-21	The system shall support basic inventory management for each product, including updating the available quantity.	Product Management	Should Have
FR-22	The system shall allow the Store Owner to view orders related to their store.	Order Management	Should Have
FR-23	The system shall allow the Store Owner to update the order status, such as New, Processing, Completed, or Cancelled.	Order Management	Should Have

FR-24	The system shall store basic order data, including products, quantities, total price, and order date.	Order Management	Should Have
FR-25	The system shall provide a text input field that allows the user to enter a store idea description to start AI-assisted store creation.	AI Store Generator	Must Have
FR-26	The system shall send the user's description to the AI service to generate an initial store draft.	AI Store Generator	Must Have
FR-27	The system shall generate structured outputs including the store name, description, categories, initial products, and suitable theme.	AI Store Generator	Must Have
FR-28	The system shall validate the AI-generated outputs before saving them in the system.	AI Validation	Must Have
FR-29	The system shall ask clarification questions if the user's description is not sufficient to create a suitable store draft.	AI Store Generator	Should Have
FR-30	The system shall allow the user to review the AI-generated store draft before approval.	AI Store Generator	Must Have
FR-31	The system shall allow the user to edit or regenerate the draft before final saving.	AI Store Generator	Should Have
FR-32	The system shall not save the AI-generated outputs permanently until the user explicitly approves them.	AI Store Generator	Must Have
FR-33	The system shall provide a set of ready-made themes that the Store Owner can choose from.	Theme System	Should Have
FR-34	The system shall allow simple store appearance customization, such as colors, logo, and font.	Theme System	Could Have
FR-35	The system shall apply the selected theme to the published store interface.	Theme System	Should Have
FR-36	The system shall allow visitors to browse products displayed in a specific store.	Customer-facing Store	Must Have
FR-37	The system shall allow visitors to view product details such as name, description, price, and image.	Customer-facing Store	Must Have
FR-38	The system shall allow visitors to browse products by categories.	Customer-facing Store	Should Have
FR-39	The system shall provide a Super Admin dashboard to view all registered stores and monitor their status.	Super Admin	Should Have

FR-40	The system shall support Arabic and English and provide basic SEO elements such as meta tags.	Localization / SEO	Could Have
NFR-01	The system shall ensure that each store's data is isolated from other stores.	Security / Multi-Tenancy	Must Have
NFR-02	The system shall enforce authentication and authorization on protected pages and APIs.	Security	Must Have
NFR-03	The system shall validate AI-generated outputs before saving or displaying them.	Security / Reliability	Must Have
NFR-04	The initial AI-generated store draft should be generated within approximately 3 to 5 seconds under normal conditions.	Performance	Should Have
NFR-05	Standard operations should respond within an acceptable time under normal conditions.	Performance	Should Have
NFR-06	If the AI service fails or becomes slow, the system shall provide an alternative response or allow the user to continue the store creation process.	Availability	Must Have
NFR-07	The system shall support an increasing number of stores, products, users, and AI requests without major architectural changes.	Scalability	Should Have
NFR-08	The system shall provide a simple user experience for non-technical users.	Usability	Must Have
NFR-09	The system shall display loading indicators, success messages, validation errors, and clear failure messages.	Usability	Should Have
NFR-10	The system shall follow an organized and layered structure that supports future modification and extension.	Maintainability	Should Have

Table 4.4 Requirements Prioritization

4.7 Requirements Modeling

4.7.1 AI-Assisted Store Creation

4.7.1.1 AI-Assisted Store Creation Use Case Diagram:

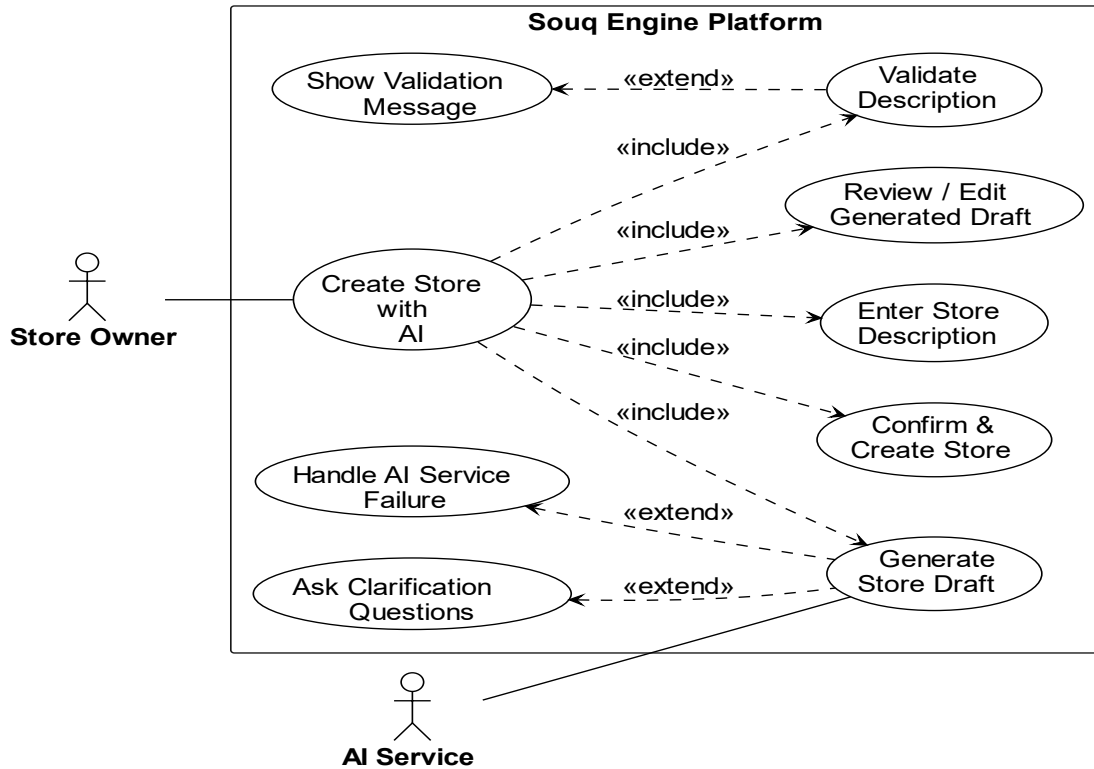


Figure 4.1 AI-Assisted Store Creation Use Case Diagram

4.7.1.2 AI-Assisted Store Creation Use case specifications:

Use Case ID	UC-AI-01
Use Case Name	AI-Assisted Store Creation
Actors	Store Owner
Description	This use case allows the Store Owner to provide a textual description of the store idea. The system analyzes the description using AI and generates an initial store draft that includes basic store information, categories, sample products, and a suitable theme.
Preconditions	1. The Store Owner is registered and authorized to use the system. 2. The Store Owner has permission to create a store. 3. The AI service is available, or an alternative flow exists in case of AI service failure.
Main Flow	1. The Store Owner starts the AI-assisted store creation process.

	2. The Store Owner provides a textual description of the store idea. 3. The system validates the description to ensure that it is not empty and contains enough initial information. 4. The system sends the description to the AI service for processing. 5. The AI service analyzes the description and generates an initial store draft. 6. The system validates that the generated draft contains the required data in the correct structure. 7. The system presents the generated draft to the Store Owner for review. 8. The Store Owner reviews the draft and may edit it or request regeneration if needed. 9. The Store Owner approves the final draft. 10. The system saves the store and its related data under the Store Owner's account while ensuring tenant data isolation. 11. The store data becomes available for later management through the Store Owner dashboard.
Alternative Flows	<u>A1 – Initial Description Is Insufficient:</u>(Starts after Step 3 of the Main Flow.) 1.The system detects that the description is too short or does not contain enough initial Information. 2. The system asks the Store Owner to provide a clearer description. 3. The Store Owner provides an updated description. 4. The flow returns to Step 3 of the Main Flow to validate the description again. <u>A2 – Clarification Questions Are Required:</u> (Starts after Step 5 of the Main Flow) 1. The AI service determines that the description is not sufficient to generate a suitable store draft. 2. The system generates clarification questions to collect the missing information. 3. The Store Owner answers the required questions. 4. The system merges the answers with the original description. 5. The AI service continues generating the store draft. 6. The flow returns to Step 6 of the Main Flow to validate the generated draft. <u>A3 – Regenerate Draft or Draft Section:</u> (Starts after Step 8 of the Main Flow) 1. The Store Owner decides that the draft or part of it needs improvement. 2. The system requests the AI service to regenerate the full draft or a specific section of it. 3. The system validates the newly generated result before presenting it to the Store Owner. 4. The flow returns to Step 7 of the Main Flow to present the updated draft for review. <u>E1 – AI Service Failure:</u> (Starts after Step 4 or Step 5 of the Main Flow) 1. The AI service fails or returns an invalid response.

	2. The system displays an appropriate message explaining that automatic generation could not be completed. 3. The system keeps the data entered by the Store Owner. 4. The system allows the Store Owner to retry later or use an alternative path if available. 5. The flow ends without creating the store, or returns later to Step 4 of the Main Flow when the Store Owner retries.
Postconditions	1. A new store is created and linked to the Store Owner. 2. The store contains initial data generated by AI or approved after review. 3. The store status is set to an initial state such as Draft or Setup. 4. The Store Owner can edit the store data later. 5. The data is stored within the correct tenant context to ensure isolation between stores.

Table 4.5 AI-Assisted Store Creation Use case specifications

4.7.1.3 AI-Assisted Store Creation Activity Diagram:

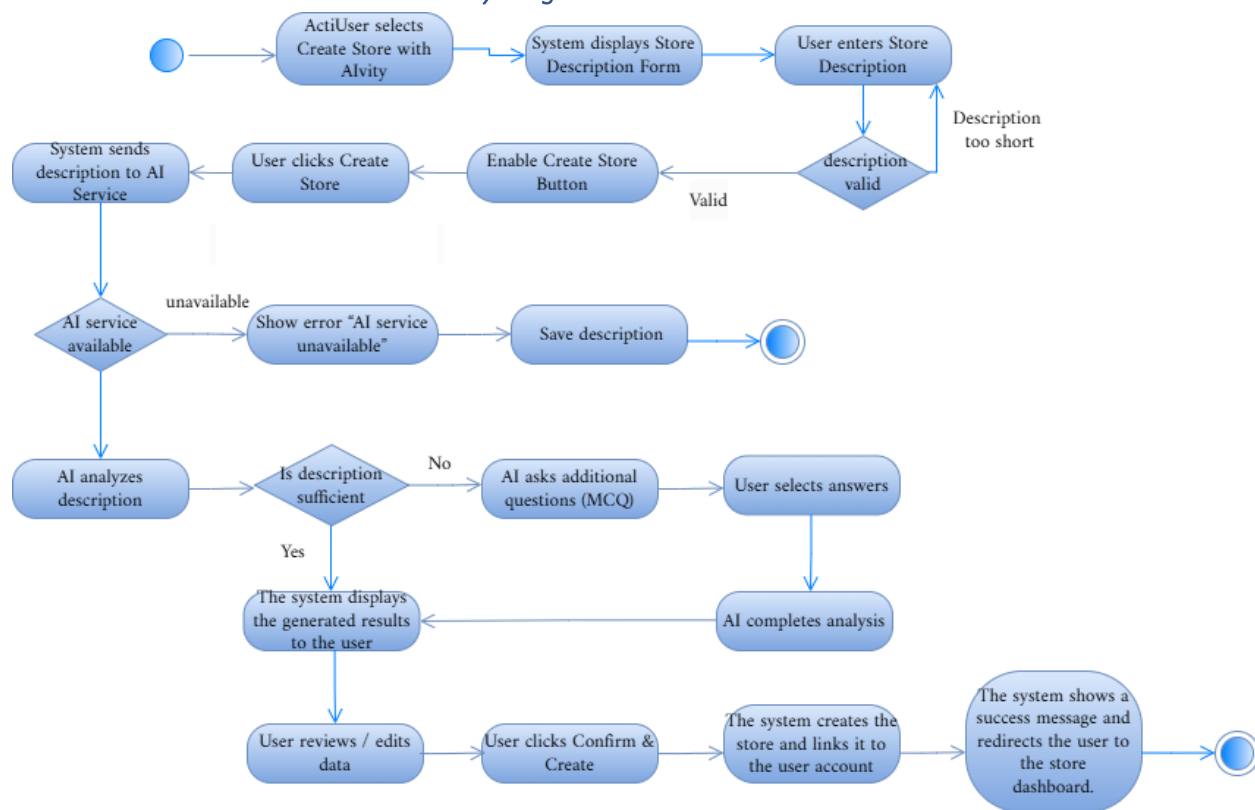


Figure 4. 3 AI-Assisted Store Creation Activity Diagram

4.7.1.4 AI-Assisted Store Creation Sequence Diagram:

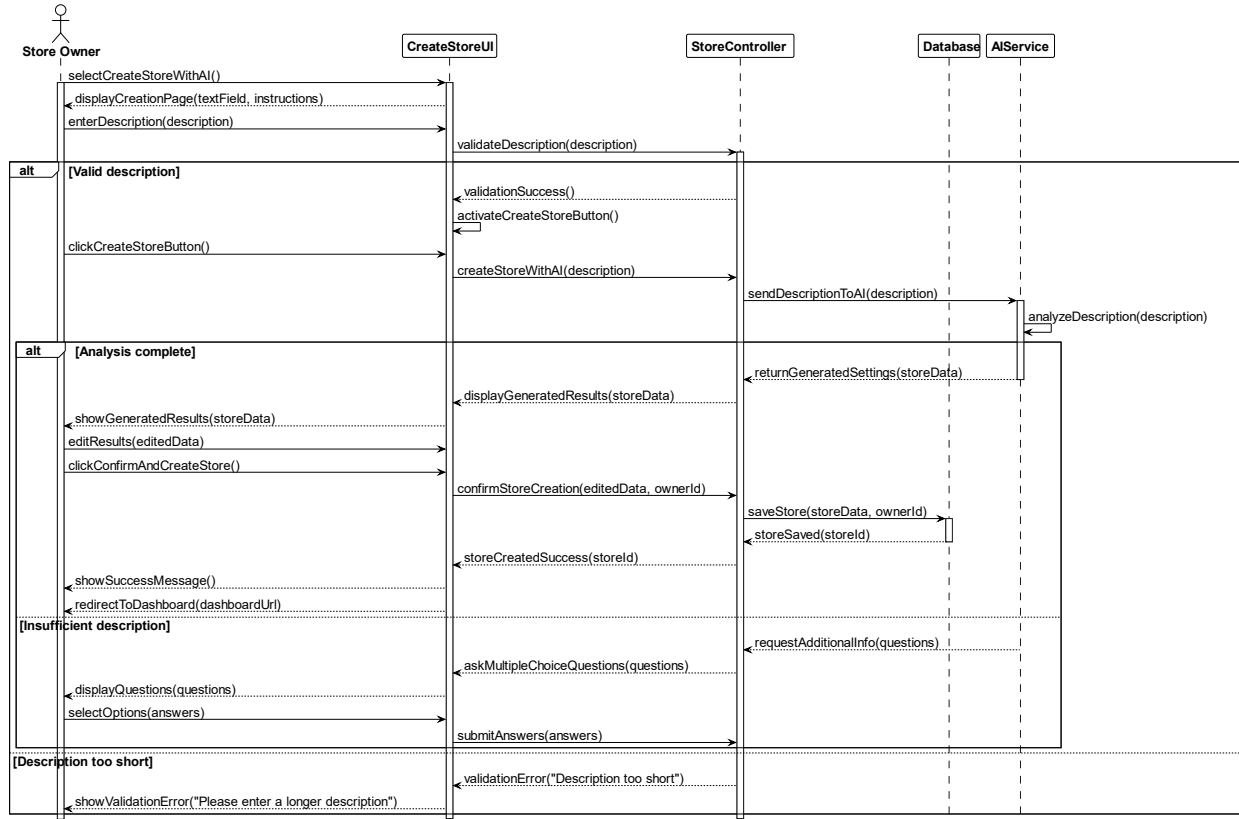


Figure 4.4 AI-Assisted Store Creation Sequence Diagram

4.7.2 Tenant Data Isolation

4.7.2.1 Tenant Data Isolation Use Case Diagram:

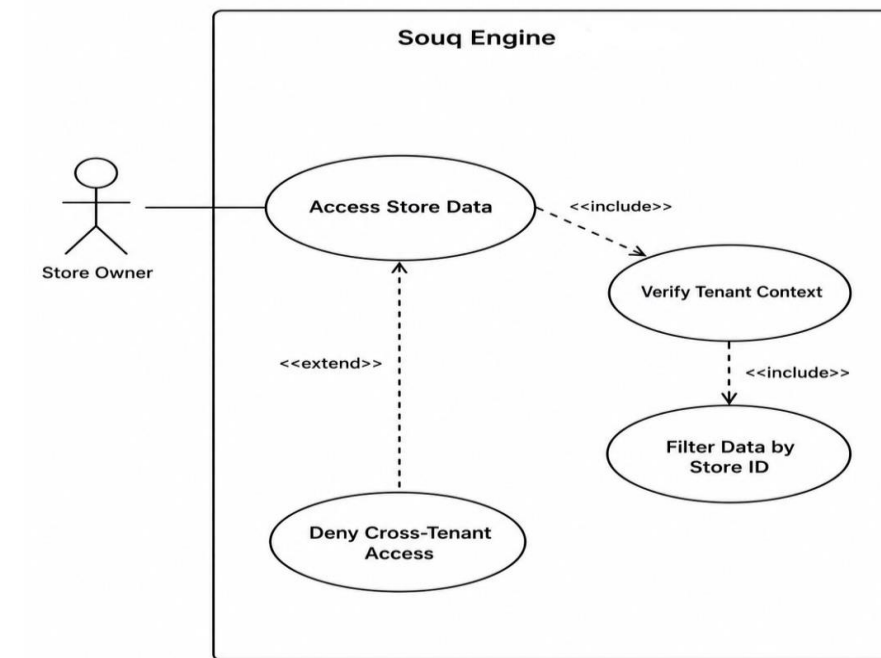


Figure 4.5 Tenant Data Isolation Use Case Diagram

4.7.2.2 Tenant Data Isolation Use case specifications:

Use case id	UC-MT-01
Use case name	Tenant Data Isolation
Actors	Store Owner / Store User, System
Description	The system ensures that each user can only access data associated with the store they belong to, and prevents access to data of any other store within the multi-tenant system.
Preconditions	1. Two or more stores exist in the system. 2. A registered user is associated with a specific store. 3. Data (products or orders) is associated with each store. 4. The user is successfully logged in.
Main Flow	1. The user logs into the system. 2. The system identifies the user and their associated store. 3. The user requests to view data, such as a list of products or orders. 4. The system processes the request and applies a filter condition using the Store ID. 5. The system displays only the data associated with the user's store.

Alternative Flows	A1: If the user attempts to access another store's data. 1. The user sends a request to access a resource that does not belong to their store. 2. The system checks the Store ID associated with the resource. 3. The system detects a mismatch between the resource's Store ID and the user's store. 4. The system denies the request and displays an access denied message (Access Denied or 403).
Postconditions	1. Only the user's own store data is displayed. 2. Any unauthorized access to other stores' data is prevented.

Table 4.6 Tenant Data Isolation Use case specifications

4.7.2.3 Tenant Data Isolation Activity Diagram:

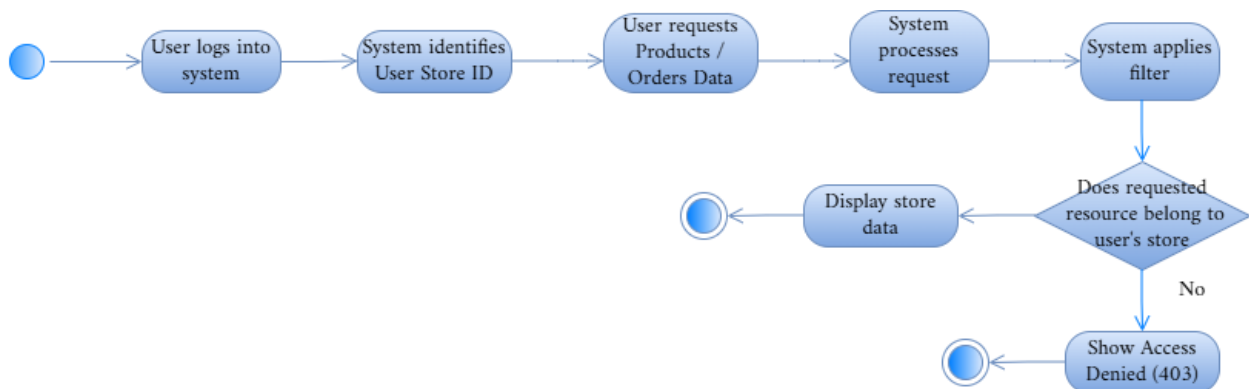


Figure 4.7 Tenant Data Isolation Activity Diagram

4.7.2.4 Tenant Data Isolation Sequence Diagram:

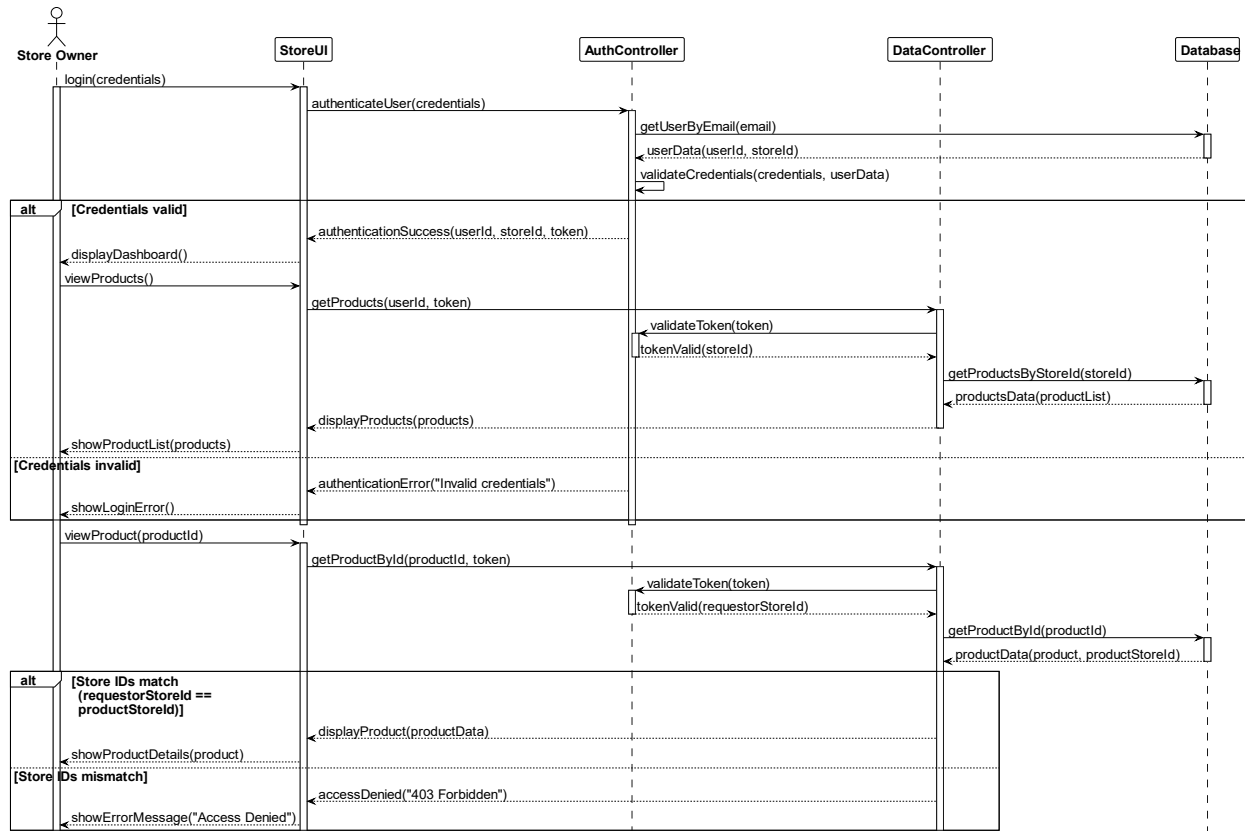


Figure 4.8 Tenant Data Isolation Sequence Diagram

4.7.3 Store Management

4.7.3.1 Store Management Use Case Diagram:

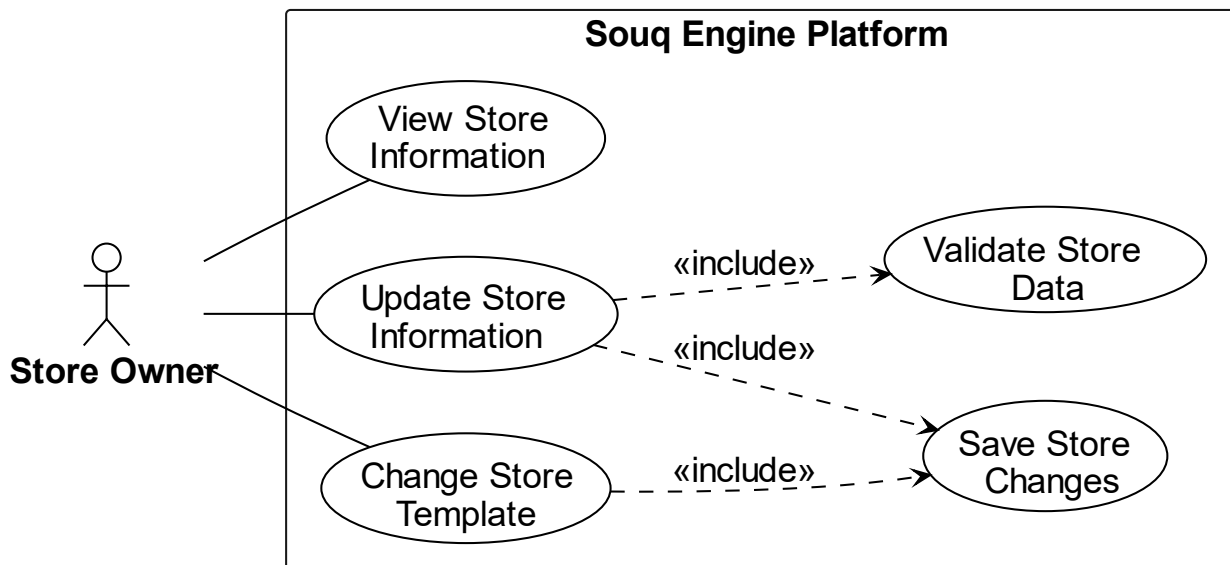


Figure 4.9 Store Management Use Case Diagram

4.7.3.2 Store Management Use case specifications:

Use case id	UC-SM-01
Use case name	Store Management
Actors	Store Owner
Description	The system allows the Store Owner to manage the basic information and settings of their store, including viewing store information, updating store details, and configuring basic store settings such as the selected template.
Preconditions	The Store Owner is authenticated and authorized to manage the store. The store exists in the system. The store belongs to the current Store Owner
Main Flow	1. The Store Owner requests to manage store information. 2. The system verifies that the Store Owner has permission to manage the selected store. 3. The system retrieves the current store information. 4. The system presents the current store details to the Store Owner. 5. The Store Owner updates one or more store settings, such as store name, description, language, currency, logo, or selected template. 6. The system validates the updated store data. 7. The system saves the valid updates. 8. The system applies the updated settings to the store. 9. The system confirms that the store information has been updated successfully.
Alternative Flows	<u>A1 – Missing Required Store Name</u> (Starts after Step 6 of the Main Flow) 1. The system detects that the store name is missing. 2. The system rejects the update request. 3. The system informs the Store Owner that the store name is required. 4. The flow returns to Step 5 of the Main Flow to allow the Store Owner to correct the data. <u>A2 – Invalid Store Data</u> (Starts after Step 6 of the Main Flow) 1. The system detects invalid store data, such as an unsupported format or invalid value. 2. The system rejects the update request. 3. The system informs the Store Owner about the invalid data.

	4. The flow returns to Step 5 of the Main Flow to allow the Store Owner to correct the data A3 – Unauthorized Store Access (Starts after Step 2 of the Main Flow) 1. The system detects that the selected store does not belong to the current Store Owner. 2. The system denies access to the store data. 3. The use case ends without displaying or updating store information.
Postconditions	The store information is updated successfully if the provided data is valid. The updated store settings are saved and applied to the store. No changes are saved if validation fails or the Store Owner is not authorized.

Table 4.7 Store Management Use case specifications

4.7.3.3 Store Management Sequence Diagram:

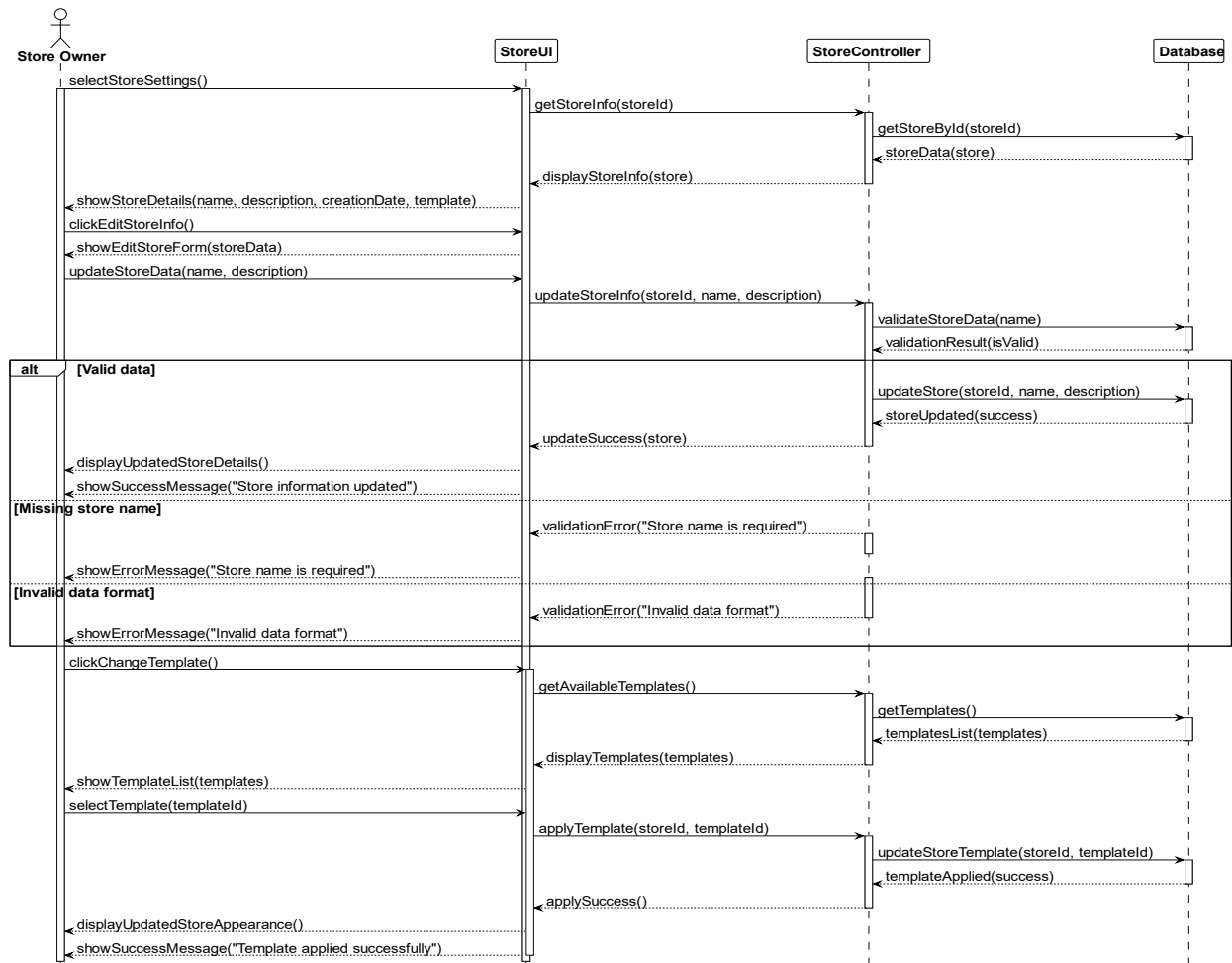


Figure 4.10 Store Management Sequence Diagram

4.7.4 Product Management

4.7.4.1 Product Management Use Case Diagram:

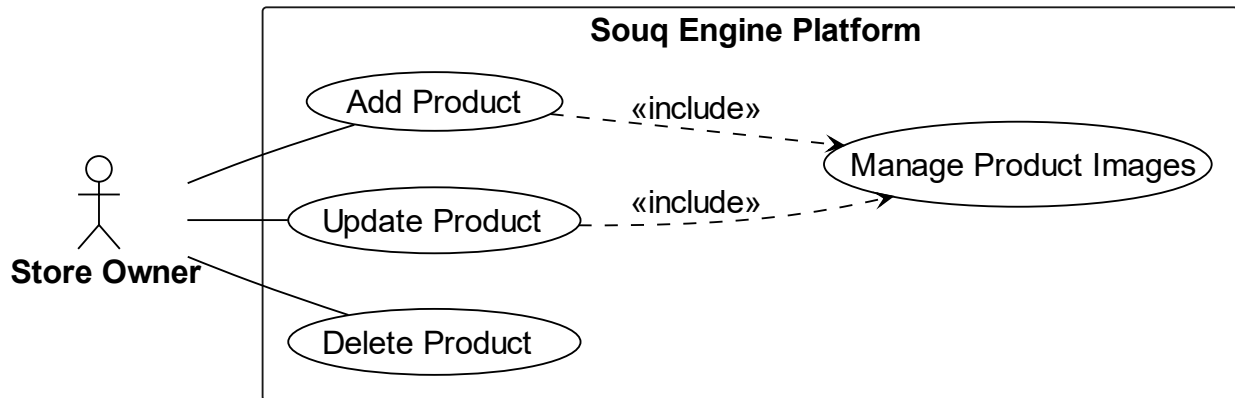


Figure 4.11 Product Management Use Case Diagram

4.7.4.2 Product Management Use case specifications:

Use case id	UC-PM-01
Use case name	Product Management
Actors	Store Owner
Description	This use case allows the Store Owner or an authorized store user to manage products within a specific store. It includes adding new products, updating existing product information, managing product images, updating price and stock quantity, and deleting products.
Preconditions	1. The user is authenticated and authorized to access the system. 2. The user is associated with a specific store. 3. The store exists in the system. 4. The user has permission to manage products within the store.
Main Flow	1. The user requests to manage store products. 2. The system verifies that the user has permission to manage products in the selected store. 3. The system retrieves the products associated with the user's store. 4. The system presents the list of products belonging to the store. 5. The user requests to add a new product, update an existing product, or delete a product. 6. When adding or updating a product, the user provides product data such as name, description, price, stock quantity, and image if available. 7. The system validates the product data and required fields.

	8. The system saves the new or updated product data and links it to the correct store. 9. When deleting a product, the system verifies that the product belongs to the user's store, then deletes it after confirmation. 10. The system presents the updated product list after applying the changes.
Alternative Flows	A1 – Missing Required Fields (Starts after Step 7 of the Main Flow) 1. The system detects that one or more required fields are missing. 2. The system rejects saving the product data. 3. The system informs the user that the required fields must be completed. 4. The flow returns to Step 6 of the Main Flow to correct the data. A2 – Unauthorized Product Access (Starts after Step 2 or Step 9 of the Main Flow) 1. The user attempts to update or delete a product that does not belong to their store. 2. The system verifies the store associated with the product. 3. The system detects that the product does not belong to the user's store. 4. The system denies the operation, and the use case ends without updating or deleting the product. A3 – Product Image Upload Failure (Starts during Step 6 of the Main Flow) 1. The user provides an invalid or unsupported product image. 2. The system rejects the image due to file type, size, or upload failure. 3. The system informs the user that a valid image must be provided. 4. The flow returns to Step 6 of the Main Flow to correct the product data or image.
Postconditions	The product information is successfully created, updated, or deleted in the system.

Table 4.8 Product Management Use case specifications

4.7.4.3 Product Management Activity Diagram:

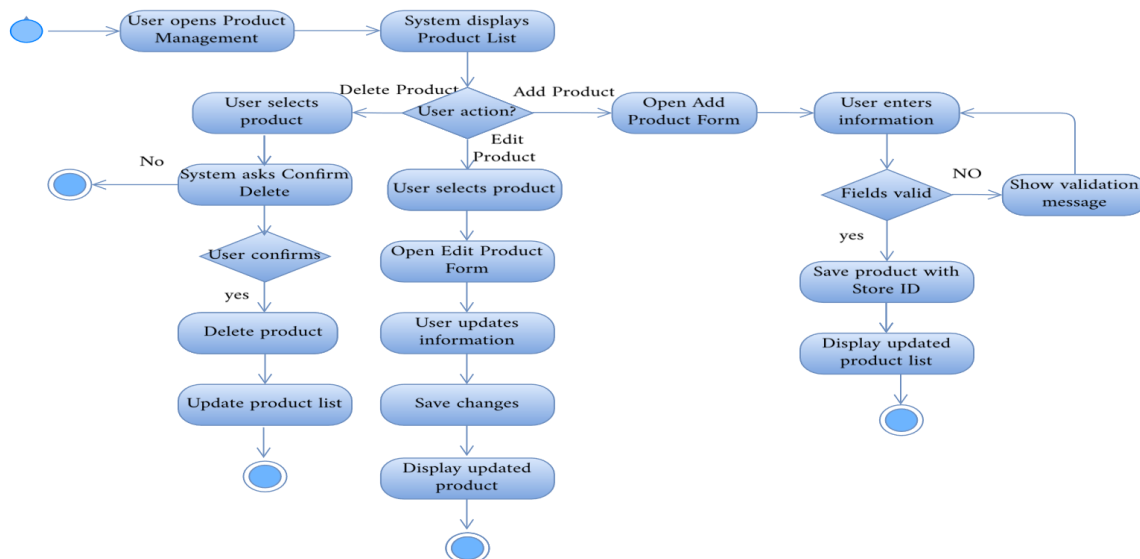


Figure 4.12 Product Management Activity Diagram

4.7.4.4 Product Management Sequence Diagram:

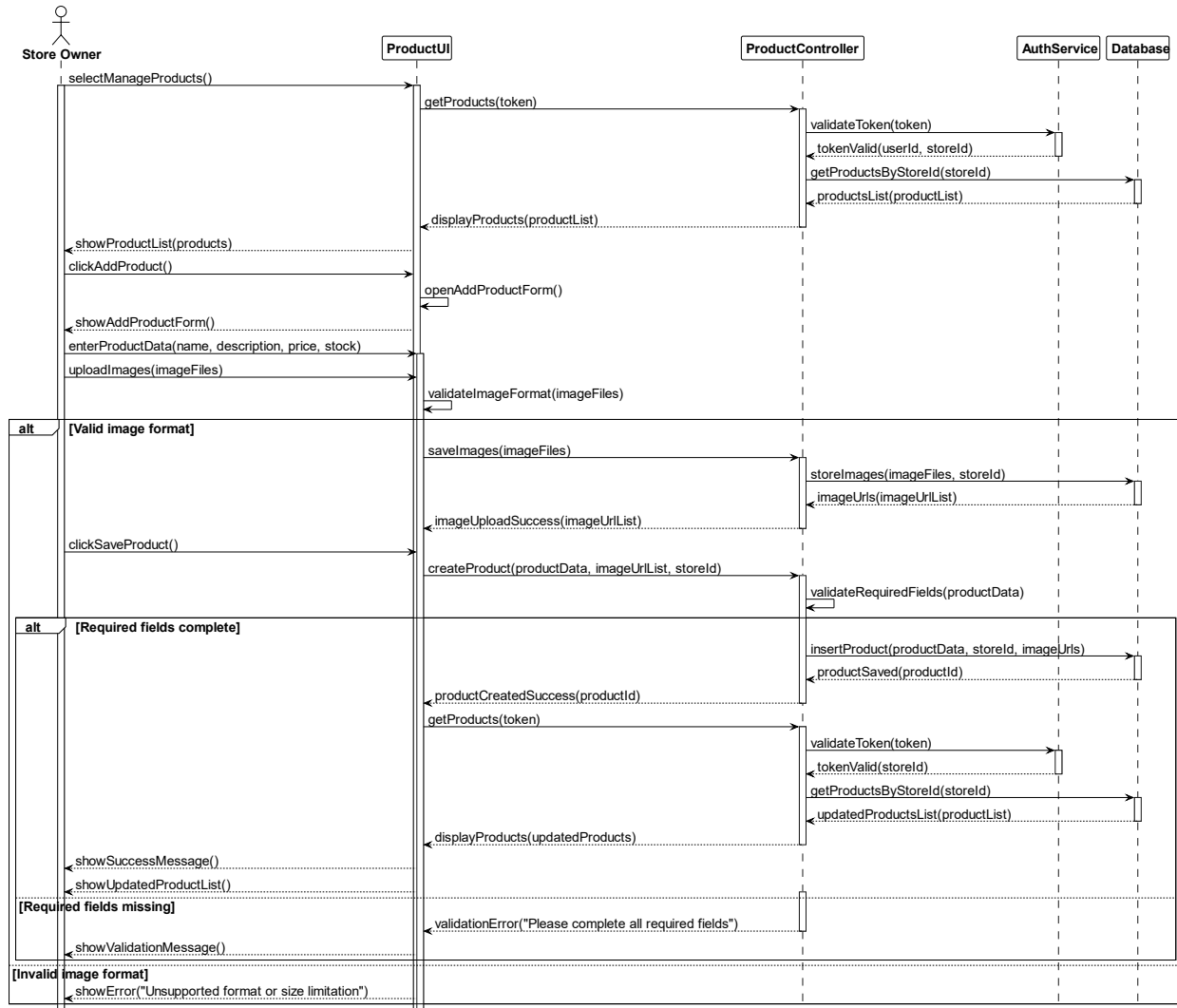


Figure 4.13 Product Management Sequence Diagram

4.7.5 Category Management

4.7.5 .1 Category Management Use Case Diagram:

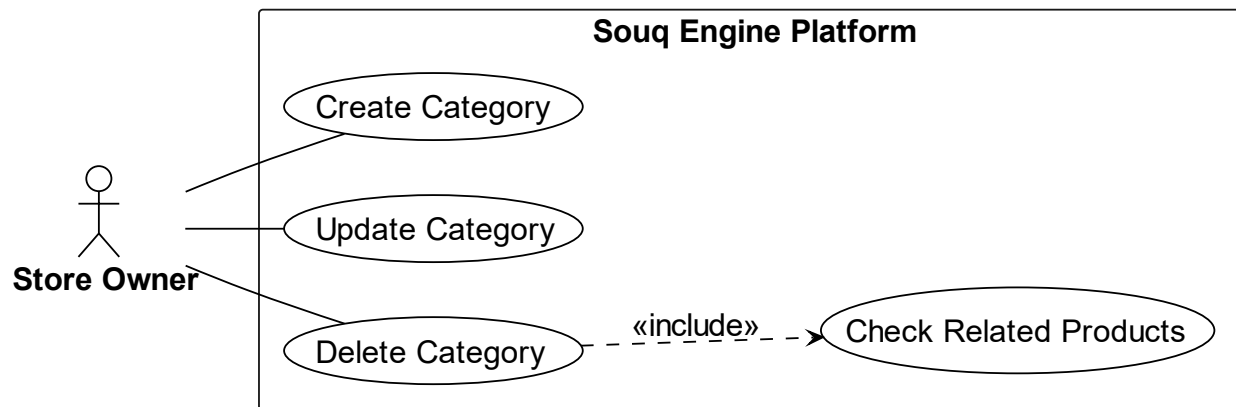


Figure 4.14 Category Management Use Case Diagram

4.7.5.2 Category Management Use case specifications:

Use case id	UC-CM-02
Use case name	Category Management
Actors	Store Owner
Description	This use case allows the Store Owner to manage product categories within their store. It includes creating new categories, updating existing category information, and deleting categories in order to organize products clearly.
Preconditions	The user is authenticated and authorized to access the system. The user is associated with a specific store. The store exists in the system. The user has permission to manage products within the store.
Main Flow	1. The Store Owner requests to manage product categories within the store. 2. The system verifies that the Store Owner has permission to manage categories in the selected store. 3. The system retrieves the categories associated with the Store Owner's store. 4. The system presents the list of categories belonging to the store. 5. The Store Owner requests to create a new category, update an existing category, or delete a category. 6. When creating or updating a category, the Store Owner provides category data such as the category name and optional description. 7. The system validates the category data and required fields.

	8. The system saves the new or updated category data and links it to the correct store. 9. When deleting a category, the system verifies that the category belongs to the Store Owner's store and checks whether it has related products. 10. The system deletes the category if it is allowed to be deleted. 11. The system presents the updated category list after applying the changes.
Alternative Flows	<u>A1 – Missing Required Category Name</u> (Starts after Step 7 of the Main Flow) 1. The system detects that the category name is missing. 2. The system rejects saving the category data. 3. The system informs the Store Owner that the category name is required. 4. The flow returns to Step 6 of the Main Flow to correct the data. <u>A2 – Category Contains Products</u> (Starts after Step 9 of the Main Flow) 1. The system detects that the category is linked to one or more products. 2. The system rejects deleting the category. 3. The system informs the Store Owner that the products must be reassigned or removed before deleting the category. 4. The flow returns to Step 4 of the Main Flow. <u>A3 – Unauthorized Category Access</u> (Starts after Step 2 or Step 9 of the Main Flow) 1. The system detects that the category does not belong to the current Store Owner's store. 2. The system denies the operation. 3. The use case ends without creating, updating, or deleting the category.
Postconditions	The category is created, updated, or deleted successfully if the data is valid and the Store Owner is authorized. The category data remains linked to the correct store. No changes are saved if validation fails or access is unauthorized.

Table 4.9 Category Management Use case specifications

4.7.5.3 Category Management Sequence Diagram:

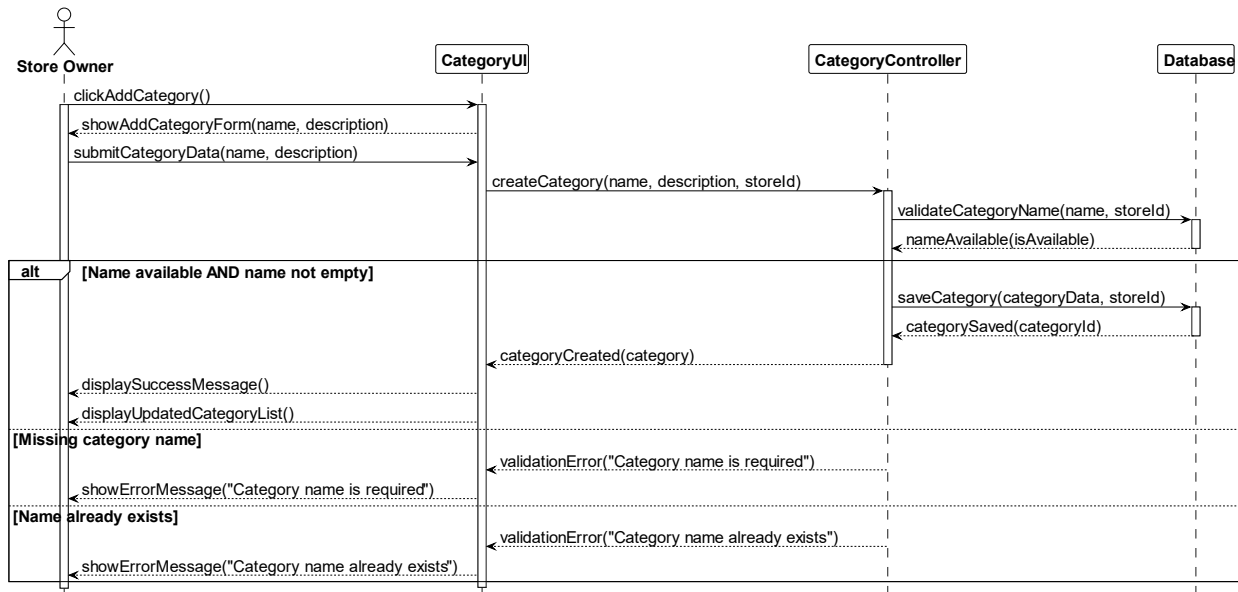


Figure 4.15 Category Management Sequence Diagram

4.7.6 Order Management

4.7.6.1 Order Management Use Case Diagram:

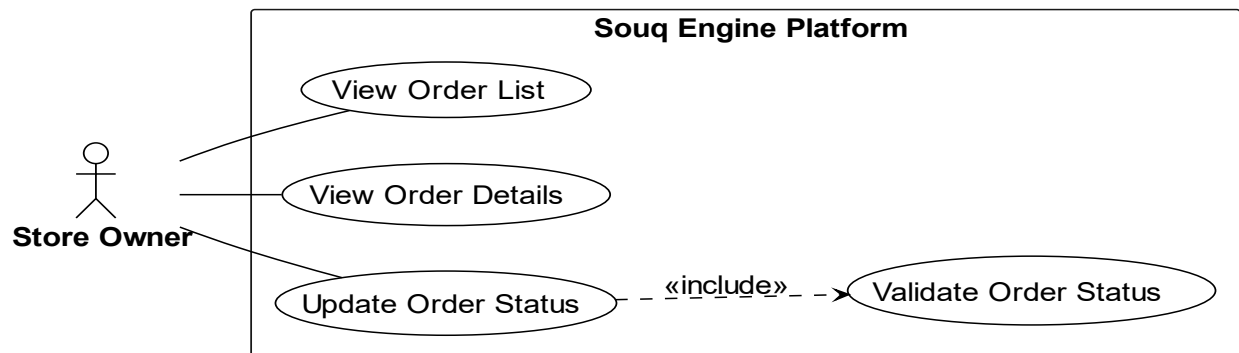


Figure 4.16 Order Management Use Case Diagram

4.7.6.2 Order Management Use case specifications:

Use case id	UC-OM-01
Use case name	Order Management
Actors	Store Owner
Description	This use case allows the Store Owner to manage customer orders that belong to their store, including viewing the order list, viewing order details, and updating the order status.
Preconditions	The Store Owner is authenticated and authorized to manage the store orders.

	The store exists in the system. The store belongs to the current Store Owner.
Main Flow	1. The Store Owner requests to manage store orders. 2. The system verifies that the Store Owner has permission to access the selected store orders. 3. The system retrieves the orders associated with the Store Owner's store. 4. The system presents the list of store orders. 5. The Store Owner requests to view details of a specific order. 6. The system presents the order details, including ordered products, quantities, total price, customer information, and order date. 7. The Store Owner updates the status of an order. 8. The system validates the requested status update. 9. The system saves the updated order status. 10. The system confirms that the order status has been updated successfully.
Alternative Flows	<u>A1 – No Orders Available</u> (Starts after Step 3 of the Main Flow) 1. The system detects that there are no orders associated with the store. 2. The system presents an empty order list with a suitable message. 3. The use case ends without updating any order. <u>A2 – Cancel Order Status Update</u> (Starts after Step 7 of the Main Flow) 1. The Store Owner cancels the status update before confirmation. 2. The system discards the requested change. 3. The order keeps its previous status. 4. The flow returns to Step 4 of the Main Flow. <u>A3 – Unauthorized Order Access</u> (Starts after Step 2 or Step 5 of the Main Flow) 1. The system detects that the selected order does not belong to the Store Owner's store. 2. The system denies access to the order information. 3. The use case ends without displaying or updating the order.
Postconditions	The order information is displayed successfully when the Store Owner is authorized. The order status is updated only if the requested change is valid. No changes are saved if the update is cancelled or unauthorized.

Table 4.10 Order Management Use case specifications

4.7.6.3 Order Management Activity Diagram

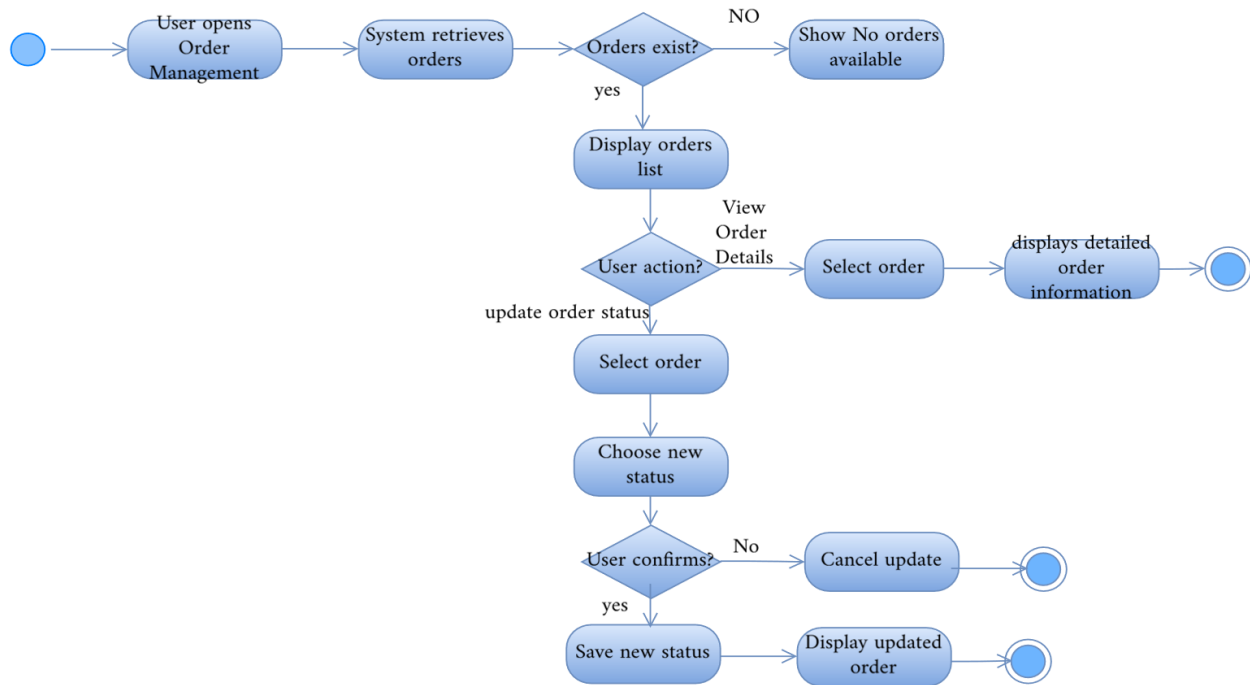


Figure 4.17 Order Management Activity Diagram

4.7.6.4 Order Management Sequence Diagram:

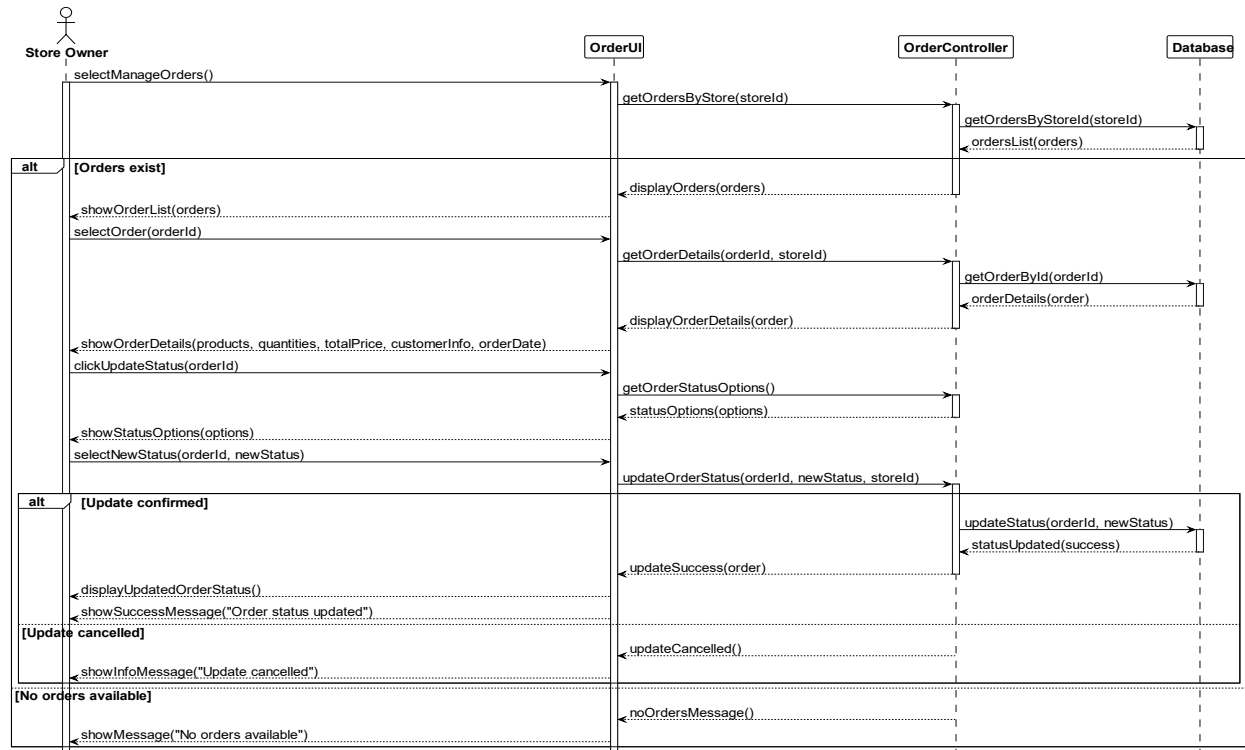


Figure 4.18 Order Management Sequence Diagram

4.7.7 Customer Store Browsing

4.7.7.1 Customer Store Browsing Use case specifications:

Use case id	UC-CUST-01
Use case name	Customer Store Browsing
Actors	Store Visitor / store user
Description	The system allows visitors to browse a specific store, view products, check product details, and add products to a virtual shopping cart for preview purposes.
Preconditions	The store exists and is in Active state. (Optional) The user may be logged in for additional features like wishlist, but browsing is public.
Main Flow	The visitor opens the store page. The system displays the list of available products with name, price, and short description. If the visitor selects a product: 3.1. The system displays detailed product information including images, full description, and specifications. If the visitor wants to add a product to the mock cart: 4.1. The visitor selects a product and clicks “Add to Mock Cart”. 4.1. The system adds the product to the virtual cart. 4.3. The visitor can view the mock cart contents, adjust quantities, or remove items.
Alternative Flows	<u>A1 – No Products Available:</u> The system detects that the store has no products. The system displays a message indicating that no products are available. <u>A3 – Mock Cart Actions</u> The visitor tries to add a product that is already in the mock cart. The system increases the quantity of the product in the cart instead of duplicating entries. The visitor can also remove or adjust quantities without affecting real inventory.
Postconditions	The visitor can view all available products and their details The visitor can simulate adding products to the mock cart without affecting real data.

Table 4.11 Customer Store Browsing Use case specifications

4.7.7.2 Customer Store Browsing Activity Diagram

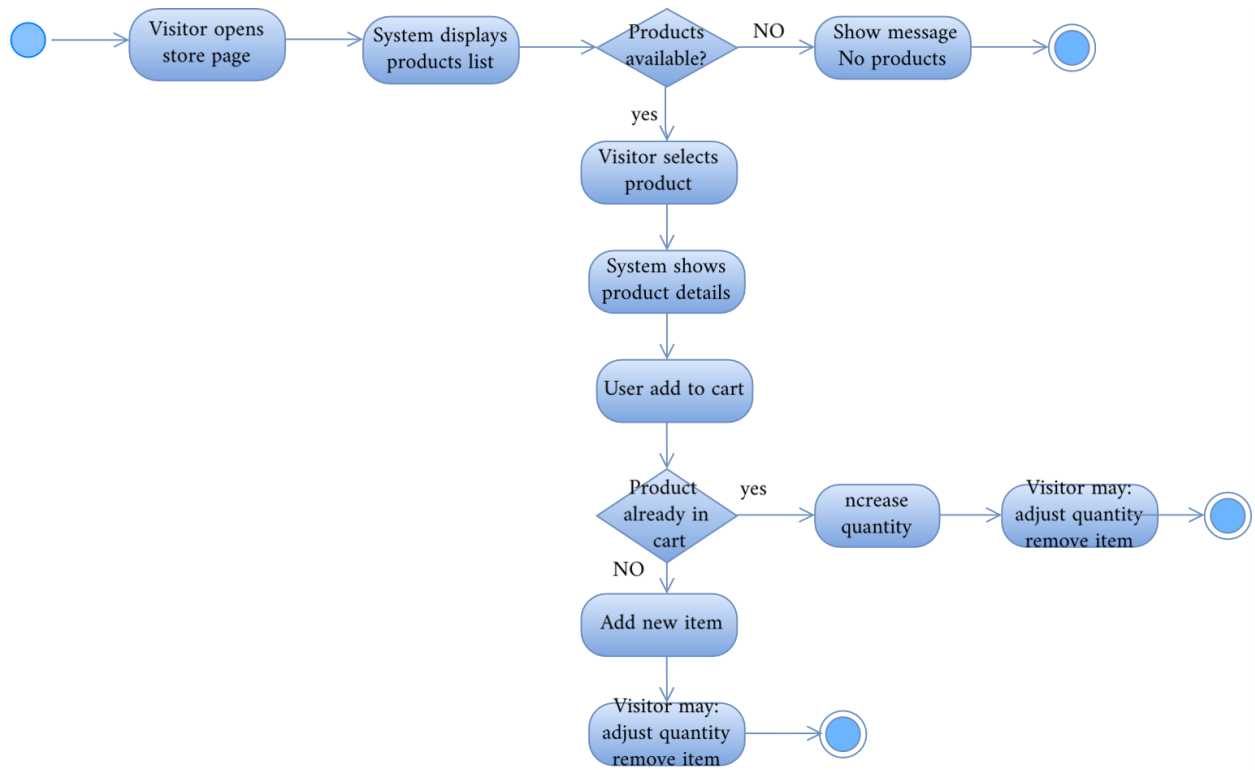


Figure 4.19 Customer Store Browsing Activity Diagram

4.8 initial test cases Definition

Test Case Scenario		SCE-01:AI-Assisted Store Creation		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-01	Check results of generating store settings using AI with a sufficient description.	1. Launch the Create Store with AI interface. 2. Enter a store description. 3. Press “Generate Store with AI”. 4. The system sends the description to the AI service. 5. The system receives the generated results.	A store that sells tech products and educational toys for children aged 3 to 12, including smart devices and programming toys, with a blue and red design, named Child Store.	The system displays generated store data including: • Suggested store name • Store description • Product categories • Sample products • Suitable store template. The user can review and confirm store creation.
TC-02	Check results of generating store settings when the description is insufficient.	1. Launch the Create Store with AI interface. 2. Enter a short description. 3. Press “Generate Store with AI”. 4. The system sends the description to the AI service. 5. The AI analyzes the description.	A store that sells tech products and educational toys for children aged 3 to 12 years.	The AI asks additional multiple-choice questions (MCQs) to collect missing information. After the user selects suitable options, the AI completes the analysis and sends the final store settings to the system.
TC-03	Check results when the AI service connection fails during store generation.	1. Launch the Create Store with AI interface. 2. Enter a store description. 3. Press “Generate Store with AI”. 4. The system sends the request to the AI service. 5. The AI service connection fails.	A store that sells tech products and educational toys for children aged 3 to 12, including smart devices and programming toys, with a blue and red design, named Child Store.	Error message “AI service unavailable” is displayed. The description remains saved in the input field and no store is created.

Table 4.12 AI-Assisted Store Creation initial TC

Test Case Scenario		SCE-02: Tenant Data Isolation		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-04	Check results of displaying data belonging only to the user's store.	1. Launch the store dashboard. 2. Open the product list page. 3. Open the order list page.	User: user_a@storeA.com Store: Store_A (Contains products X, Y, Z)	1. The product list displays only products X, Y, Z . 2. No products from other stores are shown. 3. The order list displays only orders belonging to Store_A .
TC-05	Check results when attempting to access data belonging to another store through API.	1. Launch the product management interface. 2. Capture a product request using an API testing tool. 3. Modify the Product ID to one belonging to another store. 4. Send the request.	Product ID from Store_B: PROD-456	Error message "Access Denied" or HTTP 403 response is returned and the data is not displayed.

Table 4.13 Tenant Data Isolation initial TC

Test Case Scenario		SCE-03: Product Management		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-06	Check the results of adding a new product with valid information.	1. Launch the product management interface. 2. Choose "Add product" . 3. Enter product data. 4. Press "Save" .	Product Name: Smart Toy Robot Description: Educational robot for children Price: 35 Stock: 20 Image: robot.jpg	Product added successfully and displayed in the product list.

TC-07	Check the results of adding a product when required fields are missing.	1. Launch the product management interface. 2. Choose "Add product" . 3. Enter incomplete data. 4. Press "Save"	Product Name: (empty) Description: Educational toy Price: 25 Stock: 10	Error message "Please complete all required fields."
TC-08	Check results of editing an existing product.	1. Launch the product management interface. 2. Choose a product and press "Edit" . 3. Modify product data. 4. Press "Save" .	New Price: 40 New Stock: 15	Product information updated successfully.
TC-09	Check results of deleting a product.	1. Launch the product management interface. 2. Choose a product and press "Delete" . 3. Confirm deletion.	Product ID: PROD-123	Product deleted successfully from the product list.
TC-10	Check results of uploading an image for a product.	1. Launch the product management interface. 2. Choose "Add product" . 3. Enter product data. 4. Upload product image. 5. Press "Save" .	Image file: product image.png	Image uploaded successfully and linked to the product.

Table 4.14 Product Management initial TC

Test Case Scenario		SCE-04: Category Management		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-11	Check results of adding a new category with valid data.	1. Launch the Category Management interface. 2. Choose "Add Category" . 3. Enter category name and description. 4. Press "Save" .	Category Name: Electronics Description: Devices and electronic products.	The category is saved successfully and appears in the category list.
TC-12	Check results of adding a new category with missing required name.	1. Launch the Category Management interface. 2. Choose "Add Category" . 3. Leave the category name empty. 4. Press "Save" .	Category Name: (Empty) Description: Devices and electronic products.	Error message "Category name is required" is displayed and the category is not saved.
TC-13	Check results of editing an existing category with valid data.	1. Launch the Category Management interface. 2. Select an existing category. 3. Choose "Edit" . 4. Modify category name or description. 5. Press "Save" .	Existing Category: Electronics New Name: Electronic Devices	The category information is updated successfully and the updated category appears in the list.

TC-14	Check results of editing a category with missing required name.	1. Launch the Category Management interface. 2. Select an existing category. 3. Choose "Edit" . 4. Remove the category name. 5. Press "Save" .	Category: Accessories (No products assigned)	1. The system displays a validation message indicating that the category name is required . 2. The category is not updated . 3. The user remains on the edit page until a valid name is entered.
TC-15	Check results of deleting a category that has no products assigned.	1. Launch the Category Management interface. 2. Select a category. 3. Press "Delete" . 4. Confirm the deletion.	Image file: product-image.png	1. The category is successfully deleted. 2. The category is removed from the category list. 3. A success message is displayed to the user.

Table 4.15 Category Management initial TC

Test Case Scenario		SCE-05: Store Management		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-16	Check results of updating store information with valid data.	1. Launch the Store Management interface. 2. Choose "Edit Store Information" . 3. Update store name and description. 4. Press "Save" .	Description: Online store for electronics and gadgets.	The store information is updated successfully and the new data is displayed.
TC-17	Check results of updating store information with missing store name.	1. Launch the Store Management interface. 2. Choose "Edit Store Information" . 3. Remove the store name. 4. Press "Save" .	Store Name: (Empty) Description: Online store for electronics.	Error message "Store name is required" is displayed and the changes are not saved.
TC-18	Check results of updating store information with invalid data format.	1. Launch the Store Management interface. 2. Choose "Edit Store Information" . 3. Enter invalid data in one of the fields. 4. Press "Save" .	Store Name: ###@@@	Error message indicating invalid data format is displayed and the store information is not updated.
TC-19	Check results of changing the store template successfully.	1. Launch the Store Management interface. 2. Choose "Change Store Template" . 3. Select a new template. 4. Press "Apply" .	Selected Template: Modern Blue Template	The selected template is applied successfully and the store appearance is updated.

TC-20	Check results of displaying store information.	1. Launch the Store Management interface.	Existing Store: Tech World	The system displays store details including store name, description, creation date, and template information.
-------	--	--	----------------------------	---

Table 4.]16 Store Management initial TC

Test Case Scenario		SCE-06: Order Management		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-21	Check results of displaying the list of orders for the store.	1. Launch the Order Management interface. 2. Navigate to the orders list page.	Store: Tech Store Orders: ORD-101, ORD-102	The system displays the list of orders belonging to the store.
TC-22	Check results of displaying order details.	1. Launch the Order Management interface. 2. Select an order from the order list. 3. Open the order details page.	Order ID: ORD-101	The system displays detailed order information including products, quantities, total price, customer information, and order date.
TC-23	Check results of updating the order status successfully.	1. Launch the Order Management interface. 2. Select an order from the order list. 3. Choose "Update Status" . 4. Select a new order status. 5. Press "Save" .	Order ID: ORD-101 New Status: Processing	The system updates the order status successfully and displays the updated order information.
TC-24	Check results when no orders are available for the store.	1. Launch the Order Management interface.	Store ID with no orders	The system displays a message indicating that no orders are available and the orders list remains empty.
TC-25	Check results when canceling the order status update process.	1. Launch the Order Management interface. 2. Select an order. 3. Choose "Update Status" . 4. Select a new status. 5. Press "Cancel" .	Order ID: ORD-101 Status: Processing	The system discards the changes and keeps the previous order status unchanged.

Table 4.17 Order Management initial TC

Test Case Scenario		SCE-07: Customer Store Browsing		
Test case id	Test case title	Test steps	Test Data	Expected result
TC-26	Check results of displaying the product list as a visitor	1. Open the store page. 2. Browse the homepage.	Store: Tech World (Active state) Products: Product A, Product B, Product C	The system displays the list of available products with name, price, and short description . No login is required.
TC-27	Check results of viewing product details as a visitor	1. Open the store page. 2. Click on a specific product.	Product: Smart Toy Robot Product details: Images, full description, price, specifications	The system displays the detailed product information including images, full description, and specifications. No login is required.
TC-28	Check results of adding a product to the mock cart as a visitor	1. Open the store page. 2. Select a product. 3. Click "Add to Mock Cart" . 4. View the mock cart.	Product: Smart Toy Robot Quantity: 1	1. The product is added to the virtual cart successfully. 2. A success message is displayed. 3. The cart shows the product with correct quantity.
TC-29	Check results of viewing mock cart contents	1. Open the store page. 2. Add multiple products to the mock cart. 3. Open the mock cart page.	Products: Smart Toy Robot (x1), Electronic Game (x2)	The system displays all added products with their quantities, individual prices, and total price .
TC-30	Check results of adjusting product quantity in the mock cart	1. Open the store page. 2. Add a product to the mock cart. 3. Open the mock cart. 4. Increase the quantity of the product.	Product: Smart Toy Robot Initial quantity: 1 New quantity: 3	1. The quantity is updated successfully. 2. The total price is recalculated correctly.

TC-31	Check results of removing a product from the mock cart	1. Open the store page. 2. Add products to the mock cart. 3. Open the mock cart. 4. Remove one product.	Products: Smart Toy Robot, Electronic Game Product to remove: Electronic Game	1. The product is removed from the cart successfully. 2. The remaining products and total price are updated correctly.
TC-32	Check results when adding a product that is already in the mock cart	1. Open the store page. 2. Add a product to the mock cart. 3. Add the same product again.	Product: Smart Toy Robot Initial quantity: 1	1. The system increases the quantity of the existing product instead of duplicating the entry. 2. The cart shows the product with updated quantity (e.g., 2).
TC-33	Check results when no products are available in the store	1. Open the store page.	Store: Empty Store (Active state) Products: None	The system displays a message indicating " No products available " or similar.

Table 4.18 Customer Store Browsing initial TC

4.9 RTM – version 01

Req_ID	Use cases	Title	analysis	System design	Detailed design	coding	initial test cases
FR .01	Auth UC	User Registration	-				
FR .02	Auth UC	User Login	-				
FR .03	Auth UC	User Logout	-				
FR .04	Auth UC / UC-MT-01	Permission Verification	Tenant Data Isolation-analysis				
FR .05	Auth UC / UC-MT-01	User Roles Management	Tenant Data Isolation-analysis				
FR .06	UC-AI-01	Enter Store Description	AI-Assisted Store Creation-analysis				AI-Assisted Store Creation-TC
FR .07	UC-AI-01	Send Description to AI Service	AI-Assisted Store Creation-analysis				AI-Assisted Store Creation-TC
FR .08	UC-AI-01	Generate Structured Store Draft	AI-Assisted Store Creation-analysis				AI-Assisted Store Creation-TC

FR .09	UC-AI-01	Validate AI Output	<u>AI-Assisted Store Creation-analysis</u>				<u>AI-Assisted Store Creation-TC</u>
FR .10	UC-AI-01	Ask Clarification Questions	<u>AI-Assisted Store Creation-analysis</u>				<u>AI-Assisted Store Creation-TC</u>
FR .11	UC-AI-01	Review AI Draft	<u>AI-Assisted Store Creation-analysis</u>				<u>AI-Assisted Store Creation-TC</u>
FR .12	UC-AI-01	Edit or Regenerate Draft	<u>AI-Assisted Store Creation-analysis</u>				<u>AI-Assisted Store Creation-TC</u>
FR .13	UC-AI-01	Approve Draft Before Saving	<u>AI-Assisted Store Creation-analysis</u>				<u>AI-Assisted Store Creation-TC</u>
FR .14	UC-SM-01	Create Store	<u>Store Management-analysis</u>				<u>Store Management-TC</u>
FR .15	UC-SM-01	Assign Store Owner	<u>Store Management-analysis</u>				<u>Store Management-TC</u>

FR .16	UC-MT-01	Tenant Data Isolation	<u>Tenant Data Isolation-analysis</u>				<u>Tenant Data Isolation-TC</u>
FR .17	UC-MT-01	Prevent Cross-Tenant Access	<u>Tenant Data Isolation-analysis</u>				<u>Tenant Data Isolation-TC</u>
FR .18	UC-SM-01	Unique Store Subdomain	<u>Store Management-analysis</u>				<u>Store Management-TC</u>
FR .19	UC-SM-01	View Store Information	<u>Store Management-analysis</u>				<u>Store Management-TC</u>
FR .20	UC-SM-01	Update Store Information	<u>Store Management-analysis</u>				<u>Store Management-TC</u>
FR .21	UC-SM-01	Store Status Management	<u>Store Management-analysis</u>				<u>Store Management-TC</u>
FR .22	UC-SM-01	Publish Store	<u>Store Management-analysis</u>				<u>Store Management-TC</u>
FR .23	UC-CM-02	Create Category	<u>Category Management-analysis</u>				<u>Category Management-TC</u>
FR .24	UC-CM-02	Update and Delete Category	<u>Category Management-analysis</u>				<u>Category Management-TC</u>

FR .25	UC-PM-01	Add Product	Product Management-analysis				Product Management-TC
FR .26	UC-PM-01	Update Product	Product Management-analysis				Product Management-TC
FR .27	UC-PM-01	Delete Product	Product Management-analysis				Product Management-TC
FR .28	UC-PM-01	Link Products and Categories to Store	Product Management-analysis				Product Management-TC
FR .29	UC-PM-01	Inventory Management	Product Management-analysis				Product Management-TC
FR .30	UC-OM-01	View Store Orders	Order Management-analysis				Order Management-TC
FR .31	UC-OM-01	Update Order Status	Order Management-analysis				Order Management-TC
FR .32	UC-OM-01	Store Order Data	Order Management-analysis				Order Management-TC
FR .33	UC-SM-01	Select Store Theme	Store Management-analysis				Store Management-TC

FR .34	UC-SM-01	Customize Store Appearance	Store Management-analysis				Store Management-TC
FR .35	UC-SM-01	Apply Selected Theme	Store Management-analysis				Store Management-TC
FR .36	UC-CUST-01	Browse Store Products	Customer Store Browsing-analysis				Customer Store Browsing-TC
FR .37	UC-CUST-01	View Product Details	Customer Store Browsing-analysis				Customer Store Browsing-TC
FR .38	UC-CUST-01	Browse Products by Category	Customer Store Browsing-analysis				Customer Store Browsing-TC
FR .39	UC-SA-01	Super Admin Dashboard	Customer Store Browsing-analysis				
FR .40	UC-CUST-01	Localization and Basic SEO	Customer Store Browsing-analysis				

Table 4.19 RTM – version 01

Chapter5: System Design

5.1 High-Level Design

5.1.0 Introduction

This section presents the high-level design of the Souq Engine platform. It describes the main system components, the selected architectural style, and how the frontend, backend, database, Redis, and external AI service interact to support the system functionality.

5.1.1 System Block Diagram

A System Block Diagram is a high-level architectural diagram that shows the main system components and how they interact with each other, such as the frontend, backend API layer, internal modules, database, Redis, and external AI service. It does not describe code-level details; instead, it presents the overall structure of the system.

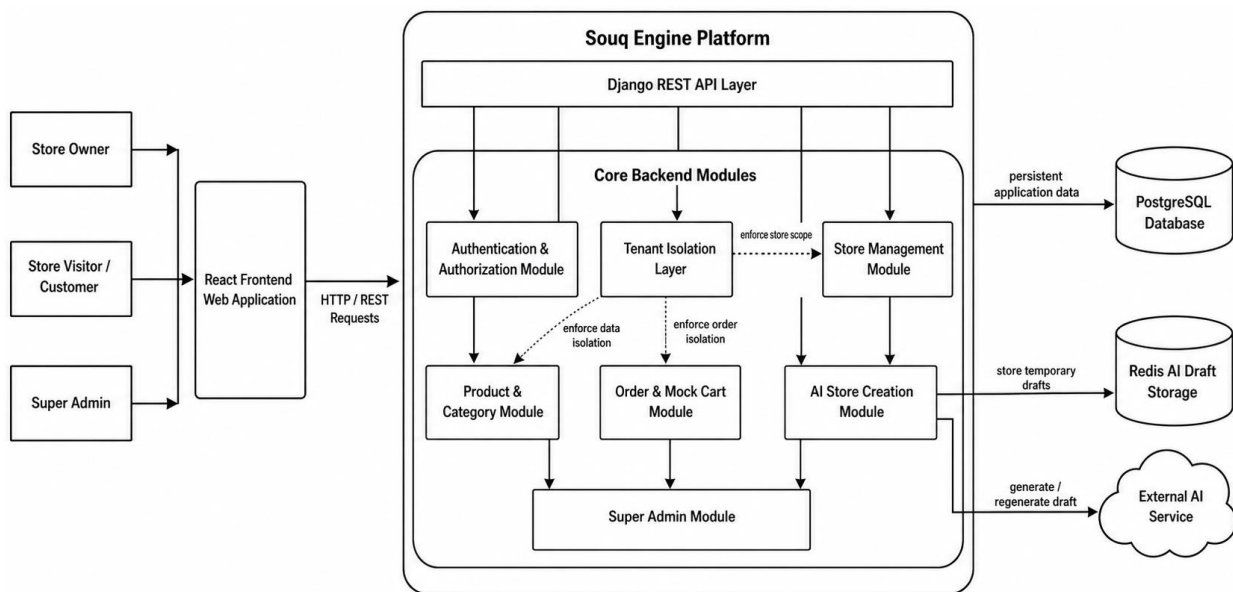
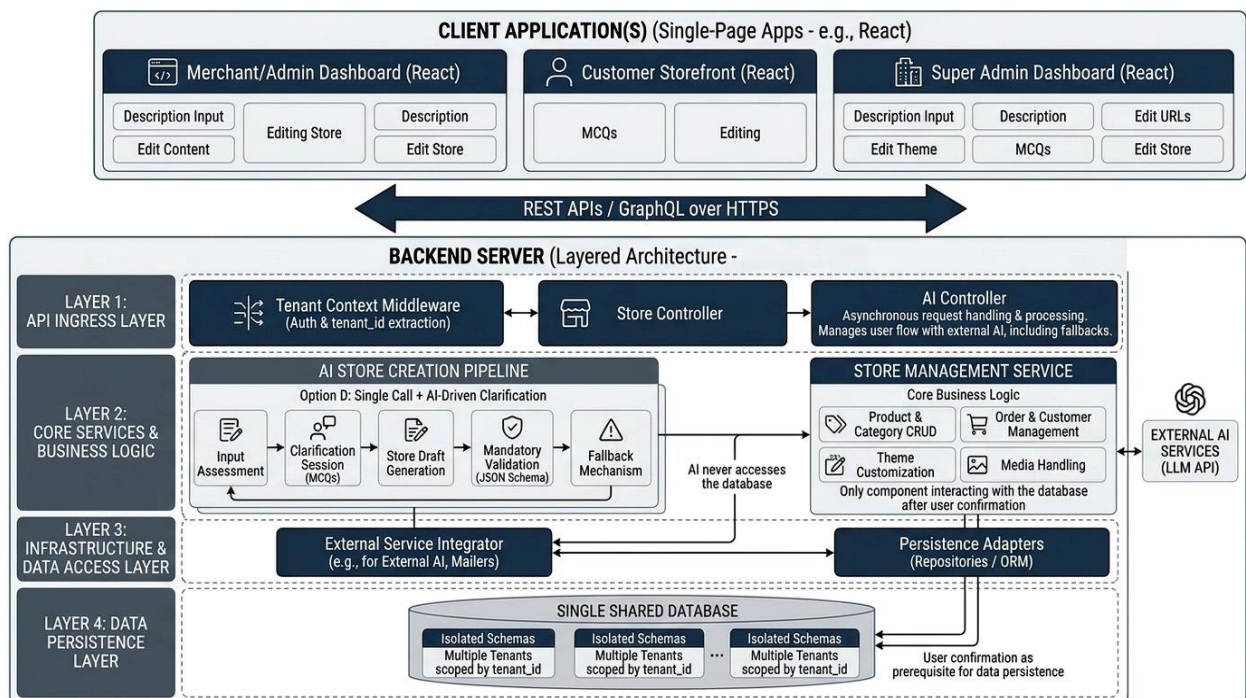


Figure 5.1 System Block Diagram

5.1.2 System Architecture

- The Souq Engine Platform uses a **Layered Architecture** as its main architectural style. This style separates the system into clear layers, where each layer has a specific responsibility. The frontend represents the presentation layer, the Django REST API represents the API layer, the core services represent the business logic layer, the repositories and ORM models represent the data access layer, and PostgreSQL represents the data persistence layer.

- This architectural style was selected because it improves maintainability, scalability, security, and tenant data isolation. Each layer is responsible for a specific part of the system, which makes the platform easier to understand, modify, test, and extend. This separation is especially important because Souq Engine is a Multi-Tenant SaaS e-commerce platform, where each store owner must only access and manage data related to their own store.
- In addition to the layered architecture, the system includes an **external AI service-based component** to support AI-assisted store creation. The AI service is not directly connected to the database. Instead, the backend sends store creation requests to the AI service, validates the AI-generated output, stores temporary drafts in Redis, and saves the final approved store data in PostgreSQL only after user confirmation.
- Therefore, the overall architecture can be described as a **Layered Architecture with an External AI Service Integration**. This design keeps the core business logic controlled by the backend while allowing AI features to be added without tightly coupling the system to a specific AI provider.



5.1.3 System Architecture Design

The architecture of **Souq Engine** is divided into two main parts: **Client Applications** and **Backend Server**. The client side represents the user-facing interfaces, while the backend server follows a layered architecture responsible for receiving requests, applying business rules, interacting with data, and integrating with external services.

1. Client Applications:

The client side represents the user interfaces built using **React**. It is the direct interaction point between users and the system. This part includes different interfaces based on user roles:

- **Store Owner Dashboard:** used by the Store Owner to manage store information, products, categories, orders, store settings, and AI-assisted store creation.
- **Customer Storefront:** used by visitors or customers to browse published stores, view products, check product details, and use the mock cart.
- **Super Admin Dashboard:** used by the Super Admin to monitor and manage the platform, stores, and users.

The client applications do not contain the core business logic. Instead, they communicate with the backend server through **REST APIs over HTTPS** and display the returned results to users.

2. Backend Server Layered Architecture:

The backend server is the core part of the system. It is designed using a layered architecture composed of four main layers. This separation helps improve maintainability, scalability, security, and tenant data isolation within the Multi-Tenant SaaS environment.

2.1 API Layer:

The **API Layer** is the entry point for requests coming from the React frontend. It receives HTTP requests through REST APIs and routes each request to the appropriate backend module.

This layer includes components such as:

- **Tenant Context Middleware:** extracts the user and tenant context, such as `tenant_id`, and ensures that each request is processed within the correct store scope.
- **Store Controller:** handles store management requests and forwards them to the related services.
- **AI Controller:** handles AI-assisted store creation requests, including draft generation, clarification questions, and failure handling.

This layer does not contain deep business logic. Its main responsibility is request handling, routing, initial validation, authentication, authorization, and response formatting.

2.2 Core Business Logic Layer:

The **Core Business Logic Layer** contains the main business rules of the system. It defines what should happen when a store is created, a product is updated, an AI draft is generated, or an order status is changed.

This layer includes several core services and modules:

- **AI Store Creation Pipeline:** analyzes the store description, determines whether clarification questions are needed, generates the store draft, validates AI outputs, and handles fallback cases.
- **Store Management Service:** manages store information, settings, templates, publishing, and store ownership rules.
- **Product & Category Management:** handles creating, updating, deleting, and organizing products and categories within the correct store scope.

- **Order & Customer Management:** manages customer orders and order status updates for each store.
- **Theme Customization:** manages the selected store theme and appearance-related settings.
- **Media Handling:** manages images and files related to stores or products.

This layer is also responsible for applying validation rules, preventing invalid data from being saved, and ensuring that all business operations respect tenant isolation.

2.3 Data Access Layer:

The **Data Access Layer** acts as a bridge between the business logic layer and the database. Its role is to organize database operations and prevent the business logic layer from directly handling raw database queries.

This layer may include:

- **Persistence Adapters**
- **Repositories**
- **ORM Models / Query Layer**

Through this layer, the system retrieves store data, saves products, updates order, checks ownership, and applies tenant-scoped queries. This separation improves maintainability and testability because data access logic remains organized and separated from business rules.

2.4 Data Persistence Layer:

The **Data Persistence Layer** represents the actual permanent storage of system data. In Souq Engine, the platform uses a **shared PostgreSQL database** to store data for all stores, while applying logical isolation using `tenant_id` and `store_id`.

This layer stores data such as:

- Users and permissions
- Stores and store settings
- Products and categories
- Orders and customer data
- Themes and customization settings
- Important system logs

Because the system is Multi-Tenant, the database is shared, but each store's data is logically isolated. This ensures that one Store Owner cannot access or modify another store's data.

3. AI as a Service-Based Component

Souq Engine treats AI as an external service-based component, not as a model running directly inside the system. The backend sends the store description to an **External AI Service**, and the service returns either a generated store draft or clarification questions when the description is insufficient.

An important design decision is that the AI service does not access the database directly and does not save data by itself. Instead, AI outputs pass through the Core Business Logic Layer, where they are validated using defined rules and schema validation before being presented to the user or saved permanently.

This design provides several benefits:

- Reduces the complexity of running AI models locally.
- Allows changing the AI provider in the future with minimal impact on the rest of the system.
- Protects the database from invalid or incomplete AI-generated outputs.
- Keeps the final save decision under the control of the system and the user.
- Supports fallback handling if the AI service fails or returns invalid results.

4. Redis Draft Storage

In addition to the permanent database, the system uses **Redis** to store AI-generated drafts temporarily.

When the AI service generates a store draft, the draft is not saved immediately as final store data. Instead, it is stored temporarily until the Store Owner reviews it, edits it, regenerates part of it, or approves it.

Redis is suitable for this purpose because AI drafts are temporary and can expire automatically using **TTL**. This prevents unfinished or outdated drafts from accumulating in the main PostgreSQL database.

5.2 AI Store Creation Pipeline Design

5.2.1 AI Pipeline Overview

This section provides an overview of the **AI Store Creation Pipeline** in the **Souq Engine** platform. The pipeline is designed to help the Store Owner generate an initial store draft based on a simple textual description of the store idea. The process starts when the user submits the store description, then the system authenticates the user and extracts the **tenant_id** from the authentication context to ensure tenant data isolation. After that, the request is forwarded to the AI service, which analyzes the input and determines whether it is sufficient to generate a store draft or whether clarification questions are required. The AI-generated output is not saved directly to the database; instead, it is reviewed, validated, and confirmed by the user before the final store is created.

5.2.2 Store Description Submission

The user submits a textual description of the store idea through the system interface. This description may include general information such as the type of products, the target audience, or the overall concept of the store.

After submitting the request, the system authenticates the user and extracts the **tenant_id** from the authentication context to ensure proper data isolation between tenants.

The request is then forwarded to the **AI Processing Service**, which is responsible for generating the initial store draft.

5.2.3 Input Assessment by the AI

The system sends a generation request to the AI service containing the store description provided by the user.

Within the same request, the AI analyzes the description to determine whether the provided information is sufficient to generate an initial store draft.

The AI response may result in one of the following outcomes:

- If the information is **sufficient**, the AI directly generates the store draft.
- If the information is **insufficient**, the AI returns clarification questions to collect the missing information.

5.2.4 Clarification Session (MCQs)

If the AI determines that the description is not sufficient, the system presents a set of **Multiple-Choice Questions (MCQs)** to the user.

These questions are designed to collect the essential information required to generate the store configuration, such as:

- the product category
- the type of products offered
- preferred store layout or template

The user selects the appropriate answers, and the responses are merged with the original store description.

The updated input is then sent again to the same AI generation process.

This clarification cycle continues until sufficient information is available for the AI to generate the store draft.

5.2.5 Store Draft Generation and Interactive Presentation

At this stage, after the initial AI-generated store draft is presented, the user has four interaction options:

- Manual Edit:** The user can directly modify any field (e.g., store name, description, product price).
- Approve & Generate Final Store:** The user confirms the draft as-is, which triggers store creation.
- Partial Field Regeneration**

Partial regeneration allows the user to request AI assistance to regenerate a **specific field** within the store draft while preserving the rest of the configuration.

Unlike manual editing, partial regeneration represents a **separate AI interaction pattern** that triggers a new request to the AI service.

When the user selects the regeneration option for a specific field, the system constructs a new AI request containing the following context:

the full current store draft structure

The AI service is explicitly instructed to **regenerate only the selected field** while maintaining compatibility with the rest of the draft.

After receiving the response, the system performs validation using the **Store Draft Validation Schema** to ensure that:

If the validation succeeds, the regenerated field replaces the previous value in the store draft.

If validation fails, the system rejects the generated value and allows the user to retry regeneration or continue editing manually.

Each partial regeneration request is executed within the **same user session**, ensuring that the AI receives the most recent version of the store draft and maintains continuity throughout the editing process.

d. Full Draft Regeneration

Full regeneration allows the user to request the AI to **generate a completely new store draft**, replacing all generated fields.

This option is useful when the user is not satisfied with the initial AI output and prefers to receive a new alternative configuration.

When full regeneration is requested, the system constructs a new AI request containing:

- the original store description provided by the user
- any clarification answers previously collected
- the current store category or constraints defined during the session

The AI service then generates a **new complete store draft**, including:

Once the new draft is generated, the system performs **mandatory validation** using the Store Draft Validation Schema to verify that all required fields, data types, and structural constraints are satisfied.

If the generated draft passes validation, it replaces the previous draft and is presented to the user for further review.

If validation fails, the system rejects the generated output and redirects the process to **Partial Regeneration or the Fallback Flow**.

Like partial regeneration, full regeneration is executed within the **same user session**, ensuring that the AI generation process maintains the correct tenant context and session continuity.

5.2.6 Validation (Mandatory)

1. Store Metadata Validation

Field	Type	Required	Constraints / Allowed Values
-------	------	----------	------------------------------

store_name	string	✓	Length 3–60 characters
store_description	string	✓	Length 20–500 characters
store_category	enum	✓	Must belong to a predefined list of allowed categories
store_template	string	✓	Must match one of the available system templates
store_language	string	✓	ISO code, e.g., "en", "ar", "es"
store_currency	string	✓	ISO currency code, e.g., "USD", "EUR", "SYP"

2. Product Draft Validation

Field	Type	Required	Constraints / Allowed Values
product_name	string	✓	Length 3–80 characters, must be unique within the store
product_description	string	✓	Length 10–300 characters
price	number	✓	Must be greater than 0
category	string	✓	Must match the store category or its subcategories
stock_quantity	integer	✗ optional	Must be ≥ 0
image_url	string	✗ optional	Must be a valid URL

3. Structural Constraints

- Each store must contain **2–4 AI-generated products**.
- All products must belong to the **store category or its subcategories**.
- Product names must be **unique within the store**.
- All mandatory fields must be **present and of the correct type** before accepting the draft

4. Validation Flow

1. After receiving the AI-generated store draft, verify **all required fields are present**.
2. Check **data types** and constraints (length, allowed values, ranges).
3. If any validation rule fails, reject the draft and route to **Partial Regeneration** or **Fallback Flow**.
4. Only after passing all validation rules is the draft presented to the user for review and editing.

5.2.7 User Confirmation and Store Creation

After reviewing and optionally modifying the generated store draft, the user confirms the final store configuration.

Once the confirmation is received, the system creates the store according to the approved configuration and stores the data in the database.

All stored data is associated with the **tenant_id** to ensure proper tenant isolation.

5.2.8 Fallback Handling

To ensure system availability and prevent workflow interruption, the platform defines explicit fallback conditions and recovery mechanisms when the AI service fails or produces unusable responses.

8.1 Fallback Trigger Conditions

The fallback mechanism is activated when any of the following conditions occur during interaction with the AI service:

- **AI API Error:** If the AI service returns an HTTP error response such as 500, 502, 503, or 429, the request is considered failed.
- **Response Timeout:** If the AI service does not respond within the defined time threshold (10 seconds), the request is considered timed out and is cancelled.
- **Malformed or Unparseable Response:** If the AI response cannot be parsed, contains invalid JSON structure, or misses critical structural elements required by the system.
- **Validation Failure:** If the generated store draft fails to pass the mandatory validation rules defined in the **Store Draft Validation Schema**, such as missing required fields or violating structural constraints.

When any of these conditions occur, the system redirects the request to the fallback flow.

8.2 Fallback Flow

When fallback is triggered, the platform provides alternative options that allow the user to continue the store creation process without relying on the AI service.

These options include:

- providing a **pre-built store template** containing default categories and placeholder products
- allowing the user to **manually configure the store structure** through the interface
- enabling the user to **retry the AI generation process later**

These mechanisms ensure that the store creation process remains functional even when the AI service becomes temporarily unavailable.

5.2.9 AI Pipeline Design Alternatives

Before selecting the final architectural approach for the AI Store Creation Pipeline, multiple design alternatives were evaluated. Each alternative represents a different way of structuring the interaction between the system and the AI service to generate store drafts from user descriptions. The evaluation focused on balancing key concerns: **performance** (response time within 3–5 seconds), **usability** (ability to handle incomplete input), **complexity** (feasibility within project scope), **scalability**, **maintainability**, and **reliability**.

The following four options were considered, ranging from the simplest single-call approach to more complex multi-step and agent-based pipelines. Each option is analyzed in terms of its advantages and disadvantages to determine the most suitable architecture for the system.

Option	Description	Advantages	Disadvantages
--------	-------------	------------	---------------

Option A: Single LLM Call	A single request is sent to the LLM, which directly generates the complete store output (metadata + mock products) based solely on the initial user input. No intermediate steps or interaction loops are involved.	Low latency because it uses one API call. Simple architecture with no orchestration or state management. Easy to implement with minimal backend logic.	Handles incomplete input poorly. No user interaction or clarification. Limited control over output quality because all logic depends on one AI response.
Option B: Multi-Step (Chained Calls)	The pipeline is broken into multiple sequential AI calls. Each step performs a specific task (e.g., extract intent → generate metadata → generate products), and the output of each step feeds into the next.	Better control over output. Clearer separation of responsibilities. Errors can be detected earlier in intermediate steps.	Higher latency due to multiple AI calls. More complex orchestration and error handling. Failure in one step may affect the whole pipeline.
Option C: Agentic Loop	The AI operates as an autonomous agent that iteratively plans, executes, and refines its output. It may perform multiple internal steps, retries, and decision-making loops while maintaining state across interactions.	Highly flexible with complex or incomplete inputs. Supports advanced reasoning and iterative refinement. Suitable for complex workflows.	High complexity due to state management and orchestration. Execution time is unpredictable. Over-engineered for simple store draft generation.
Option D: Single Call + AI-Driven Clarification	A hybrid approach where the AI first evaluates the input. If insufficient, it generates clarification questions. After collecting user responses, a final AI call generates the store draft.	Balanced between simplicity and refinement. Efficient because extra steps happen only when needed. Improves usability by guiding the user. Avoids complex multi-step or agentic pipelines.	Requires logic to manage the Q&A flow. May add slight latency when clarification is needed.

5.2.10 Architectural Evaluation Criteria

To determine the most appropriate AI pipeline design, the alternatives were evaluated based on several architectural criteria derived from the system requirements and quality attributes.

The evaluation considered the following aspects:

Performance: Response time within 3–5 seconds.

Usability: Handles vague input and guides clarification.

Implementation Complexity: Feasible and not over-engineered.

Scalability: Supports concurrent store owners.

Maintainability: Easy to extend and modify.

Reliability: Works despite AI errors or invalid response.

These criteria were used to evaluate the proposed pipeline alternatives and guide the final design decision

AI Pipeline Design Alternatives & Comparison Table:

Criterion / Option	Option A: Single LLM Call	Option B: Multi-Step (Chained Calls)	Option C: Agentic Loop	Option D: Single Call + AI-Driven Clarification
Performance ($\leq 5s$)	High – only one call	Moderate – multiple sequential calls	Low – unpredictable, may exceed target	High – mostly one call, extra only if clarification needed
Usability	Low – no interaction	Medium – some refinement possible	High – adaptive, iterative guidance	High – dynamic clarification improves experience
Implementation Complexity	Very Low – simple	Medium – requires orchestration	High – very complex, heavy state management	Low – controlled complexity, simple orchestration
Scalability	High – stateless, easy to scale	Medium – sequential calls may slow under high load	Low – stateful, complex scaling	High – mostly stateless, only conditional extra calls
Maintainability	High – minimal logic	Moderate – multiple steps	Low – complex logic, harder to maintain	High – clear separation, easy extension
Reliability	Medium – no clarification handling	Medium – sequential error handling	Low – unpredictable outputs	High – validation + fallback integrated

5.2.11 Design Decision for AI

Selected Approach:

Single Call with AI-Driven Conditional Clarification and Interactive Refinement

Justification:

1. Alignment with Performance Requirements

- Avoids unnecessary multiple AI calls
- Maintains response time within **3–5 seconds**
- Only introduces additional interaction when required

2. Improved Usability

- Users are not forced to provide all data upfront
- AI dynamically guides the user through missing information
- Interactive editing + partial regeneration enhances flexibility

3. Controlled Complexity

- Avoids over-engineering (no agentic loops or heavy orchestration)
- Keeps implementation feasible within project scope

4. Scalability Support

AI service remains **stateless**

Supports concurrent tenant requests via horizontal scaling

5. Maintainability

Clear separation:

- AI logic
- validation
- business logic

Easy to extend (e.g., adding new fields or generation logic)

6. Robustness and Reliability

Validation ensures safe persistence of AI output

Fallback mechanism guarantees system availability even if AI fails

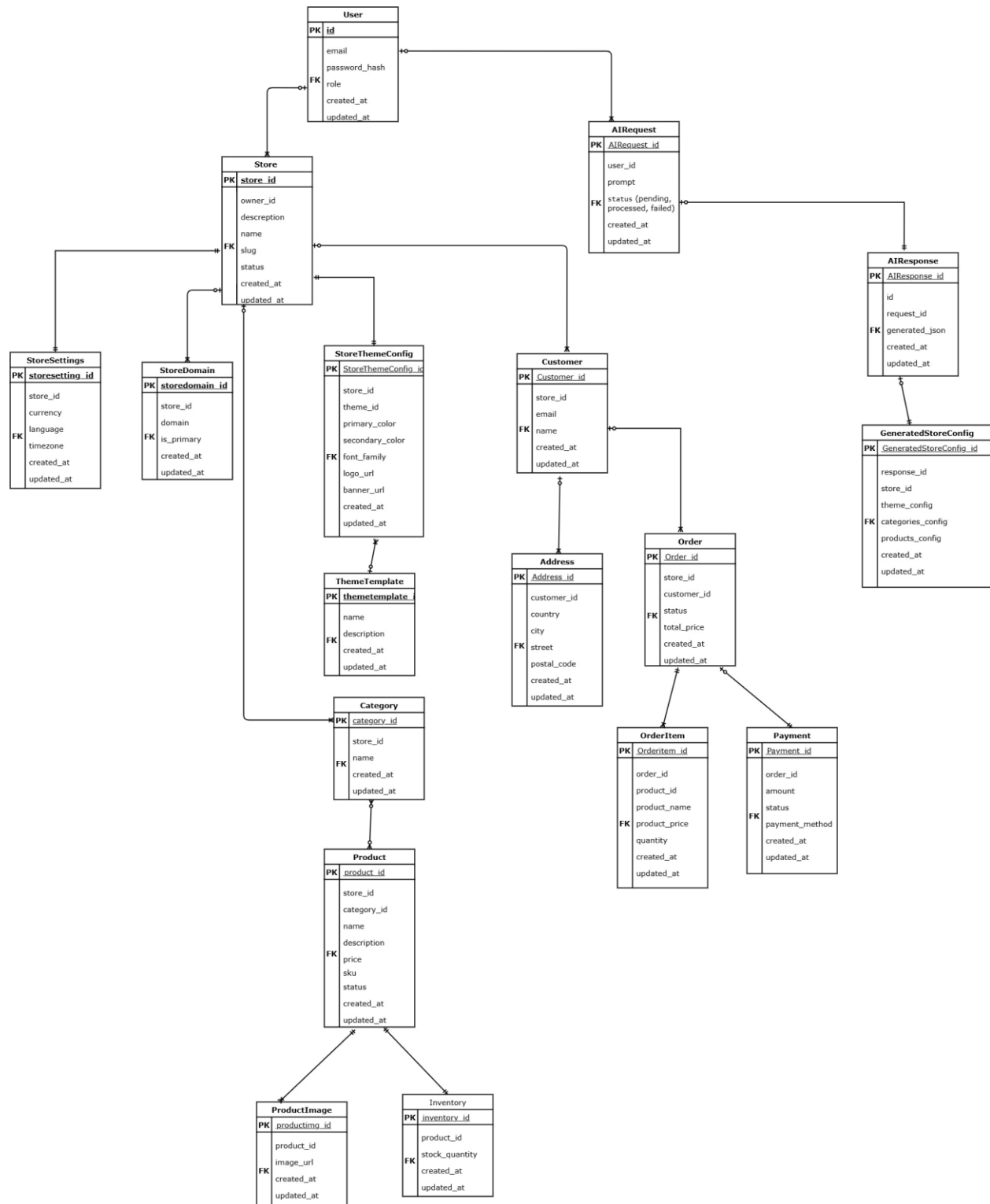
This design intentionally avoids complex AI orchestration (e.g., agentic loops) and instead adopts a controlled, interaction-driven approach that satisfies system requirements while maintaining simplicity, performance, and extensibility.

This approach was selected because the system primarily generates a relatively small amount of structured content (store metadata and a limited number of mock products). Therefore, a lightweight interaction-driven pipeline provides sufficient flexibility without introducing unnecessary architectural complexity.

5.3 Database Design

5.3.1 ER Diagram

5.3.2 Relational Schema



The relational schema defines the database tables, attributes, primary keys, and foreign keys used to implement the entities shown in the ERD. The schema is based on the main domains of the system: platform management, store customization, product catalog, orders, billing, and AI integration. Each table represents a system entity, while foreign keys define the relationships between these entities. Since Souq Engine is a multi-tenant e-commerce platform, most store-related tables include `store_id` to ensure that data is linked to the correct store and logically isolated from other stores.

1. Platform Tables

Table	Main Attributes	Keys / Relationships
User	user_id, email, password_hash, role, created_at, updated_at	user_id is the Primary Key. A user can own one or more stores.
Store	store_id, owner_id, name, description, slug, status, created_at, updated_at	store_id is the Primary Key. owner_id is a Foreign Key referencing User(user_id).
StoreSettings	storesetting_id, store_id, currency, language, timezone, created_at, updated_at	storesetting_id is the Primary Key. store_id is a Foreign Key referencing Store(store_id).
StoreDomain	storedomain_id, store_id, domain, is_primary, created_at, updated_at	storedomain_id is the Primary Key. store_id is a Foreign Key referencing Store(store_id).

2. Store Customization Tables

Table	Main Attributes	Keys / Relationships
ThemeTemplate	themetemplate_id, name, description, created_at, updated_at	themetemplate_id is the Primary Key.
StoreThemeConfig	storethemeconfig_id, store_id, theme_id, primary_color, secondary_color, font_family, logo_url, banner_url, created_at, updated_at	storethemeconfig_id is the Primary Key. store_id references Store(store_id). theme_id references ThemeTemplate(themetemplate_id).

3. Catalog Domain Tables

Table	Main Attributes	Keys / Relationships
Category	category_id, store_id, name, created_at, updated_at	category_id is the Primary Key. store_id references Store(store_id).
Product	product_id, store_id, category_id, name, description, price, sku,	product_id is the Primary Key. store_id references Store(store_id). category_id references Category(category_id).

	status, created_at, updated_at	
ProductImage	productimage_id, product_id, image_url, created_at, updated_at	productimage_id is the Primary Key. product_id references Product (product_id).
Inventory	inventory_id, product_id, stock_quantity, created_at, updated_at	inventory_id is the Primary Key. product_id references Product (product_id).

4. Order Domain Tables

Table	Main Attributes	Keys / Relationships
Customer	customer_id, store_id, email, name, created_at, updated_at	customer_id is the Primary Key. store_id references Store (store_id).
Address	address_id, customer_id, country, city, street, postal_code, created_at, updated_at	address_id is the Primary Key. customer_id references Customer (customer_id).
Order	order_id, store_id, customer_id, status, total_price, created_at, updated_at	order_id is the Primary Key. store_id references Store (store_id). customer_id references Customer (customer_id).
OrderItem	orderitem_id, order_id, product_id, product_name, product_price, quantity, created_at, updated_at	orderitem_id is the Primary Key. order_id references Order (order_id). product_id references Product (product_id).

5. Billing Domain (Mock Billing)

Table	Main Attributes	Keys / Relationships
Payment	payment_id, order_id, amount, status, payment_method, created_at, updated_at	payment_id is the Primary Key. order_id references Order (order_id).

6. AI Integration Domain

Table	Main Attributes	Keys / Relationships
AIRquest	airequest_id, user_id, prompt, status, created_at, updated_at	airequest_id is the Primary Key. user_id references User (user_id).

AIResponse	airesponse_id, request_id, generated_json, created_at, updated_at	airesponse_id is the Primary Key. request_id references AIRequest(airequest_id).
GeneratedStoreConfig	generatedstoreconfig_id, response_id, store_id, theme_config, categories_config, products_config, created_at, updated_at	generatedstoreconfig_id is the Primary Key. response_id references AIResponse(airesponse_id). store_id references Store(store_id).

5.3.3 Database Constraints and Keys

The Souq Engine database uses constraints and keys to ensure data integrity, consistency, and correct relationships between tables. Each table has a **Primary Key (PK)** that uniquely identifies each record, such as **id** in the User, Store, Product and Order tables.

The database also uses **Foreign Keys (FKs)** to connect related tables. For example, **owner_id** in the Store table references the User table, which means each store belongs to a specific user. Similarly, **store_id** in tables such as Product, Category, Customer, Order, StoreSettings, and StoreDomain connects these records to the correct store. This supports multi-tenant data isolation because each store-related record is linked to its own store.

Additional constraints are used to prevent invalid or duplicated data. For example, the store `slug` should be unique for each store, the product `sku` should be unique within the same store, product `price` should be greater than zero, and `stock_quantity` should be greater than or equal to zero. Some fields such as `email`, `store name`, `product name`, and `order status` should not be empty. These constraints help protect the database from inconsistent or incorrect data.

5.3.4 Multi-Tenant Data Isolation

Souq Engine uses a shared database model with logical data isolation between stores. Since the platform supports multiple stores, each store-related entity is linked to a specific store using **store_id**. This ensures that the data of one store is separated from the data of other stores inside the same database.

The Store entity represents the tenant context in the system. Tables such as **Category**, **Product**, **Customer**, **Order**, **StoreSettings**, **StoreDomain**, and **StoreThemeConfig** include **store_id** to ensure that each record belongs to the correct store. For example, when a Store Owner requests the product list, the backend retrieves only products where **store_id** matches the current store context.

This isolation is also enforced at the application layer. The backend extracts the authenticated user and store context, then applies tenant-scoped queries before accessing or modifying data. As a result, one Store Owner cannot access, update, or delete data that belongs to another store.

Example:

```
SELECT *
FROM Product
WHERE store_id = current_store_id;
```

This approach allows the platform to use one shared PostgreSQL database while maintaining secure and organized data separation between tenants.

5.3.5 AI Generated Store Data Structure

The AI-generated store data structure represents the initial store configuration returned by the AI service. This configuration is generated as a structured JSON object and is later processed by the system to create the related database entities, such as the store, theme configuration, categories, and products.

The generated configuration may include basic store information such as **store_name** and **store_description**, **theme settings** such as **theme_template**, **primary_color**, **secondary_color**, and **font_family**, in addition to initial categories and products. This data is stored through the AI integration entities, mainly **AIRequest**, **AIResponse**, and **GeneratedStoreConfig**, before being transformed into permanent store records after user confirmation.

Example of Complete AI Generated Configuration

```
{
  "store_name": "Tech World",
  "store_description": "Modern electronics and gadgets store",
  "theme_config": {
    "theme_template": "modern",
    "primary_color": "#1E3A8A",
    "secondary_color": "#F59E0B",
    "font_family": "Inter"
  },
  "categories_config": [
    {"name": "Smartphones"},
    {"name": "Laptops"},
    {"name": "Accessories"}
  ],
  "products_config": [
    {
      "name": "iPhone 14",
      "description": "Latest Apple smartphone",
      "price": 999,
      "category": "Smartphones"
    }
  ]
}
```

This design allows the system to keep AI-generated outputs organized, traceable, and separated from final approved store data until the Store Owner reviews and confirms the generated configuration.

5.3.6 Database Design Summary

The database design of Souq Engine provides a structured relational model that supports multi-tenant store management, product catalog organization, customer ordering, store customization, mock billing, and AI-assisted store creation. The use of primary keys, foreign keys, and store-based isolation ensures data integrity and prevents data mixing between stores. This design forms a solid foundation for implementing the backend services and APIs of the platform.

5.4 API Design

5.4.1 API Design Overview

This section describes the API design of the **Souq Engine** platform. The system uses **RESTful APIs** with **Django REST Framework** to connect the frontend with the backend services. These APIs support the main system functions, including authentication, store management, products, categories, orders, themes, Super Admin operations, and AI-assisted store creation.

The backend also provides API documentation using **Swagger** and **Redoc**. Protected APIs rely on **JWT authentication**, where the system extracts the authenticated user and **tenant_id** from each request to enforce tenant-based data isolation.

5.4.2 API Routing Structure

The API routes in the **Souq Engine** platform are organized by functional modules. Each module has its own base path and is responsible for a specific part of the system, such as authentication, store management, products, categories, orders, themes, AI store creation, and platform administration. This routing structure makes the backend easier to maintain, extend, and document.

The system uses **/api/** as **the main API** prefix. Requests are then routed to the appropriate application based on the required functionality. In addition, the backend provides API documentation endpoints using Swagger, Redoc, and schema generation.

API Module	Base Path	Purpose
Authentication APIs	/api/auth/	User registration, login, account activation, current user data, and token refresh
Store APIs	/api/stores/	Store creation, update, deletion, settings, domains, subdomain, and publish actions
Category APIs	/api/stores/{store_id}/categories/	Category listing, creation, update, and deletion within a specific store
Product APIs	/api/products/	Product management, product images, inventory, and public product browsing
Order and Cart APIs	/api/stores/{store_id}/orders/ and /api/public/store/{subdomain}/cart/	Store orders, customers, dashboard data, public cart, and checkout

Theme APIs	/api/stores/{store_id}/theme/	Theme templates, store theme configuration, appearance settings, and logo upload
AI Store Creation APIs	/api/ai/	Starting AI draft creation, clarification, draft regeneration, section regeneration, and applying the draft
Platform Admin APIs	/api/admin/	Super Admin dashboard, store management, users, settings, and store status updates
API Documentation	/api/docs/, /api/redoc/, /api/schema/	Swagger, Redoc, and OpenAPI schema documentation

This structure separates public APIs from protected APIs. Public endpoints are used by visitors and customers to browse published stores, view products, manage carts, and create orders. Protected endpoints require authentication and are used by Store Owners or Super Admins to manage platform resources.

5.4.3 API Endpoints

5.4.3.1 API Documentation Endpoints

Endpoint	Method	Purpose
/api/schema/	GET	Generate the OpenAPI schema
/api/docs/	GET	Display Swagger API documentation
/api/redoc/	GET	Display Redoc API documentation

5.4.3.2 Authentication APIs

These endpoints support user registration, login, account activation, current user retrieval, and token refresh.

Endpoint	Method	Purpose
/api/auth/register/	POST	Register a new Store Owner account
/api/auth/login/	POST	Authenticate the user and return JWT tokens
/api/auth/me/	GET	Retrieve the current authenticated user data
/api/auth/activate/{token}/	GET	Activate a user account using an activation token
/api/auth/token/	POST	Obtain JWT access and refresh tokens
/api/auth/token/refresh/	POST	Refresh the JWT access token

5.4.3.3 Store Management APIs

Store APIs handle store creation, updates, settings, domains, subdomains, and publishing actions. Access to protected store operations is checked using tenant and ownership rules.

Endpoint	Method	Purpose
/api/stores/	GET	List stores visible to the authenticated user
/api/stores/	POST	Create a new store

/api/stores/{id}/	PUT / PATCH	Update store information
/api/stores/{id}/delete/	DELETE	Delete a store
/api/stores/{store_id}/settings/	GET / PATCH	Retrieve or update store settings
/api/stores/slug/check/	POST	Check store slug availability
/api/stores/slug/suggest/	POST	Suggest available store slugs
/api/stores/public/store/{subdomain}/	GET	Retrieve public store details by subdomain
/api/stores/{store_id}/subdomain/	PATCH	Set or update the store public subdomain
/api/stores/{store_id}/publish/	PATCH	Publish or unpublish a store
/api/stores/{store_id}/domains/	GET / POST	List or add store domains
/api/stores/{store_id}/domains/{domain_id}/	GET / PUT / PATCH / DELETE	Manage a specific store domain

5.4.3.4 Theme and Appearance APIs

Theme APIs allow the Store Owner to manage theme templates, appearance settings, colors, fonts, logo, and branding configuration.

Endpoint	Method	Purpose
/api/stores/{store_id}/themes/templates/	GET	List available theme templates
/api/stores/{store_id}/theme/	GET	Retrieve store theme configuration
/api/stores/{store_id}/theme/	PATCH	Update store theme configuration
/api/stores/{store_id}/appearance/	GET	Retrieve store appearance settings
/api/stores/{store_id}/appearance/	PUT	Update store appearance settings
/api/stores/{store_id}/assets/logo/	POST	Upload store logo

5.4.3.5 Category APIs

Category APIs manage product categories within a tenant-owned store. Each operation is scoped by `store_id` and tenant context.

Endpoint	Method	Purpose
/api/stores/{store_id}/categories/	GET	List categories for a specific store
/api/stores/{store_id}/categories/	POST	Create a new category
/api/stores/{store_id}/categories/{category_id}/	GET	Retrieve a specific category
/api/stores/{store_id}/categories/{category_id}/	PUT / PATCH	Update a specific category
/api/stores/{store_id}/categories/{category_id}/	DELETE	Delete a specific category

5.4.3.6 Product APIs

Product APIs support product CRUD operations, product images, inventory updates, and public product browsing. Product access is filtered using both `store_id` and `tenant_id`.

Endpoint	Method	Purpose
/api/products/public/store/{subdomain}/products/	GET	List public products for a published store
/api/products/public/store/{subdomain}/products/{product_id}/	GET	Retrieve public product details
/api/products/{store_id}/products/	GET	List products for a store
/api/products/{store_id}/products/	POST	Create a new product
/api/products/{store_id}/products/{product_id}/	GET	Retrieve a specific product
/api/products/{store_id}/products/{product_id}/	PUT / PATCH	Update product information
/api/products/{store_id}/products/{product_id}/	DELETE	Delete a product
/api/products/{store_id}/products/{product_id}/images/	GET / POST	List or add product images
/api/products/{store_id}/products/{product_id}/images/{image_id}/	DELETE	Delete a product image
/api/products/{store_id}/products/{product_id}/inventory/	PATCH	Update product inventory

5.4.3.7 Order, Customer, and Cart APIs

Order APIs include public customer-facing operations such as cart and checkout, as well as protected Store Owner operations such as viewing customers, orders, dashboard statistics, and updating order status.

Endpoint	Method	Purpose
/api/public/store/{subdomain}/orders/	POST	Create a customer order from the public storefront
/api/public/store/{subdomain}/cart/	GET	Retrieve the public cart
/api/public/store/{subdomain}/cart/checkout/	POST	Checkout the cart and create an order
/api/public/store/{subdomain}/cart/items/	GET / POST	List or add cart items
/api/public/store/{subdomain}/cart/items/{product_id}/	PATCH / DELETE	Update or remove a cart item
/api/stores/{store_id}/dashboard/	GET	Retrieve store dashboard statistics
/api/stores/{store_id}/customers/	GET	List store customers

/api/stores/{store_id}/orders/	GET	List store orders
/api/stores/{store_id}/orders/{order_id}/	GET	Retrieve order details
/api/stores/{store_id}/orders/{order_id}/status/	PATCH	Update order status

5.4.3.8 AI Store Creation APIs

AI Store Creation APIs support the intelligent store creation workflow, including draft generation, clarification, full regeneration, section regeneration, and applying the final approved draft.

Endpoint	Method	Purpose
/api/ai/stores/draft/start/	POST	Start the AI store draft workflow
/api/ai/stores/{store_id}/draft/	GET	Retrieve the current AI-generated draft
/api/ai/stores/{store_id}/draft/clarify/	POST	Submit clarification answers
/api/ai/stores/{store_id}/draft/regenerate/	POST	Regenerate the full store draft
/api/ai/stores/{store_id}/draft/regenerate-section/	POST	Regenerate a specific draft section
/api/ai/stores/{store_id}/draft/apply/	POST	Apply the AI draft to create final store data

5.4.3.9 Platform Admin APIs

Platform Admin APIs are restricted to users with the **Super Admin** role. They support platform monitoring, store management, user listing, settings management, and store status updates.

Endpoint	Method	Purpose
/api/admin/dashboard/	GET	Retrieve Super Admin dashboard metrics
/api/admin/settings/	GET / PATCH	Retrieve or update platform settings
/api/admin/stores/	GET	List platform stores
/api/admin/stores/{store_id}/status/	PATCH	Update store status
/api/admin/users/	GET	List platform users

5.4.3.11 API Response and Error Handling

The API design uses clear response codes to describe the result of each request. Public endpoints are accessible without authentication, while protected endpoints require JWT authentication and tenant-aware permission checks. This design improves security, consistency, and maintainability across the backend.

Status Code	Meaning	Example Use
200 OK	Request completed successfully	Retrieving stores, products, orders, or settings
201 Created	Resource created successfully	Registering user, creating store, creating category, creating product
204 No Content	Resource deleted successfully	Deleting store, category, product, or image

400 Bad Request	Invalid request data or validation error	Missing fields, invalid payload, invalid AI draft
401 Unauthorized	Authentication is missing or invalid	Missing or invalid JWT token
403 Forbidden	User is authenticated but not allowed	Tenant mismatch, ownership violation, non-admin accessing admin API
404 Not Found	Resource does not exist or access is denied	Store, product, order, or draft not found

5.5 Security and Permissions Design

5.5.1 Security Design Overview

The security design of **Souq Engine** focuses on protecting user accounts, store data, and tenant-specific resources. The system uses JWT-based authentication, role-based access control, and tenant-based isolation to ensure that each user can access only the resources they are authorized to use.

Protected APIs require authentication, while public APIs are limited to customer-facing actions such as viewing published stores, browsing products, and creating orders. Store management, product management, order management, theme customization, and AI draft operations require authenticated access.

5.5.2 Authentication Mechanism

The system uses **JWT authentication** to verify users. Authentication APIs allow users to register, log in, activate their account, retrieve current user information, and refresh access tokens. After a successful login, the backend returns an access token, refresh token, user role, and **tenant_id**.

For each protected request, **JWTTenantMiddleware** extracts the authenticated user and attaches both **request.user** and **request.tenant_id** to the request. If the token is invalid, the user is inactive, or the tenant context is missing for a non-admin user, the request is rejected.

Security Element	Description
Access Token	Used to authenticate protected API requests
Refresh Token	Used to generate a new access token
tenant_id	Identifies the tenant scope of the authenticated user
request.user	Represents the current authenticated user
request.tenant_id	Used by backend services to apply tenant isolation

5.5.3 User Roles and Permissions

The system supports different access levels depending on the user role. The main roles are **Guest**, **Store Owner**, and **Super Admin**. In the backend user model, authenticated platform roles include **Store Owner** and **Super Admin**, while guests represent unauthenticated public users.

Role	Access Level
Visitor	Can browse published stores and products, manage public cart, and create orders
Store Owner	Can manage only their own store, products, categories, orders, theme settings, and AI drafts
Super Admin	Can access platform administration features such as dashboard, users, stores, settings, and store status management

5.5.4 Permission Enforcement

The backend uses permission classes to enforce access control. `TenantAuthenticated` ensures that the user is authenticated and that the accessed resource belongs to the same tenant. `IsSuperAdmin` restricts platform administration APIs to users with the **Super Admin** role only.

Permission Class	Purpose
<code>TenantAuthenticated</code>	Allows access only to authenticated users within the correct tenant scope
<code>IsSuperAdmin</code>	Allows access only to users with the Super Admin role
<code>AllowAny</code>	Allows public access for customer-facing endpoints

5.5.5 Tenant-Based Data Isolation

Since **Souq Engine** is a Multi-Tenant SaaS platform, tenant isolation is a core security requirement. Store-related data is linked to `tenant_id` and `store_id`, and backend queries filter resources using these values. This prevents one Store Owner from accessing or modifying data that belongs to another store. For example, store, product, category, order, theme, and AI operations all check the tenant context before returning or modifying data. This ensures that every protected operation is executed within the correct store and tenant boundary.

5.5.6 Error Handling for Security

The API returns clear error codes when access is denied or authentication fails.

Status Code	Meaning	Example
401 Unauthorized	Authentication token is missing or invalid	Invalid JWT token
403 Forbidden	User is authenticated but not allowed	Tenant mismatch or insufficient role
404 Not Found	Resource does not exist or is not accessible	Store, product, or draft not found
400 Bad Request	Invalid request data	Invalid payload or validation error

This security design protects user accounts, separates tenant data, restricts admin operations, and ensures that sensitive backend actions are only performed by authorized users.

Chapter6: Implementation Environment and Tools

6.1 Software Tools and Libraries Used

The development of the **Souq Engine** platform used several tools to support coding, testing, database management, and frontend development. **Visual Studio Code** was used as the main code editor. **Postman** and **Swagger/Redoc** were used to test and document the backend APIs. For database management, the project used **PostgreSQL** with **pgAdmin** to inspect and manage database tables and records. The project was managed using **Git/GitHub** for version control. The system was executed locally on **localhost** without Docker. For frontend development, **React** with **Vite** was used to build and run the user interface.

6.2 Development Environment and Project Structure

- The project was developed in a local development environment and divided into two main parts: the backend and the frontend. The backend project folder is named **ai_creation_store**, and it was built using Django and Django REST Framework. The frontend project folder is named **SouqEngineFrontEnd-main**, and it was built using **React** with **Vite**.
- The backend was executed locally on **http://localhost:8000**, while the frontend was executed on **http://localhost:3000**. The backend project is organized into several Django apps, including **users**, **stores**, **categories**, **products**, **orders**, **themes**, **platform_admin**, **seo**, and **AI_Store_Creation_Service**. The project also includes configuration files such as **env**, **requirements.txt**, and **manage.py**.
- The frontend is organized using React project structure, where the user interface is divided into reusable pages, components, and service files. This separation between backend and frontend improves maintainability and makes the system easier to develop, test, and extend.

6.3 Programming Languages and Frameworks

- The backend of the **Souq Engine** platform was implemented using **Python 3.12.2**, **Django**, and **Django REST Framework**. Django was used to build the backend structure and manage the main system modules, while Django REST Framework was used to create RESTful APIs for communication with the frontend. The backend also uses **Simple JWT** for authentication and **drf-spectacular** to generate Swagger and Redoc API documentation.
- The frontend was implemented using **React** with **Vite**. React was used to build the user interface, while Vite was used as the frontend development and build tool. The frontend communicates with the backend through REST API requests.

6.4 Database and Operating Environment

The project uses **PostgreSQL** as the main database system, and **pgAdmin** was used to manage and inspect the database during development. A local `db.sqlite3` file also existed during development for local testing or temporary database use. The backend database configuration was managed through the `.env` file, which stores environment-related settings such as database credentials and API keys.

The project was developed and executed locally on a **Windows** operating system. Both the backend and frontend were run on localhost, without using an external deployment server or Docker environment.

6.5 Dependencies and Packages

- The project uses several backend and frontend dependencies to support development, API creation, authentication, database integration, caching, documentation, testing, and user interface implementation.
- On the backend side, the main dependencies include **Django**, **Django REST Framework**, **Simple JWT**, **drf-spectacular**, **django-cors-headers**, **psycpg2-binary**, **Pillow**, **redis**, and **django-redis**. These packages support REST API development, JWT authentication, PostgreSQL database connectivity, image handling, Redis-based caching, and Swagger/Redoc API documentation.
- On the frontend side, the project uses **React**, **Next.js**, **React DOM**, **Tailwind CSS**, **Radix UI**, **React Query**, **Zustand**, **Zod**, **Lucide React**, and **Recharts**. These libraries support building the user interface, managing frontend state, validating forms and data, handling API requests, creating reusable UI components, and displaying charts or visual dashboard elements.

Side	Main Packages	Purpose
Backend	Django, Django REST Framework	Backend structure and REST API development
Authentication	Simple JWT, PyJWT	JWT-based authentication
Database	psycpg2-binary	PostgreSQL connection
API Documentation	drf-spectacular	Swagger and Redoc documentation
Caching	redis, django-redis	Redis cache and temporary storage
Media Handling	Pillow	Image validation and processing
Frontend	React, Next.js, React DOM	User interface development
UI Components	Radix UI, Tailwind CSS, Lucide React	Interface components, styling, and icons
State/Data Management	React Query, Zustand	API state and frontend state management
Validation	Zod, React Hook Form	Form and data validation
Charts	Recharts	Dashboard charts and visual data

6.6 Version Control

The project used **Git** and **GitHub** for version control during development. Git was used to track code changes, manage project history, and organize updates across the backend and frontend parts of the system. GitHub was used as the remote repository platform to store the project code and keep a backup of the implementation.

The project repositories are divided into two separate repositories:

Repository	GitHub Link
Backend Repository	ai_creation_store
Frontend Repository	SouqEnginFrontEnd

Using version control helped organize the development process, save changes safely, and separate backend and frontend implementation work clearly.

6.7 CI/CD Pipeline

The project applies a **CI/CD pipeline** for the backend using **GitHub Actions**. The workflow is triggered automatically when code is pushed to the `main` or `develop` branches, or when a pull request targets these branches. The purpose of this pipeline is to ensure that backend changes are tested before being accepted into the repository.

The backend workflow runs on **Ubuntu**, sets up **Python 3.12**, installs the required dependencies from `requirements.txt`, starts a **PostgreSQL 15** service for testing, runs database migrations, and executes Django tests with coverage. It also generates coverage reports and uploads them as workflow artifacts.

CI/CD Item	Description
Platform	GitHub Actions
Scope	Backend only
Trigger	Push or Pull Request to <code>main</code> or <code>develop</code>
Environment	Ubuntu latest
Python Version	Python 3.12
Database Service	PostgreSQL 15
Main Steps	Checkout code, install dependencies, run migrations, run tests with coverage
Reports	Coverage XML and HTML report artifacts

6.8 Testing Environment

The project testing environment included both local testing and automated backend testing through GitHub Actions. The backend was tested locally using Django tests, while the APIs were also verified manually using

Postman and **Swagger/Redoc**. The CI workflow runs backend tests automatically on GitHub to ensure that backend changes do not break existing functionality.

The backend tests cover important parts of the system such as authentication, AI store creation workflow, tenant-based access control, products, categories, orders, and platform administration. The project also includes optional live AI provider tests, which are disabled by default and only run when the required environment variables and API keys are provided.

Frontend testing was mainly performed manually through the browser by checking the main user flows, including login, store management, product management, cart usage, and AI-assisted store creation.

Testing Type	Tools / Environment	Purpose
Backend Automated Tests	Django Test Framework, GitHub Actions	Verify backend logic and APIs
API Manual Testing	Postman, Swagger/Redoc	Test API requests and responses
Database Testing	PostgreSQL, pgAdmin	Check stored data and relationships
Frontend Manual Testing	Browser, localhost	Verify user interface flows
AI Workflow Testing	Django tests, optional live provider tests	Test AI draft generation and provider integration

Chapter7: Software Implementation

7.1 Project Files and Structure

The project is divided into two main parts: the **backend** and the **frontend**. The backend contains the Django applications responsible for APIs, business logic, database access, authentication, AI store creation, and administration. The frontend contains the React/Next.js application responsible for the user interface.

7.1.1 Backend Structure

```
ai_store_creation/
├── config/                # Main Django project configuration
├── users/                 # Authentication, users, roles, tenant middleware
├── stores/                # Store management, settings, domains, publishing
├── categories/           # Product category management
├── products/             # Product, inventory, and product image management
├── orders/               # Orders, customers, cart, and dashboard logic
├── themes/               # Theme templates and store appearance
├── AI_Store_Creation_Service/ # AI-assisted store creation workflow
├── platform_admin/       # Super Admin dashboard and management APIs
├── seo/                  # SEO-related functionality
├── utils/                # Shared utilities and middleware
├── .env                  # Environment variables
├── requirements.txt      # Backend dependencies
└── manage.py             # Django management script
```

7.1.2 Frontend Structure

```
SouqEnginFrontEnd-main/
├── src/                  # Main frontend source code
├── public/               # Public static assets
├── tests/                # Frontend tests
├── .next/                # Generated Next.js build folder
├── node_modules/         # Installed frontend packages
├── .env.local             # Frontend environment variables
├── package.json           # Frontend dependencies and scripts
├── package-lock.json      # Locked dependency versions
├── next.config.mjs        # Next.js configuration
├── tsconfig.json          # TypeScript configuration
├── playwright.config.ts   # End-to-end testing configuration
├── vitest.config.ts       # Unit testing configuration
└── README.md              # Frontend project documentation
```

This structure separates backend and frontend responsibilities clearly. The backend is organized into Django apps based on system modules, while the frontend is organized around the user interface source code, static assets, configuration files, and testing files.

7.2 Modules Implementation

This section describes the implementation of the main system modules in the **Souq Engine** platform. The backend is implemented as a set of Django applications, where each application represents a major functional module. This modular structure separates responsibilities and makes the system easier to maintain, test, and extend.

Module	Implementation Description
Users Module	Implements user registration, login, account activation, JWT authentication, current user retrieval, and tenant context handling.
Stores Module	Implements store creation, update, deletion, settings management, domains, subdomains, and publish/unpublish actions.
Categories Module	Implements category listing, creation, update, and deletion within a specific store scope.
Products Module	Implements product CRUD operations, product images, inventory management, and public product browsing.
Orders Module	Implements customer orders, cart operations, checkout, store dashboard statistics, customer listing, and order status updates.
Themes Module	Implements theme templates, store theme configuration, appearance settings, and logo upload.
AI Store Creation Module	Implements the AI-assisted store creation workflow, including draft generation, clarification, draft regeneration, section regeneration, validation, and applying the final draft.
Platform Admin Module	Implements Super Admin features such as dashboard metrics, store management, user listing, platform settings, and store status updates.

Each backend module follows a similar internal structure. The `models.py` file defines the database entities, `serializers.py` defines request and response data formats, `views.py` handles API requests, `services.py` contains business logic, `selectors.py` contains database query logic, and `urls.py` defines the API routes. This organization supports the **layered architecture** used in the backend implementation.

For example, the **AI Store Creation Module** includes endpoints for starting a draft, retrieving the current draft, submitting clarification answers, regenerating the full draft, regenerating a specific section, and

applying the final draft to the store. This module is implemented separately to keep the intelligent workflow isolated from the normal store management logic.

7.3 APIs Implementation

This section explains how the APIs were implemented in the **Souq Engine** platform. The backend APIs were implemented using **Django REST Framework**. Each API module defines its own **urls.py**, **views.py**, **serializers.py**, and service functions. The **urls.py** files define the endpoint paths, the **views.py** files receive HTTP requests, the serializers validate request and response data, and the service layer handles the business logic.

The APIs are organized by system modules such as authentication, stores, categories, products, orders, themes, platform administration, and AI store creation. For example, the AI Store Creation module defines endpoints for starting a draft, retrieving the current draft, submitting clarification answers, regenerating a draft, regenerating a specific section, and applying the final draft.

Code Example: AI Store Creation API URLs

```
urlpatterns = [
    path("stores/draft/start/", AIStartDraftAPIView.as_view(), name="start-draft"),
    path("stores/<int:store_id>/draft/", AICurrentDraftAPIView.as_view(), name="current-draft"),
    path("stores/<int:store_id>/draft/clarify/", AIClarificationAPIView.as_view(), name="clarify-draft"),
    path("stores/<int:store_id>/draft/regenerate/", AIRegenerateDraftAPIView.as_view(),
        name="regeneratedraft"),
    path("stores/<int:store_id>/draft/apply/", AIApplyDraftAPIView.as_view(), name="apply-draft"),
    path("stores/<int:store_id>/draft/regenerate-section/",
        AIRegenerateSectionAPIView.as_view(), name="regenerate-draft-section"),
]
```

Each API request passes through a view class. The view validates the request data using a serializer, checks authentication and permissions, calls the related service function, and returns a structured JSON response. For example, protected APIs use authentication and tenant-aware permission checks to ensure that Store Owners can only access resources that belong to their own stores.

This implementation style keeps the API layer clean and focused on request handling, while business rules are implemented in the service layer and database queries are organized in selector functions.

7.4 Data Access Layer Implementation

- The **Data Access Layer** in Souq Engine is implemented mainly through **selectors.py** files inside the backend modules. These files are responsible for reading data from the database and applying query-level filtering before returning results to the service or API layers.

- This layer helps separate database queries from business logic. Instead of writing database queries directly inside views or services, the system uses selector functions to retrieve stores, products, categories, orders, customers, themes, and AI-related data.
- A major responsibility of the Data Access Layer is enforcing **multi-tenant isolation**. Most queries include both **tenant_id** and **store_id** to ensure that each Store Owner can only access data that belongs to their own store. For example, product queries are filtered by **store_id** and **tenant_id**, and store queries return only stores owned by the authenticated user within the correct tenant scope.

Module	Data Access Responsibility
Stores	Retrieve stores owned by the current user and tenant
Categories	Retrieve categories within a specific store and tenant
Products	Retrieve products, product images, and inventory using tenant-safe queries
Orders	Retrieve orders, customers, dashboard statistics, and recent orders by store and tenant
Themes	Retrieve available theme templates and store theme configuration
AI Store Creation	Retrieve store, categories, products, and theme configuration for the AI workflow

Code Example. Get a single product by ID with tenant/store isolation.*

```
def get_product_by_id(product_id: int, store_id: int, tenant_id: int) -> Product:
    """
    MULTI-TENANT RULE: Verify ownership before returning
    Raises Product.DoesNotExist if not found or unauthorized
    """
    return Product.objects.select_related('category', 'store').prefetch_related('images').get(
        id=product_id,
        store_id=store_id,
        tenant_id=tenant_id
    )
```

7.5 Business Logic Layer Implementation

The **Business Logic Layer** in Souq Engine is implemented mainly inside `services.py` files. This layer contains the core rules of the system, such as validating user permissions, checking tenant ownership, creating products, updating stores, processing orders, managing themes, and handling the AI store creation workflow. The API views do not contain deep business logic; instead, they call service functions after validating the request data. This keeps the code organized and supports the layered architecture.

Module	Business Logic Examples
Stores	Create stores, update store settings, validate ownership, manage subdomains
Products	Create/update/delete products, validate price, SKU, category, tenant ownership

Categories	Create/update/delete categories within the correct store scope
Orders	Create customer orders, manage cart checkout, update order status
Themes	Update store appearance, validate theme configuration, upload logo
AI Store Creation	Generate drafts, process clarification, validate AI output, apply final draft

Code Example: Product Creation Service

```
def create_product(user, store, name, price, sku=None,
description="", category=None, status='active'):
    """ Create a new product with validation and multi-tenant isolation. """
    _validate_store_authorization(user, store)
    if not store.tenant_id:
        raise ValidationError("Store has no valid tenant context")
    if not name or not name.strip():
        raise ValidationError("Product name is required")
    if price <= 0:
        raise ValidationError("Price must be greater than 0")
    if sku and sku.strip():
        sku = sku.strip().upper()
    else:
        sku = _generate_unique_sku(store=store, name=name)
    if Product.objects.filter(store_id=store.id, sku=sku).exists():
        raise ValidationError(f"SKU '{sku}' already exists in this store")
    if category is not None:
        if category.store_id != store.id or category.tenant_id != store.tenant_id:
            raise ValidationError("Category does not belong to this store")
    product = Product.objects.create(
        store=store,
        tenant_id=store.tenant_id,
        name=name.strip(),
        description=description.strip() if description else "",
        price=price,
        sku=sku,
        category=category,
        status=status
    )
    Inventory.objects.create(product=product, stock_quantity=0)
    return product
```

This example shows how business rules are applied before saving data. The service checks authentication, tenant ownership, product name, price, SKU uniqueness, and category ownership. After validation succeeds, the product is created and an inventory record is automatically generated.

7.6 UI Implementation

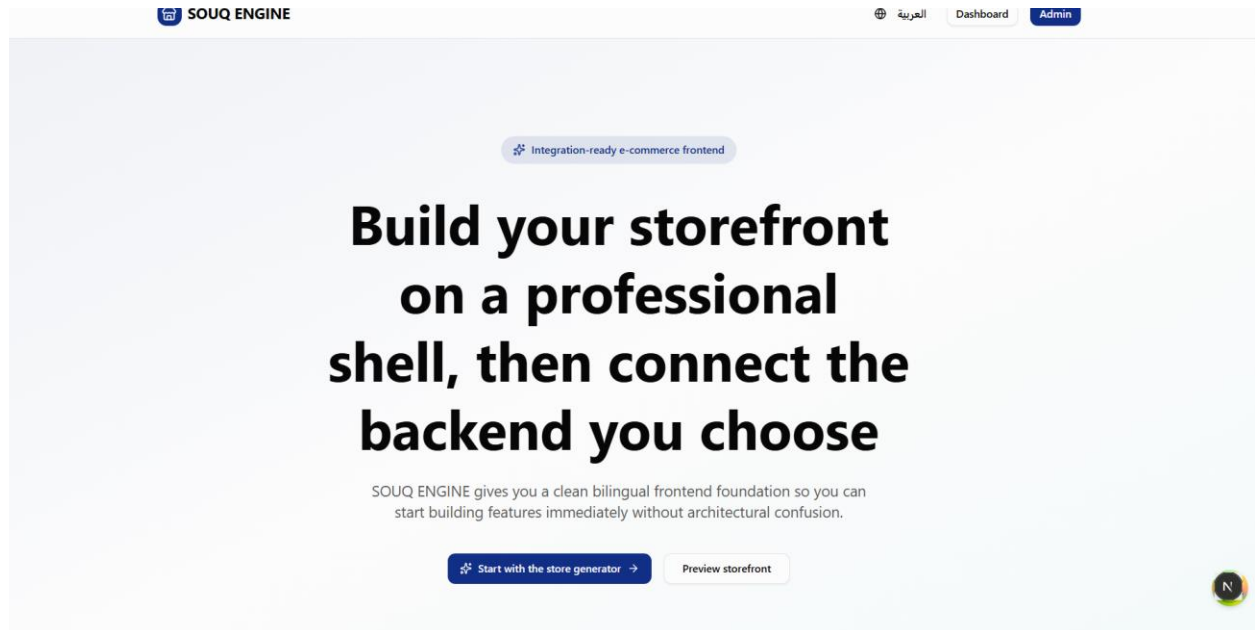


Figure 7.1 Landing Page

7.7 Intelligent Algorithms Implementation

The intelligent part of **Souq Engine** is implemented in the **AI_Store_Creation_Service** module. This module is responsible for generating an initial store draft from a textual store description provided by the Store Owner. The implementation includes prompt construction, AI provider communication, response parsing, draft validation, temporary draft storage, clarification handling, regeneration, and final draft application.

The AI workflow starts when the user submits a store description. The system sends the description to the AI provider and expects a structured JSON response. The response can either contain a complete store draft or clarification questions if the input is not sufficient. The AI prompt enforces strict output rules, such as returning valid JSON only, avoiding system-controlled fields, and generating required sections such as store information, settings, theme, categories, and products.

The system parses the raw AI response and converts it into a Python dictionary. If the response cannot be parsed correctly, the system treats it as a failure and prevents invalid data from being accepted. After parsing,

the draft is validated against the expected schema to make sure all required fields, data types, and constraints are satisfied.

Temporary AI drafts are stored using Redis/cache instead of being saved immediately in the main database. This allows the user to review, edit, clarify, regenerate, or apply the draft before it becomes final store data.

Code Example: Saving AI Draft Temporarily

```
def save_ai_draft(store_id: int, draft_json: dict, ttl_seconds: int | None = None) -> None:
    """Save the full generated draft JSON for a store with TTL."""
    key = build_ai_draft_key(store_id)
    timeout = settings.AI_DRAFT_TTL if ttl_seconds is None else int(ttl_seconds)
    serialized = _serialize_json(draft_json)
    cache.set(key, serialized, timeout=timeout)
```

The AI implementation also supports clarification and regeneration. If the user description is incomplete, the AI returns multiple-choice clarification questions. After the user answers them, the system merges the answers with the original context and continues the generation process. The user can also regenerate the full draft or only a specific section, such as the theme, categories, or products.

Finally, the validated draft can be applied to the store. At this stage, the system creates or updates the related database records, such as store information, categories, products, images, inventory, and theme configuration. This ensures that AI-generated content is not saved permanently until it is validated and approved by the Store Owner.

Chapter8: Testing and Evaluation

8.1 Test Plan

The testing plan for **Souq Engine** was designed to verify that the implemented system works correctly, securely, and reliably. The initial test cases defined during the analysis phase were used as a baseline for testing the main system scenarios, including AI-assisted store creation, tenant data isolation, product management, category management, store management, order management, and customer store browsing.

Testing was performed using both automated and manual methods. Backend functionality was tested using Django automated tests and GitHub Actions. API behavior was verified using Django APIClient, Postman, and Swagger/Redoc. The user interface and main user flows were also checked manually through the browser.

Testing Activity	Description
Automated Backend Testing	Running Django test cases for models, services, selectors, APIs, permissions, and workflows
API Testing	Verifying endpoints, status codes, request validation, and response structures
Security Testing	Checking authentication, authorization, tenant isolation, and role-based access
AI Workflow Testing	Testing AI draft generation, clarification, validation, regeneration, and fallback behavior
Manual UI Testing	Checking main frontend flows through the browser
CI Testing	Running backend tests automatically through GitHub Actions

8.2 Types of Testing

Several testing types were used to evaluate different parts of the system.

Test Type	Description	Example
Unit Testing	Tests individual functions, models, services, and selectors.	Category creation, store selectors, product validation
API Testing	Verifies backend endpoints and returned status codes.	Auth APIs, product APIs, category APIs, order APIs
Integration Testing	Tests the interaction between multiple modules.	Cart checkout, order creation, AI draft workflow
Security Testing	Verifies access control and tenant isolation.	Preventing Store Owner from accessing another store
AI Workflow Testing	Tests the intelligent store creation process.	Draft generation, clarification questions, regeneration

Regression Testing	Ensures that new changes do not break existing features.	GitHub Actions backend test pipeline
Manual Usability Testing	Checks whether main flows are usable through the interface.	Login, dashboard, AI creation, product management

8.3 Test Case Summary

The detailed initial test cases were defined earlier in the analysis phase. In this chapter, they are summarized according to the main tested scenarios.

TC id	Test case title	Tested data	Expected result	Actual result	Pass / Fail
TC-01	Check results of generating store settings using AI with a sufficient description.	Store description: tech products and educational toys for children aged 3–12, blue/red design, name Child Store.	The system displays generated store data including store name, description, categories, sample products, and suitable template.	The system generated and displayed the store draft successfully for user review.	Pass
TC-02	Check results of generating store settings when the description is insufficient.	Short description: store sells tech products and educational toys for children aged 3–12.	The AI asks MCQs to collect missing information, then generates final store settings.	The system requested clarification questions and continued the generation process after answers.	Pass
TC-03	Check results when the AI service connection fails during store generation.	Store description with AI service connection failure.	Error message “AI service unavailable” is displayed, input remains saved, and no store is created.	The system handled the failure and prevented creating an invalid store.	Pass
TC-04	Check results of displaying data belonging only to the user’s store.	User: user_a@storeA.com , Store_A with products X, Y, Z.	Only Store_A products and orders are displayed.	The system displayed only data belonging to the current user’s store.	Pass
TC-05	Check results when attempting to access data belonging to another store through API.	Product ID from another store: PROD-456.	Access denied or HTTP 403 response is returned.	The system denied unauthorized cross-store access.	Pass
TC-06	Check the results of adding a new product	Product Name: Smart Toy Robot, Price: 35, Stock: 20, Image: robot.jpg.	Product added successfully and	Product was added and displayed successfully.	Pass

	with valid information.		displayed in the product list.		
TC-07	Check the results of adding a product when required fields are missing.	Product Name: empty, Description: Educational toy, Price: 25, Stock: 10.	Error message "Please complete all required fields."	The system rejected the incomplete product data and displayed a validation message.	Pass
TC-08	Check results of editing an existing product.	New Price: 40, New Stock: 15.	Product information updated successfully.	Product data was updated successfully.	Pass
TC-09	Check results of deleting a product.	Product ID: PROD-123.	Product deleted successfully from the product list.	Product was deleted successfully.	Pass
TC-10	Check results of uploading an image for a product.	Image file: product image.png.	Image uploaded successfully and linked to the product.	Product image was uploaded and linked successfully.	Pass
TC-11	Check results of adding a new category with valid data.	Category Name: Electronics, Description: Devices and electronic products.	Category is saved successfully and appears in the category list.	Category was created and displayed successfully.	Pass
TC-12	Check results of adding a new category with missing required name.	Category Name: Empty, Description: Devices and electronic products.	Error message "Category name is required" is displayed and category is not saved.	The system rejected the request and displayed a validation message.	Pass
TC-13	Check results of editing an existing category with valid data.	Existing Category: Electronics, New Name: Electronic Devices.	Category information is updated successfully.	Category was updated successfully.	Pass
TC-14	Check results of editing a category with missing required name.	Category: Accessories, name removed.	Validation message is displayed and category is not updated.	The system prevented the update and kept the user on the edit flow.	Pass
TC-15	Check results of deleting a category that has no products assigned.	Category with no products assigned.	Category is deleted, removed from the list, and success message is displayed.	Category was deleted successfully.	Pass
TC-16	Check results of updating store information with valid data.	Description: Online store for electronics and gadgets.	Store information updated successfully and new data displayed.	Store information was updated successfully.	Pass

TC-17	Check results of updating store information with missing store name.	Store Name: Empty, Description: Online store for electronics.	Error message “Store name is required” is displayed and changes are not saved.	The system rejected the invalid update and displayed a validation message.	Pass
TC-18	Check results of updating store information with invalid data format.	Store Name: ###@@@.	Error message indicating invalid data format is displayed and data is not updated.	The system handled the invalid input and prevented unsafe update.	Pass
TC-19	Check results of changing the store template successfully.	Selected Template: Modern Blue Template.	Selected template is applied and store appearance is updated.	Store template was applied successfully.	Pass
TC-20	Check results of displaying store information.	Existing Store: Tech World.	Store details are displayed including name, description, creation date, and template information.	Store information was displayed successfully.	Pass
TC-21	Check results of displaying the list of orders for the store.	Store: Tech Store, Orders: ORD-101, ORD-102.	The system displays orders belonging to the store.	Store orders were displayed successfully.	Pass
TC-22	Check results of displaying order details.	Order ID: ORD-101.	Detailed order information is displayed including products, quantities, total price, customer information, and order date.	Order details were displayed successfully.	Pass
TC-23	Check results of updating the order status successfully.	Order ID: ORD-101, New Status: Processing.	Order status is updated successfully.	Order status was updated successfully.	Pass
TC-24	Check results when no orders are available for the store.	Store ID with no orders.	Message indicating no orders are available is displayed.	The system displayed an empty order list message.	Pass
TC-25	Check results when canceling the order status update process.	Order ID: ORD-101, Status: Processing.	Changes are discarded and previous order status remains unchanged.	The system canceled the update and kept the previous status.	Pass
TC-26	Check results of displaying the product list as a visitor.	Store: Tech World active, Products: Product A, Product B, Product C.	Products are displayed with name, price, and short description without login.	Public product list was displayed without authentication.	Pass

TC-27	Check results of viewing product details as a visitor.	Product: Smart Toy Robot.	Product details are displayed including images, full description, price, and specifications.	Product details were displayed successfully.	Pass
TC-28	Check results of adding a product to the mock cart as a visitor.	Product: Smart Toy Robot, Quantity: 1.	Product is added to cart, success message is displayed, and quantity is correct.	Product was added to the mock cart successfully.	Pass
TC-29	Check results of viewing mock cart contents.	Smart Toy Robot x1, Electronic Game x2.	Cart displays products, quantities, individual prices, and total price.	Cart contents and total price were displayed correctly.	Pass
TC-30	Check results of adjusting product quantity in the mock cart.	Product: Smart Toy Robot, quantity from 1 to 3.	Quantity is updated and total price is recalculated.	Quantity and total price were updated correctly.	Pass
TC-31	Check results of removing a product from the mock cart.	Products: Smart Toy Robot, Electronic Game. Remove Electronic Game.	Product is removed and remaining cart data is updated.	Product was removed and cart totals were updated.	Pass
TC-32	Check results when adding a product that is already in the mock cart.	Product: Smart Toy Robot, Initial quantity: 1.	System increases existing quantity instead of duplicating the item.	Quantity was increased without duplicating the cart item.	Pass
TC-33	Check results when no products are available in the store.	Empty Store, Products: None.	Message “No products available” or similar is displayed.	The system displayed an empty products message.	Pass

These test scenarios cover the main functional requirements of the system and were used to guide both automated and manual testing.

8.4 Test Results and Analysis

The backend test suite was executed successfully. A total of **396 tests** were run, with **0 failed tests** and **1 skipped optional test**. The skipped test is related to optional live AI provider testing, which requires external environment variables and API keys.

Test Result Item	Result
Total tests	396
Passed tests	396
Failed tests	0
Skipped tests	1
Execution time	1281.180 seconds(~20min)
Final status	OK

The results show that all executed backend tests passed successfully. This indicates that the tested backend functionality, including authentication, store management, products, categories, orders, cart, permissions, admin features, and AI workflow, behaved as expected.

8.5 Quality Metrics

The following quality metrics were collected from the automated test execution and coverage report.

Quality Metric	Result
Automated backend tests	396
Passed tests	396
Failed tests	0
Test pass rate	100%
Skipped tests	1
Execution time	1281.180 seconds
Overall code coverage	78%
CI/Test status	Passed

8.2 Final Requirements Traceability Matrix (RTM Version 2)

Req_ID	Use cases	Title	analysis	System design	Detailed design	coding	initial test cases
FR .01	Auth UC	User Registration	-	System Design	Detail design	Source code- backend Source code frontend	-
FR .02	Auth UC	User Login	-	System Design	Detail design	Source code- backend Source code frontend	-
FR .03	Auth UC	User Logout	-	System Design	Detail design	Source code- backend Source code frontend	-
FR .04	Auth UC / UC-MT-01	Permission Verification	Tenant Data Isolation- analysis	System Design	Detail design	Source code- backend Source code frontend	-
FR .05	Auth UC / UC-MT-01	User Roles Management	Tenant Data Isolation- analysis	System Design	Detail design	Source code- backend Source code frontend	-
FR .06	UC-AI-01	Enter Store Description	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .07	UC-AI-01	Send Description to AI Service	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC

FR .08	UC-AI-01	Generate Structured Store Draft	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .09	UC-AI-01	Validate AI Output	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .10	UC-AI-01	Ask Clarification Questions	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .11	UC-AI-01	Review AI Draft	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .12	UC-AI-01	Edit or Regenerate Draft	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .13	UC-AI-01	Approve Draft Before Saving	AI-Assisted Store Creation- analysis	System Design	Detail design	Source code- backend Source code frontend	AI-Assisted Store Creation-TC
FR .14	UC-SM-01	Create Store	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC
FR .15	UC-SM-01	Assign Store Owner	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC

FR .16	UC-MT-01	Tenant Data Isolation	Tenant Data Isolation- analysis	System Design	Detail design	Source code- backend Source code frontend	Tenant Data Isolation-TC
FR .17	UC-MT-01	Prevent Cross-Tenant Access	Tenant Data Isolation- analysis	System Design	Detail design	Source code- backend Source code frontend	Tenant Data Isolation-TC
FR .18	UC-SM-01	Unique Store Subdomain	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC
FR .19	UC-SM-01	View Store Information	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC
FR .20	UC-SM-01	Update Store Information	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC
FR .21	UC-SM-01	Store Status Management	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC
FR .22	UC-SM-01	Publish Store	Store Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Store Management- TC
FR .23	UC-CM-02	Create Category	Category Management- analysis	System Design	Detail design	Source code- backend Source code frontend	Category Management- TC

FR .24	UC-CM-02	Update and Delete Category	Category Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Category Management-TC
FR .25	UC-PM-01	Add Product	Product Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Product Management-TC
FR .26	UC-PM-01	Update Product	Product Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Product Management-TC
FR .27	UC-PM-01	Delete Product	Product Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Product Management-TC
FR .28	UC-PM-01	Link Products and Categories to Store	Product Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Product Management-TC
FR .29	UC-PM-01	Inventory Management	Product Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Product Management-TC
FR .30	UC-OM-01	View Store Orders	Order Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Order Management-TC
FR .31	UC-OM-01	Update Order Status	Order Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Order Management-TC

FR .32	UC-OM-01	Store Order Data	Order Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Order Management-TC
FR .33	UC-SM-01	Select Store Theme	Store Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Store Management-TC
FR .34	UC-SM-01	Customize Store Appearance	Store Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Store Management-TC
FR .35	UC-SM-01	Apply Selected Theme	Store Management-analysis	System Design	Detail design	Source code-backend Source code frontend	Store Management-TC
FR .36	UC-CUST-01	Browse Store Products	Customer Store Browsing-analysis	System Design	Detail design	Source code-backend Source code frontend	Customer Store Browsing-TC
FR .37	UC-CUST-01	View Product Details	Customer Store Browsing-analysis	System Design	Detail design	Source code-backend Source code frontend	Customer Store Browsing-TC
FR .38	UC-CUST-01	Browse Products by Category	Customer Store Browsing-analysis	System Design	Detail design	Source code-backend Source code frontend	Customer Store Browsing-TC
FR .39	UC-SA-01	Super Admin Dashboard	Customer Store Browsing-analysis	System Design	Detail design	Source code-backend Source code frontend	

FR .40	UC-CUST-01	Localization and Basic SEO	Customer Store Browsing-analysis	System Design	Detail design	Source code-backend Source code frontend	
--------	------------	----------------------------	--	-------------------------------	-------------------------------	---	--

Chapter9: Results and Analysis

9.1 System Implementation Results

The implemented **Souq Engine** platform provides a working prototype for AI-assisted e-commerce store creation and management. The system allows Store Owners to register, log in, create stores, manage products and categories, customize store appearance, manage orders, and use the AI-assisted store creation workflow.

The main implemented results include the Store Owner dashboard, customer storefront, Super Admin dashboard, product and category management, cart and checkout flow, theme customization, and AI-generated store draft workflow. The AI feature allows the user to enter a simple store description, generate an initial store draft, answer clarification questions when needed, review or regenerate the draft, and finally approve it before saving it permanently.

9.2 Comparative Analysis with Existing Systems

This section compares **Souq Engine** with two existing e-commerce platforms: **Shopify** and **Salla**. Shopify is a mature global e-commerce platform and provides AI-related tools such as Shopify Magic for generating store content, product descriptions, headings, and other business-related text. Shopify has also introduced AI-powered store building features that can generate store layouts from descriptive keywords.

Salla is a regional SaaS e-commerce platform focused on the MENA market, especially Saudi Arabia and Arabic-speaking merchants. Its strength is its regional localization, Arabic-market focus, and support for local commerce needs.

Feature / Criterion	Souq Engine	Shopify	Salla
AI-assisted store creation	Supported as a core project feature	Supported through Shopify AI tools	May provide AI-related features, but the main focus is regional e-commerce
Text-based store draft generation	Generates store metadata, categories, products, and theme suggestions	Supports AI content and AI store-building features	Not clearly the main workflow
Clarification questions	Supported when user input is insufficient	Not clearly exposed as the standard store creation flow	Not clearly exposed as the standard store creation flow
Multi-Tenant SaaS	Supported by design	Supported as a commercial SaaS platform	Supported as a commercial SaaS platform

Store management	Products, categories, orders, settings, themes	Full commercial store management	Full commercial store management
Customer storefront	Supported in prototype	Supported	Supported
Super Admin dashboard	Implemented for platform management	Internal platform administration is not exposed to merchants	Internal platform administration is not exposed to merchants
Payment integration	Mock billing only in current stage	Real payment integrations	Regional payment integrations
Production readiness	Academic prototype	Mature commercial platform	Mature commercial platform
Main advantage	Transparent AI pipeline with editable generated drafts	Mature global ecosystem	Strong regional and Arabic-market focus
Main limitation	Not production-level yet	Subscription/apps/customization may be required	Less suitable for custom academic AI pipeline experimentation

9.3 Achievement of Objectives

The project objectives defined in Chapter 1 focused on building a Multi-Tenant SaaS e-commerce platform powered by AI. These objectives included generating an initial store draft from a simple textual description, supporting clarification questions, allowing review and regeneration, validating data before saving, isolating tenant data, and providing basic store, product, order, and customer storefront features.

Objective	Achievement Status	Explanation
Build a Multi-Tenant SaaS e-commerce system	Achieved	The system supports multiple stores with tenant/store data isolation.
Generate store draft from text description	Achieved	The AI workflow generates initial store data from user input.
Generate store name, categories, products, and theme	Achieved	The draft includes store metadata, categories, products, and theme configuration.
Support clarification questions	Achieved	The system asks for additional information when the description is insufficient.
Allow review, edit, regeneration, and approval	Achieved	The user can review, edit, regenerate, or apply the generated draft.
Validate data before saving	Achieved	Store and product data are validated before being saved permanently.

Support tenant isolation	Achieved	Store-related data is scoped by tenant/store context.
Provide store management features	Achieved	Store Owners can manage products, categories, settings, themes, and orders.
Provide customer storefront and mock cart	Achieved	Visitors can browse products and use cart/checkout features.
Achieve 3–5 seconds AI response target	Partially Achieved	The design targets this range, but the actual time depends on the external AI provider.
Provide a simple UI for non-technical users	Achieved	The system provides guided dashboards and AI-assisted store creation.

9.4 Strengths and Weaknesses Analysis

Strengths	Explanation
AI-assisted store creation	The system generates an initial store draft from a textual description.
Clarification and regeneration	The workflow supports clarification questions and draft regeneration.
Multi-tenant isolation	Store data is separated using tenant/store context.
Layered backend implementation	APIs, services, selectors, and models are separated clearly.
Automated testing and CI	Backend tests are integrated with GitHub Actions.
Editable AI output	The user can review and approve generated content before saving.
Good backend coverage	The latest coverage result reached 78%.

Weaknesses	Explanation
Prototype stage	The system is not yet a full commercial production platform.
No real payment gateway	Payment is mock/sandbox only in the current scope.
External AI dependency	AI response quality and availability depend on the external provider.
Limited frontend automated testing	Frontend testing was mainly manual.
No production deployment yet	The system was mainly tested locally and through backend CI.
Optional live AI tests require API keys	Live AI provider tests are skipped unless credentials are enabled.

9.5 Key Performance Indicators (KPIs)

The following KPIs summarize the measurable results of the implemented system and the backend testing process. The latest test execution showed that **396 tests** were run successfully with **0 failures**, **1 skipped test**, and **78% overall code coverage**.

KPI	Result
Automated backend tests	396
Passed tests	396
Failed tests	0
Skipped tests	1
Test pass rate	100%
Test execution time	1281.180 seconds
Overall code coverage	78%
Backend CI status	Passed
Main supported roles	Store Owner, Super Admin, Visitor
Main AI feature	AI-assisted store draft generation

These indicators show that the backend test suite completed successfully without failures. The **78% code coverage** also indicates that a large part of the backend implementation is covered by automated tests, while future work can further improve frontend testing and edge-case coverage.

Chapter 10: Conclusion and Recommendations

10.1 Project Summary and Main Achievements

This project presented **Souq Engine**; a Multi-Tenant SaaS e-commerce platform supported by artificial intelligence. The main goal was to help business owners create and manage online stores more easily, especially users who do not have advanced technical knowledge. The system allows the Store Owner to enter a simple textual description, then the AI-assisted workflow generates an initial store draft containing store information, categories, products, and theme suggestions.

The project achieved the main objectives defined at the beginning of the research, including AI-assisted store creation, clarification questions, draft review and regeneration, data validation before saving, tenant-based data isolation, store management, product and category management, order management, customer storefront, and Super Admin features.

The backend was implemented using Django and Django REST Framework, while the frontend was implemented using React/Next.js. The system was also tested through automated backend tests and CI using GitHub Actions. The automated test execution completed successfully with **396 passed tests, 0 failed tests**, and an overall coverage of **78%**.

10.2 Challenges Faced During the Project

During the development of the project, several challenges were encountered. From a technical perspective, one of the main challenges was integrating the AI service in a controlled and safe way. Since AI responses may be incomplete, invalid, or inconsistent, the system needed validation rules, fallback handling, and user confirmation before saving generated data permanently.

Another challenge was implementing **Multi-Tenant data isolation**. The system had to ensure that each Store Owner could only access and manage data that belongs to their own store. This required careful handling of tenant_id, store_id, authentication, permissions, and tenant-scoped database queries.

The project also faced implementation challenges related to connecting the frontend with multiple backend APIs, organizing the backend into modules, managing Redis-based temporary AI drafts, and testing many system flows. In addition, the project was developed under time and budget constraints, and real payment integration, advanced AI features, and production deployment were postponed to future work.

10.3 Future Development

Several improvements can be added in future versions of the project:

Future Improvement	Description
Production Deployment	Deploy the system on a real server or cloud platform with production-level configuration.
Real Payment Integration	Add real payment gateways instead of mock billing.
Advanced AI Features	Add AI-based product description generation, product image analysis, and smarter store recommendations.
Subscription Plans	Add pricing plans and subscription management for Store Owners.
Analytics Dashboard	Provide sales analytics, customer behavior analysis, and product performance reports.
AI Provider Flexibility	Support multiple AI providers and switch between them based on cost, availability, or quality.

10.4 Recommendations

Based on the experience gained from this project, several recommendations can be made. For future students, it is recommended to define the project scope clearly from the beginning and avoid adding too many advanced features before completing the core system. It is also important to design the architecture in a modular way, separate business logic from API handling, and start testing early during development.

For developers working on AI-based systems, it is recommended not to save AI-generated outputs directly without validation. AI results should be parsed, validated, reviewed by the user, and only then stored permanently. This reduces the risk of saving incomplete or incorrect data.

For institutions and future researchers, this project can be extended as a practical foundation for studying AI-assisted e-commerce systems. Further research can focus on improving AI accuracy, measuring user satisfaction, evaluating response time, and comparing different AI providers in terms of quality, cost, and reliability.